

 Devin AI Export - PDF Version

Generated on: 11/4/2025, 10:06:12 PM

Source: Devin AI Platform

Overview

Relevant source files

- [README.md](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/README.md> (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/README.md>))
- [package.json](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json> (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json>))
- [src/index.ts](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts> (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts>))

This document provides a comprehensive introduction to the `viet-stock-bridge` repository, explaining its role as a financial data bridge service that connects internal Tradex systems with Vietnamese stock market APIs. This overview covers the system's purpose, high-level architecture, and key integration points.

For detailed information about specific subsystems, see [System Architecture](#) for architectural components, [External API Integration](#) for VietStock API details, and [Data Management](#) for storage strategies.

Purpose and Scope

The `viet-stock-bridge` is a specialized microservice that acts as a data bridge between Vietnamese stock market APIs (primarily VietStock) and internal Tradex trading infrastructure. The system processes real-time financial data requests through Kafka messaging and provides unified access to stock information, financial statements, company profiles, and market news.

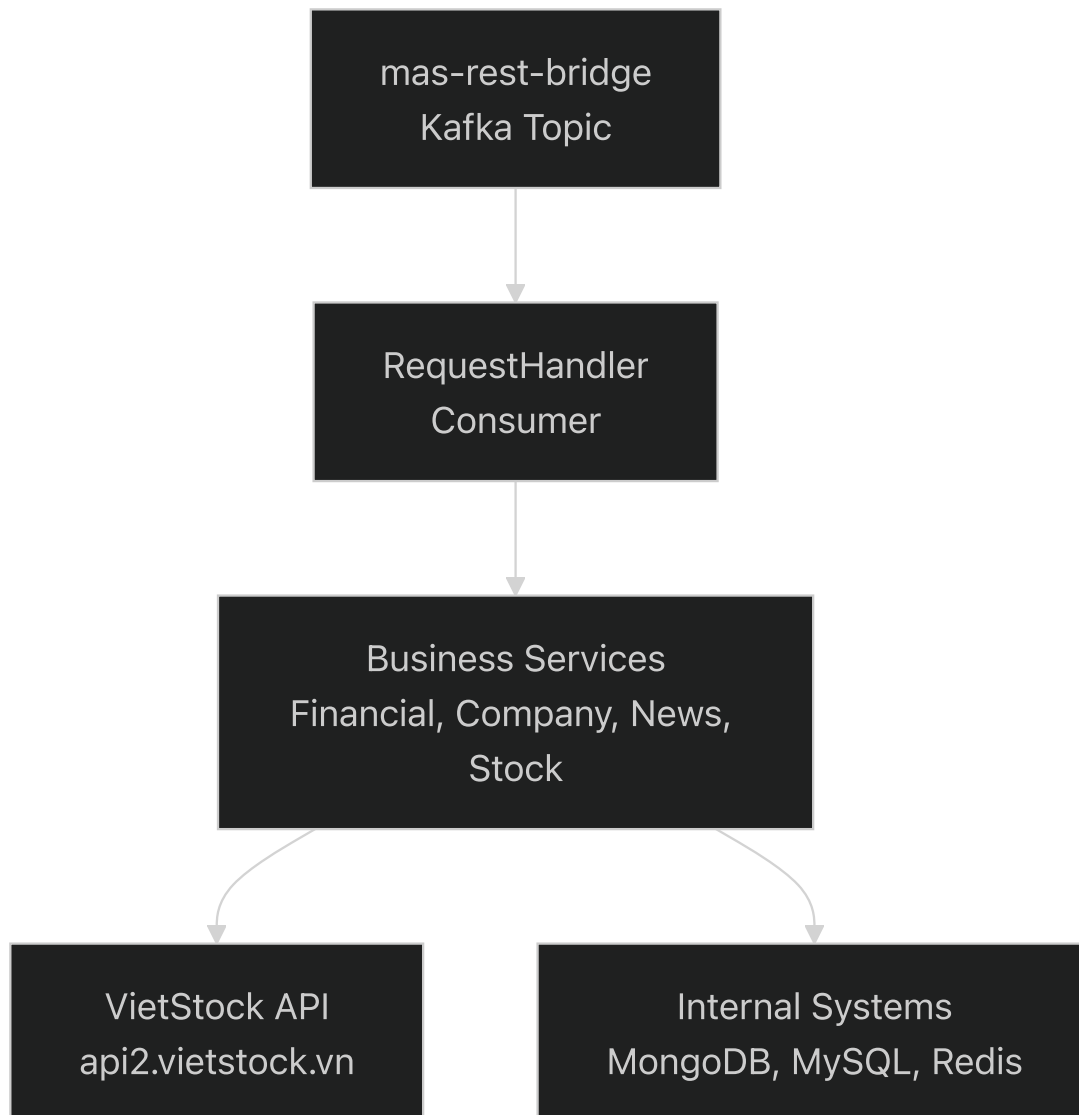
Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/README.md#L1-L4>)

[README.md1-4](#) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/README.md#L1-L4>)

[package.json2-4](#) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L2-L4> (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L2-L4>))

System Role

The bridge service operates as a Kafka consumer that receives requests from the `mas-rest-bridge` topic and fulfills them by calling external APIs. Despite its name referencing "mas" (a specific securities firm), the system is designed to support multiple securities firms including MAS and KIS, with extensibility for additional firms like VCSC.



Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/README.md#L3-L3>

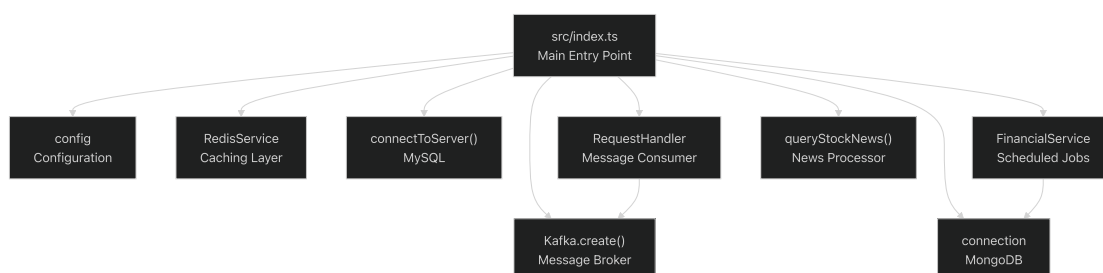
[README.md3](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/README.md#L3-L3) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/README.md#L3-L3>

[README.md38-48](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/README.md#L38-L48) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/README.md#L38-L48>

<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/README.md#L38-L48>

Core Architecture Components

The system initializes several critical components during startup, as orchestrated by the main entry point:



The system follows a dependency injection pattern using the `typedi` container, with each major service component being initialized and connected during the startup sequence defined in <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L14-L28>.

[src/index.ts14-28](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L14-L28) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L14-L28>).

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L1-L37>)

[src/index.ts1-37](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L1-L37) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L1-L37>).

Business Services

The bridge provides four main categories of financial data services:

Service Type	Primary Function	Data Sources
CompanyInfoService	Company profiles, ownership data	VietStock API
NewsService	Stock news, market articles	VietStock API
StockService	Market events, trading data	VietStock API
FinancialService	Financial statements, ratios	VietStock API

Each service implements caching strategies, rate limiting, and error handling for reliable data retrieval. The `FinancialService` additionally provides scheduled background jobs for quarterly and yearly financial statement updates.

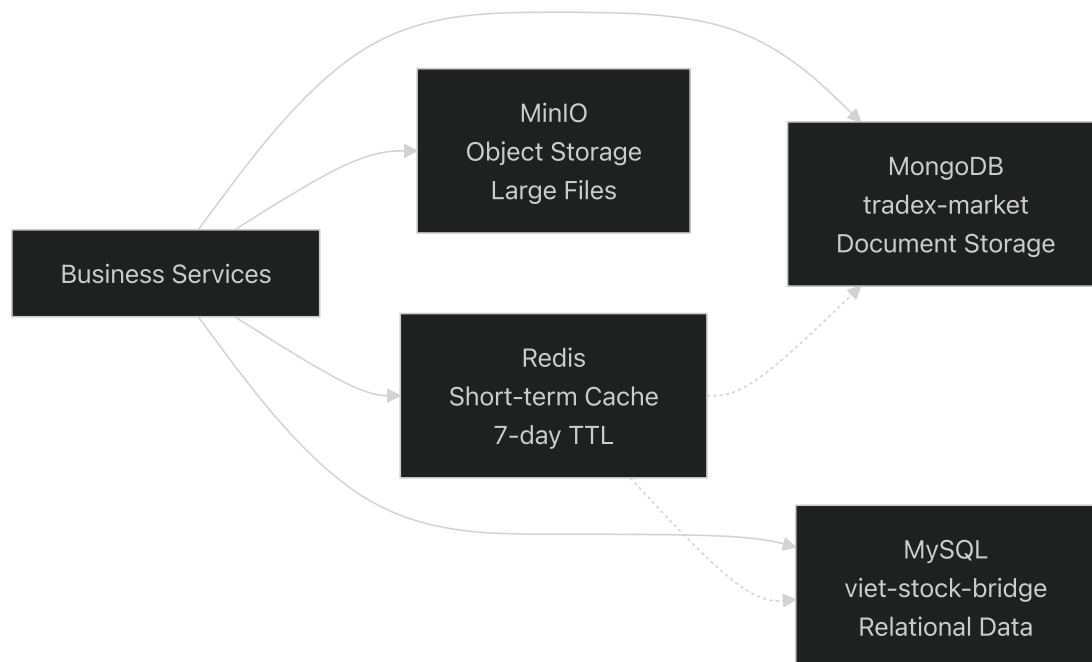
Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L8-L8>)

[src/index.ts8](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L8-L8) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L8-L8>)

[src/index.ts25-27](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L25-L27)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L25-L27>
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L25-L27>))

Data Persistence Strategy

The system employs a multi-database approach optimized for different data characteristics:



This architecture provides optimal performance through Redis caching while maintaining data persistence across MongoDB for flexible document storage and MySQL for structured relational data.

Sources: [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L16-L21\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L16-L21)

[src/index.ts#L16-L21](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L16-L21) [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L16-L21\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L16-L21)

[package.json#L40-L44](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L40-L44) [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L40-L44\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L40-L44)

Technology Stack

The system is built using modern Node.js technologies with enterprise-grade dependencies:

- **Runtime:** Node.js 10.16.3 with TypeScript compilation
- **Messaging:** Kafka integration via `tradex-common` library
- **Databases:** MongoDB 3.3.0, MySQL 2.18.1, Redis 3.0.2
- **HTTP Client:** Axios for external API calls
- **Dependency Injection:** TypeDI for service container management
- **Scheduling:** node-schedule for background job execution

Sources: [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L36-L55\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L36-L55)

[package.json#L36-L55](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L36-L55) [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L36-L55\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L36-L55)

Next Steps

This overview provides the foundation for understanding the `viet-stock-bridge` system. For implementation details, proceed to:

- [System Architecture](#) for detailed component relationships
- [External API Integration](#) for VietStock API implementation
- [Data Management](#) for storage and caching patterns
- [Development Setup](#) for local development configuration

System Architecture

Overview

Relevant source files

- [[src/config.ts](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts) [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts)]
- [[src/consumers/RequestHandler.ts](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts) [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts)]
- [[src/consumers/RequestHandler.ts](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts) [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts)]

src/index.ts](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts>
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts>))

Purpose and Scope

This document provides a comprehensive overview of the `viet-stock-bridge` system architecture, detailing how the major components interact, data flows through the system, and how external integrations are managed. The system serves as a bridge between Vietnamese stock market APIs and the internal Tradex infrastructure, processing financial data requests through Kafka messaging and maintaining data across multiple storage systems.

For information about specific service implementations, see [Core Components](#). For details on Kafka message routing patterns, see [Message Processing](#). For database schema and data models, see [Data Models](#).

Overall System Architecture

The `viet-stock-bridge` application follows a microservice architecture pattern with clear separation between message handling, business logic, and data persistence layers.

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L1-L38>)

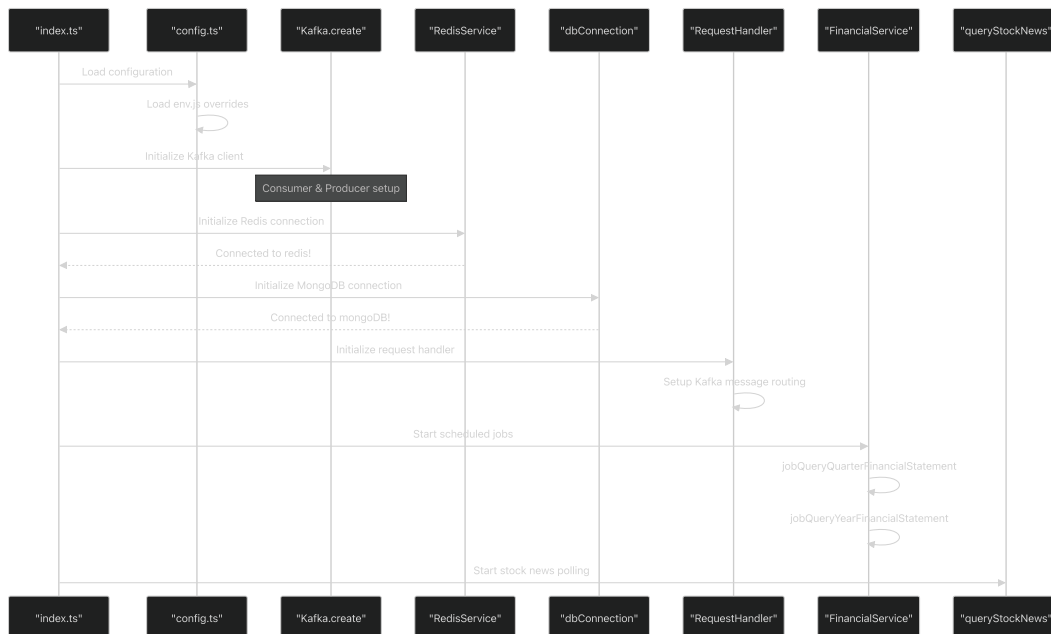
[src/index.ts1-38](#) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L1-L38>)

src/config.ts9-82](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L9-L82>](
([%5B">https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L9-L82">%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L9-L82)).

src/consumers/RequestHandler.ts10-23](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L10-L23> (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L10-L23>))

System Initialization Flow

The application follows a structured initialization sequence that ensures all dependencies are properly established before message processing begins.



Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L14-L28>)

[src/index.ts14-28](#) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L14-L28>)

src/config.ts84-96](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L84-L96>)
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L84-L96>))

Message Processing Architecture

The system processes incoming Kafka messages through a centralized routing mechanism that maps URI patterns to specific service methods.

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L24-L34>)

[src/consumers/RequestHandler.ts24-34](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L24-L34) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L24-L34>)

src/consumers/RequestHandler.ts36-81](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L36-L81>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L36-L81>)

Data Storage Architecture

The system employs a multi-database strategy with each storage system optimized for specific data types and access patterns.

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L10-L17>)

[src/config.ts10-17](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L10-L17) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L10-L17>)

src/config.ts18-26](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L18-L26>)
([%5B">https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L18-L26">%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L18-L26))

src/config.ts67-74](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L67-L74>)
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L67-L74>))

External API Integration Architecture

The system integrates with VietStock APIs through a structured approach with rate limiting and error handling.

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L75-L81>)

[src/config.ts75-81](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L75-L81) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L75-L81>)

src/config.ts27-34](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L27-L34>)
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L27-L34>))

Configuration Management

The system uses a layered configuration approach that allows for environment-specific overrides and dynamic configuration loading.

Configuration Layer	Source	Purpose
Base Configuration	[	
src/config.ts9-82](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L9-L82 (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L9-L82))	Default system settings	
Environment Variables	<code>process.env.TRADEX_ENV_DOMAIN</code>	Environment detection
External Overrides	<code>env.js</code> file	Dynamic configuration changes
Kafka Options	[	

src/config.ts98-105](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L98-L105>
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L98-L105>)) | Messaging system tuning |

Key Configuration Components:

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L6-L7>).

[src/config.ts6-7](#) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L6-L7>)

src/config.ts53-56](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L53-L56>)([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L53-L56)).

src/config.ts67-74](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L67-L74>)([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L67-L74)).

src/config.ts84-96](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L84-L96>
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L84-L96>))

Core Components

Relevant source files

- [

entrypoint.sh](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh>
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh>))
- [

src/config.ts](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts>
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts>))
- [

src/index.ts](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts>
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts>))

This page documents the fundamental components that form the backbone of the viet-stock-bridge application, including system initialization, configuration management, and service orchestration. These components

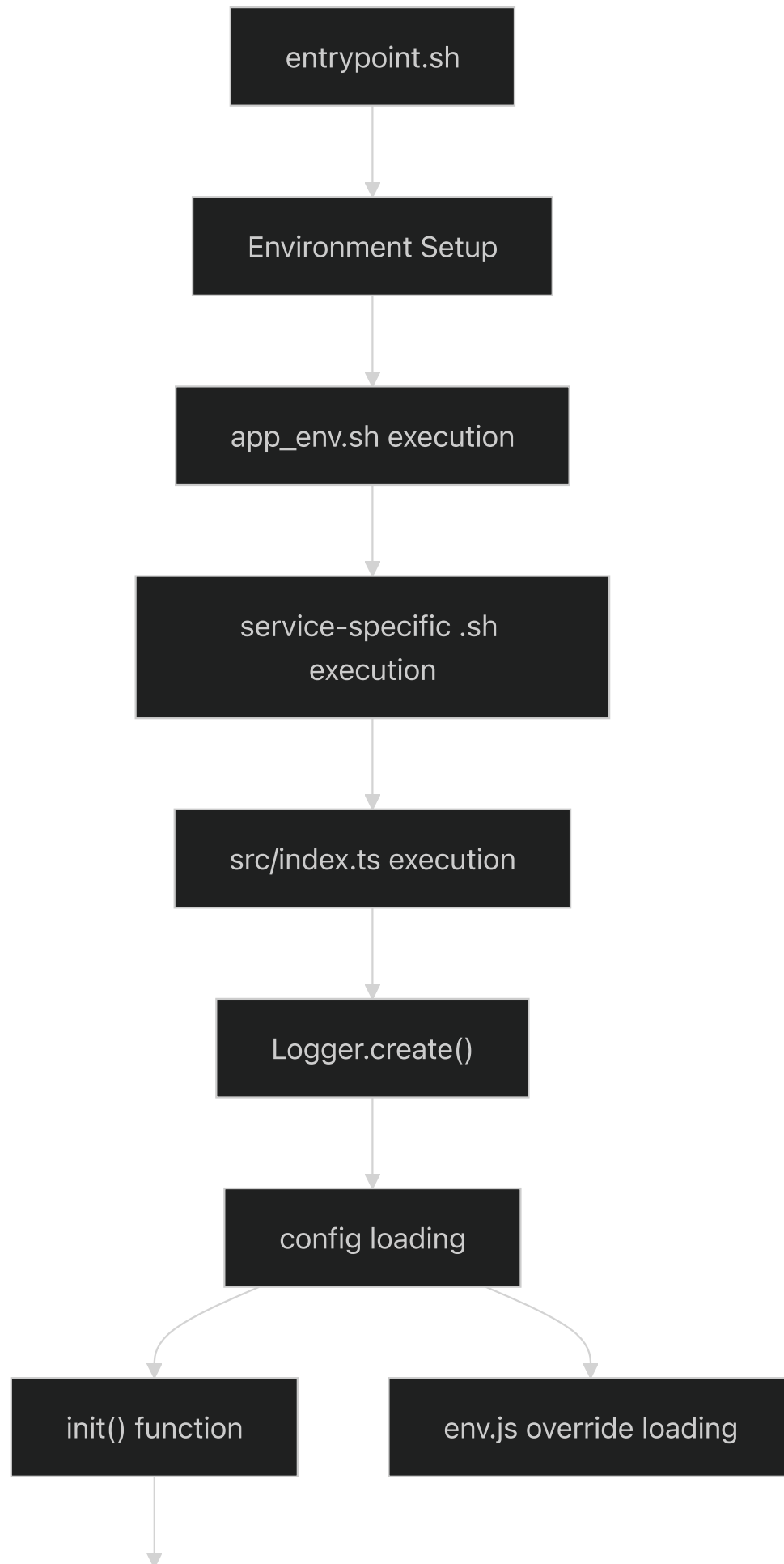
establish the runtime environment and coordinate the startup sequence for all application services.

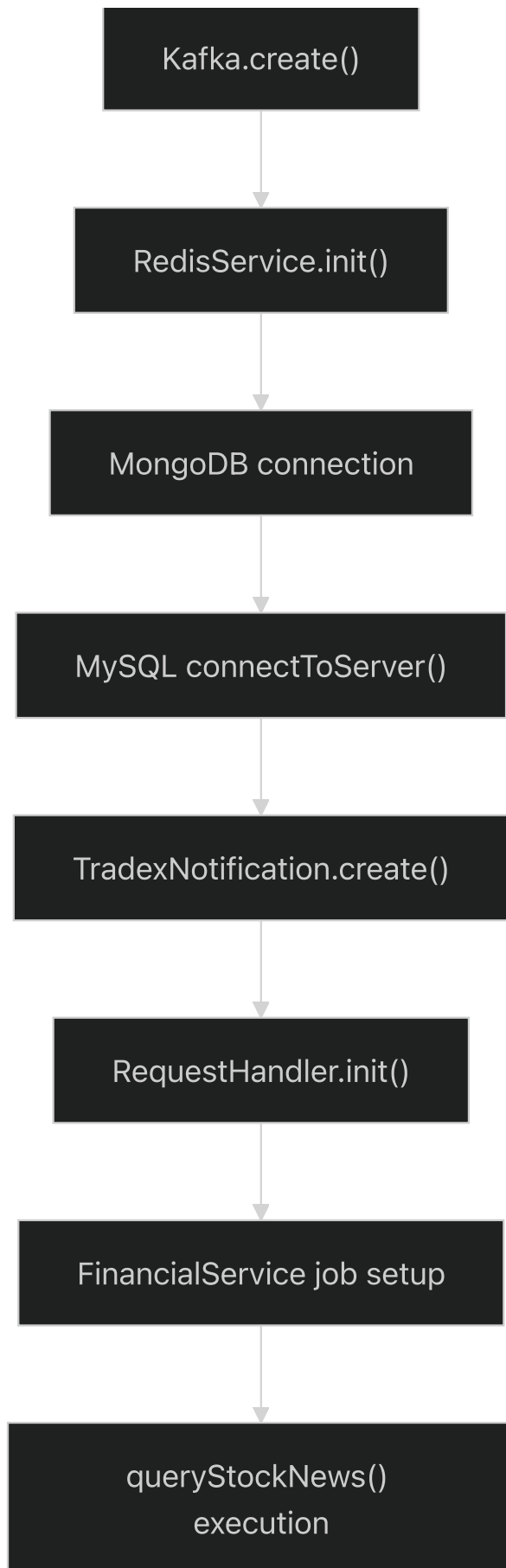
For information about message processing and request handling, see [Message Processing](#). For details about external API integration patterns, see [External API Integration](#). For deployment and containerization specifics, see [Service Orchestration](#).

System Initialization Flow

The application follows a structured initialization sequence orchestrated by the main entry point and supported by configuration management and environment setup scripts.

Application Bootstrap Sequence





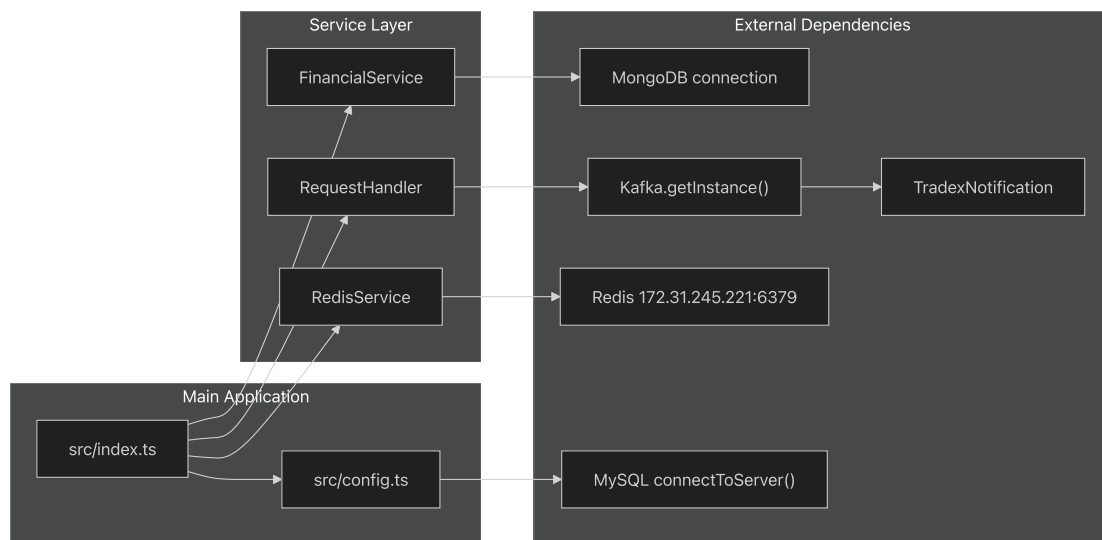
Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L1-L38>.

[src/index.ts1-38](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L1-L38) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L1-L38>

entrypoint.sh1-90](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L1-L90>
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L1-L90>))

Core Service Dependencies

The system establishes connections to multiple external services during initialization, with each dependency managed by specific service classes.



Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L14-L28>).

[src/index.ts#L14-L28](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L14-L28) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L14-L28>)

src/config.ts10-26](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L10-L26>]([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L10-L26))

src/config.ts67-71](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L67-L71>)
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L67-L71>)

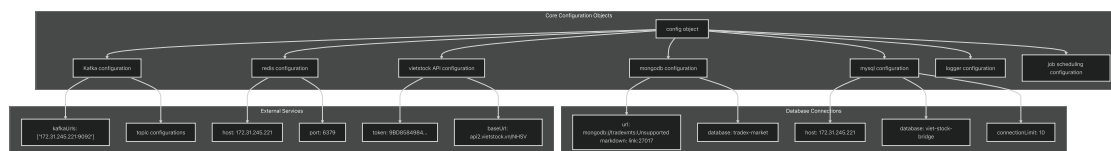
Configuration Management

The configuration system uses a layered approach with static defaults, environment-specific overrides, and dynamic configuration loading capabilities.

Configuration Architecture

Component	Purpose	Location	Override Priority
Base Configuration	Default settings for all environments	src/config.ts	1 (lowest)
Environment Variables	Runtime environment configuration	process.env	2
Service Scripts	Service-specific environment setup	/appEnv/services/	3
External Configuration	Dynamic configuration override	env.js	4 (highest)

Configuration Categories

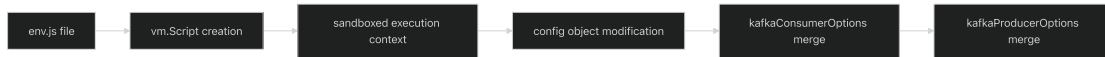


Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L9-L82>.

[src/config.ts#9-82](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L9-L82) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L9-L82>)

Dynamic Configuration Override

The system supports runtime configuration modification through a sandboxed JavaScript execution environment:



The configuration loading process uses Node.js VM module to safely execute external configuration scripts with access to the main `config` object, allowing for environment-specific customizations without code changes.

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L84-L96>.

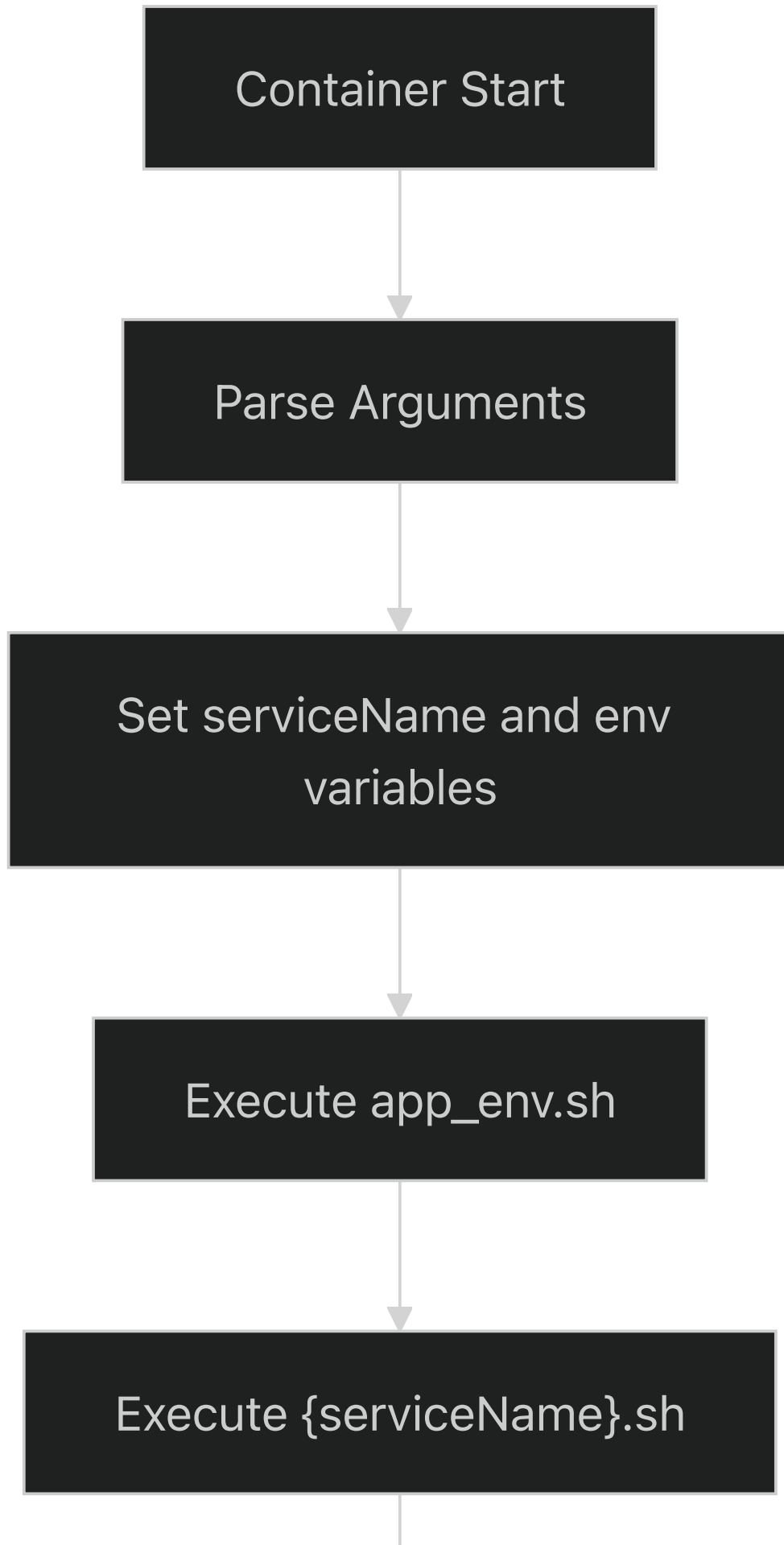
[src/config.ts84-96](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L84-L96) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L84-L96>)

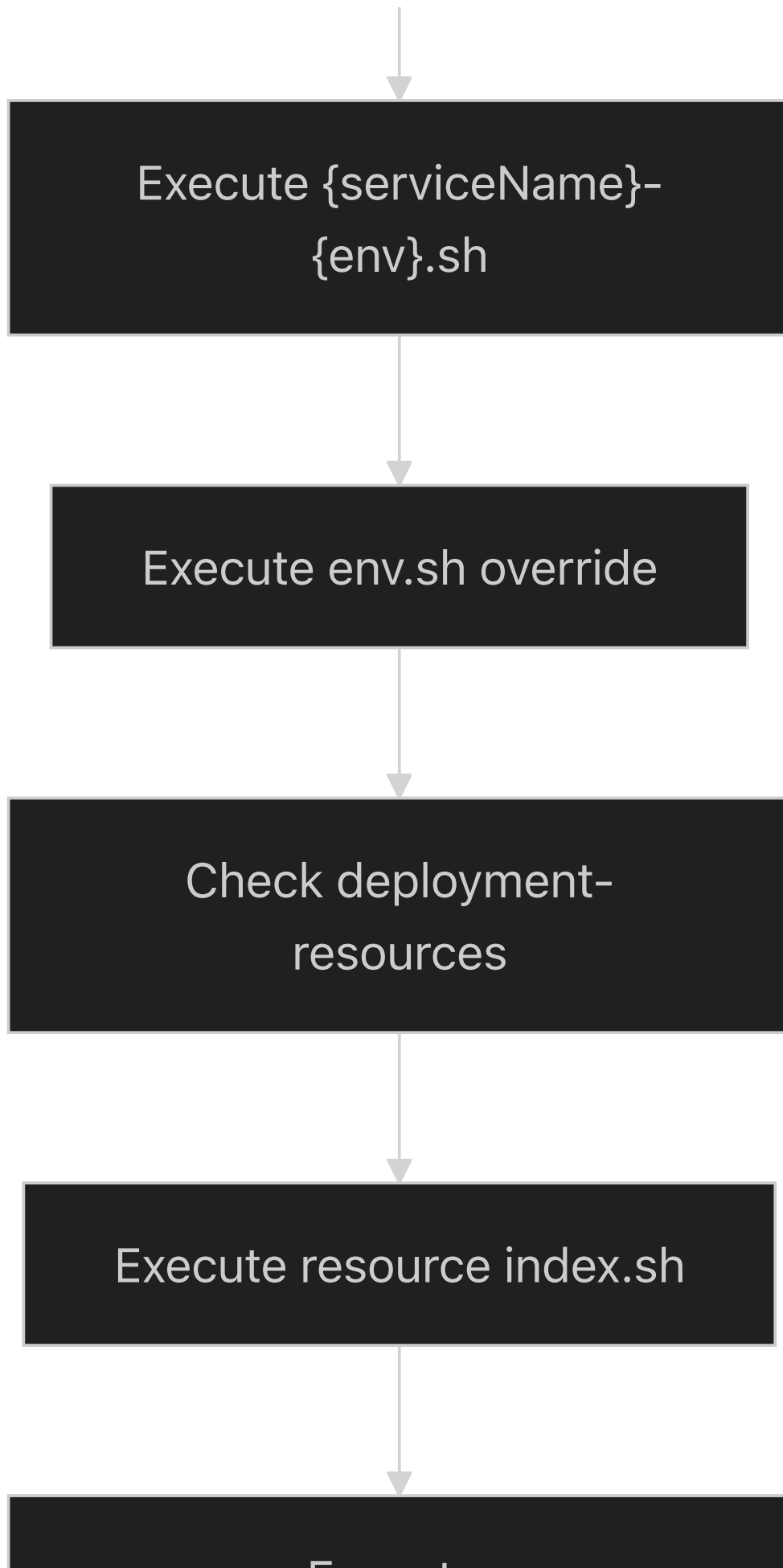
[src/config.ts98-105](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L98-L105) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L98-L105>)

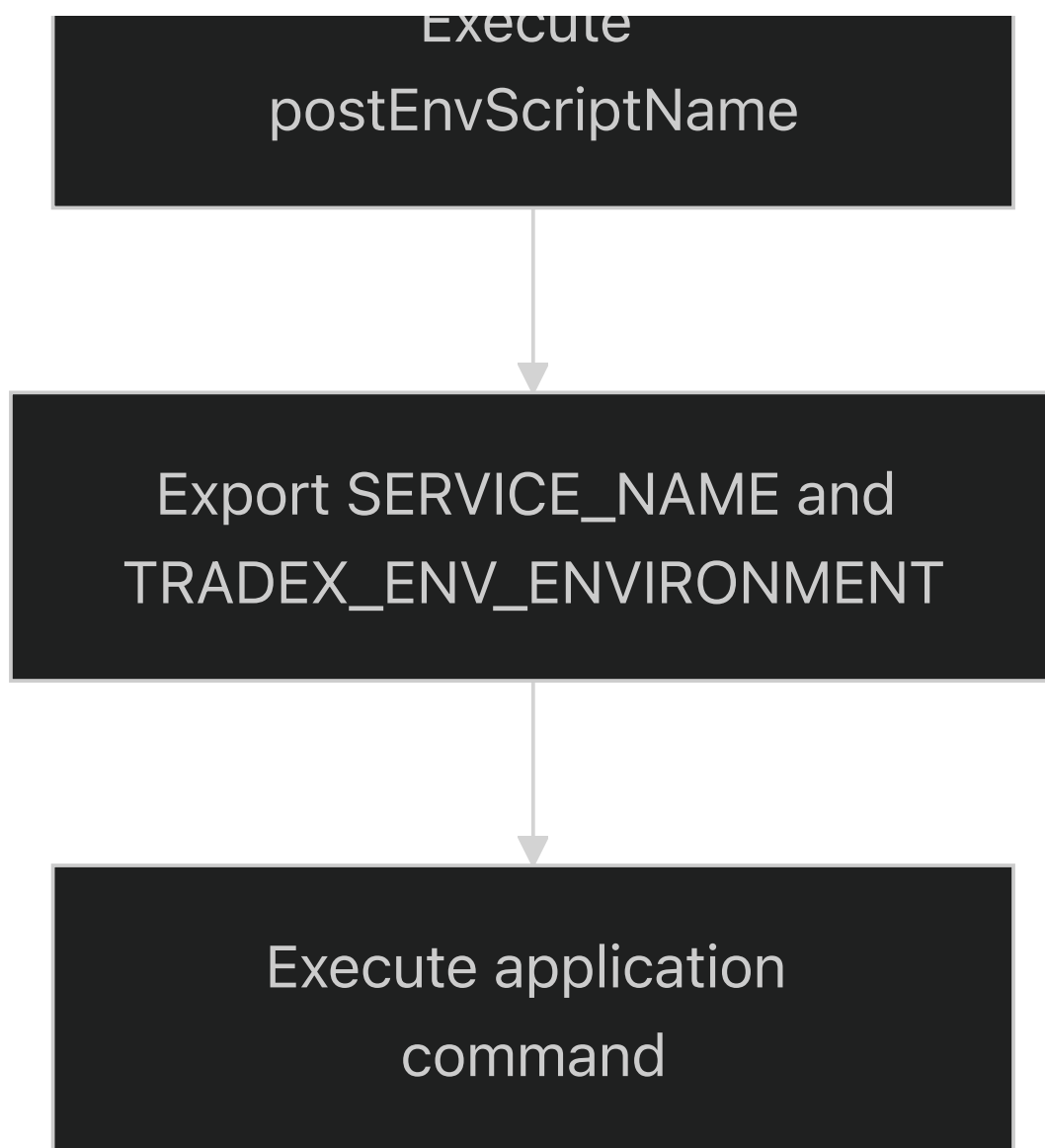
Service Orchestration

The service orchestration layer manages environment setup, service-specific configuration, and application lifecycle through shell scripts and environment management.

Environment Setup Process







Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L7-L89>

[entrypoint.sh7-89](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L7-L89) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L7-L89>)

Environment Variable Management

The orchestration system manages multiple layers of environment configuration:

Environment Layer	File Pattern	Purpose
Global Environment	<code>app_env.sh</code>	System-wide environment variables
Service Configuration	<code>{serviceName}.sh</code>	Service-specific settings
Environment-Specific	<code>{serviceName}-{env}.sh</code>	Environment-specific overrides
Local Overrides	<code>env.sh</code>	Local development overrides
Deployment Resources	<code>deployment-resources/{serviceName}-{env}/index.sh</code>	Deployment-specific setup

Application Lifecycle Management

The system exports critical environment variables and working directory information for the Node.js application:

- `TRADEX_WORKING_DIR` : Current working directory for resource location
- `SERVICE_NAME` : Service identifier for logging and monitoring
- `TRADEX_ENV_ENVIRONMENT` : Runtime environment specification (development, staging, production)

These variables are consumed by the configuration management system to determine runtime behavior and external service connections.

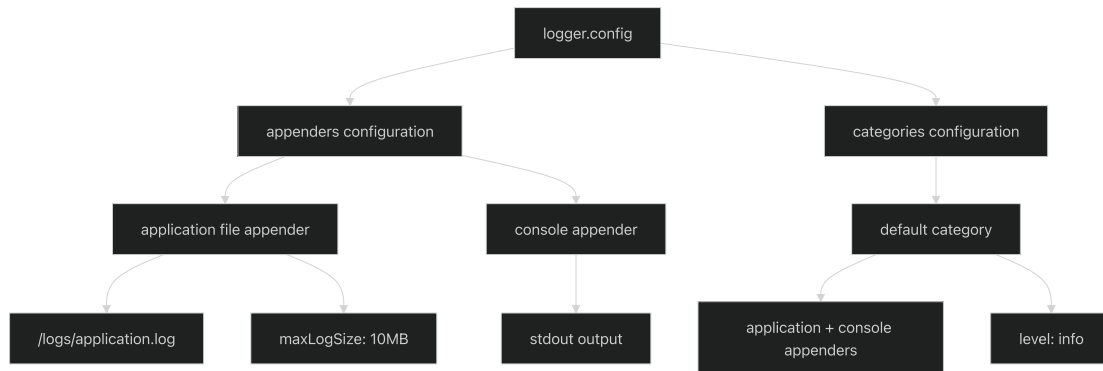
Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L35-L87>

[entrypoint.sh35-87](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L35-L87) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L35-L87>)

Logging and Monitoring Infrastructure

The application establishes comprehensive logging infrastructure during initialization using the `tradex-common` Logger with dual output streams.

Logger Configuration Structure



The logging system is initialized early in the application lifecycle via `Logger.create(config.logger.config, true)` and provides structured logging output for both development debugging and production monitoring.

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L35-L52>)

[src/config.ts35-52](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L35-L52) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/config.ts#L35-L52>)

[src/index.ts11-12](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L11-L12) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L11-L12>)
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/index.ts#L11-L12>)

Message Processing

Relevant source files

- [[src/consumers/RequestHandler.ts](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts>)
- [[src/models/IContext.ts](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts>)

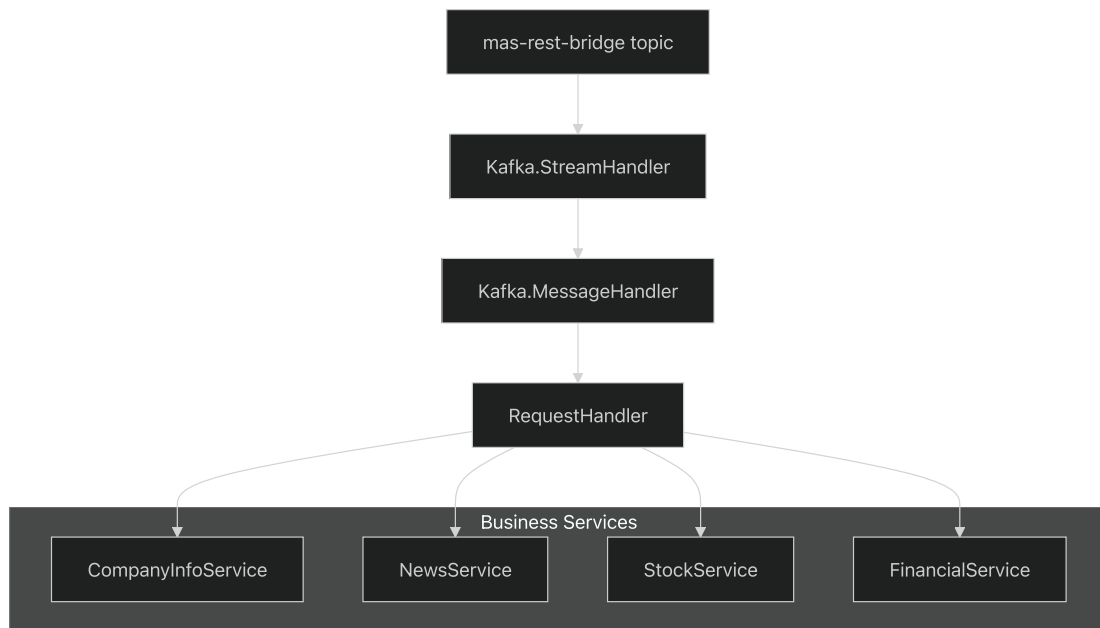
This document describes how the viet-stock-bridge system processes incoming Kafka messages and routes them to appropriate business services. It covers the message consumption architecture, URI-based routing patterns, and context management for request handling.

For information about the specific business services that process these routed messages, see [Core Components](#). For details about the external API integrations that these services use, see [External API Integration](#).

Message Flow Architecture

The system uses a Kafka-based messaging architecture to process incoming requests asynchronously. Messages are consumed from Kafka topics and routed to business services based on URI patterns.

Message Processing Flow



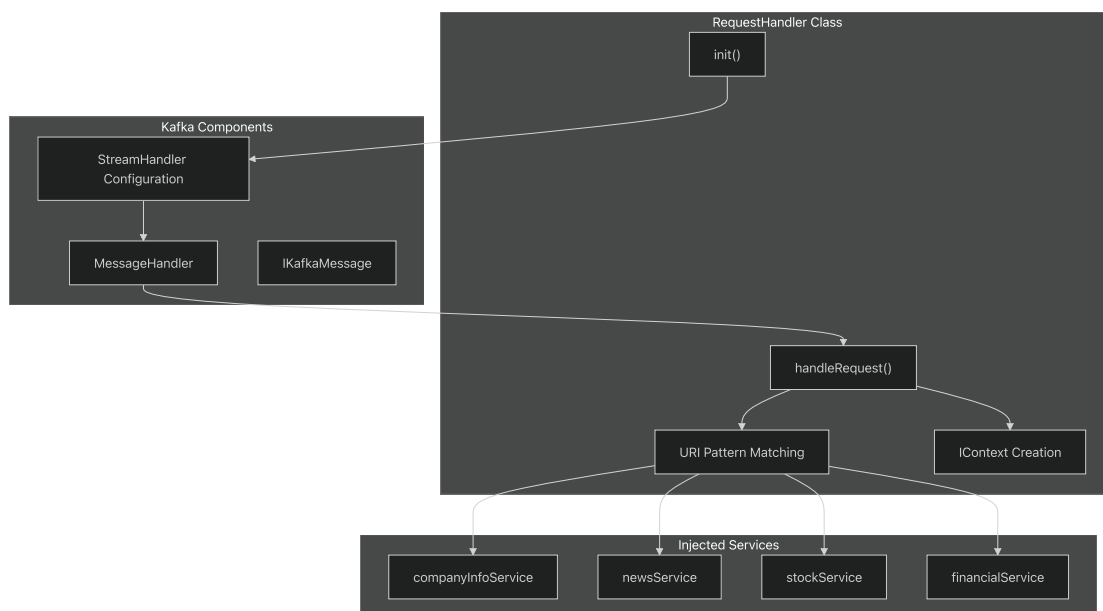
Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L24-L34>.

[src/consumers/RequestHandler.ts24-34](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L24-L34) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L24-L34>.

RequestHandler Implementation

The `RequestHandler` class serves as the central message processing component that consumes Kafka messages and routes them to appropriate services based on URI patterns.

RequestHandler Architecture



The `RequestHandler` class initializes the Kafka consumer using `tradex-common` components and establishes message processing through the `init()` method <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L24-L34>.

[src/consumers/RequestHandler.ts24-34](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L24-L34) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L24-L34>. The message handling logic is implemented in the private `handleRequest` method[

src/consumers/RequestHandler.ts36-81](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L36-L81>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L36-L81>))

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L10-L34>).

[src/consumers/RequestHandler.ts10-34](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L10-L34) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L10-L34>)

URI-Based Message Routing

The system routes messages to services based on URI patterns contained in the Kafka message. The routing logic uses a series of conditional statements to match URI patterns and dispatch to appropriate service methods.

API Endpoint Routes

URI Pattern	Service	Method	Purpose
<code>get:/api/v1/company/transactionHistory</code>	CompanyInfoService	<code>getTransactionHistory</code>	Retrieve company transaction history
<code>get:/api/v1/company/profile</code>	CompanyInfoService	<code>getCompanyProfile</code>	Get company profile information
<code>get:/api/v1/company/ownershipStructure</code>	CompanyInfoService	<code>getOwnershipStructure</code>	Fetch ownership structure data
<code>get:/api/v1/company/majorShareholders</code>	CompanyInfoService	<code>getMajorShareholders</code>	Get major shareholders information
<code>get:/api/v1/news/latest</code>	NewsService	<code>getLatestNews</code>	Retrieve latest news
<code>get:/api/v1/news/stockNews</code>	NewsService	<code>getStockNews</code>	Get stock-specific news
<code>get:/api/v1/news/newsDetail</code>	NewsService	<code>getNewsDetail</code>	Fetch detailed news information

Job-Based Routes

URI Pattern	Service	Method	Purpose
<code>job:/vietstock/queryQuarterFinancialStatement</code>	FinancialService	<code>queryQuarterFinancialStatement</code>	Process quarterly financial data
<code>job:/vietstock/queryYearFinancialStatement</code>	FinancialService	<code>queryYearFinancialStatement</code>	Process annual financial data

Financial Data Routes

URI Pattern	Service	Method	Purpose
<code>get:/api/v1/finance/incomeStatement</code>	FinancialService	<code>getIncomeStatement</code>	Retrieve income statement data
<code>get:/api/v1/finance/balanceSheet</code>	FinancialService	<code>getBalanceSheet</code>	Get balance sheet information
<code>get:/api/v1/finance/cashFlow</code>	FinancialService	<code>getCashFlow</code>	Fetch cash flow statements
<code>get:/api/v1/financial/latestFinancialRatio</code>	FinancialService	<code>getLatestFinancialRatio</code>	Get latest financial ratios

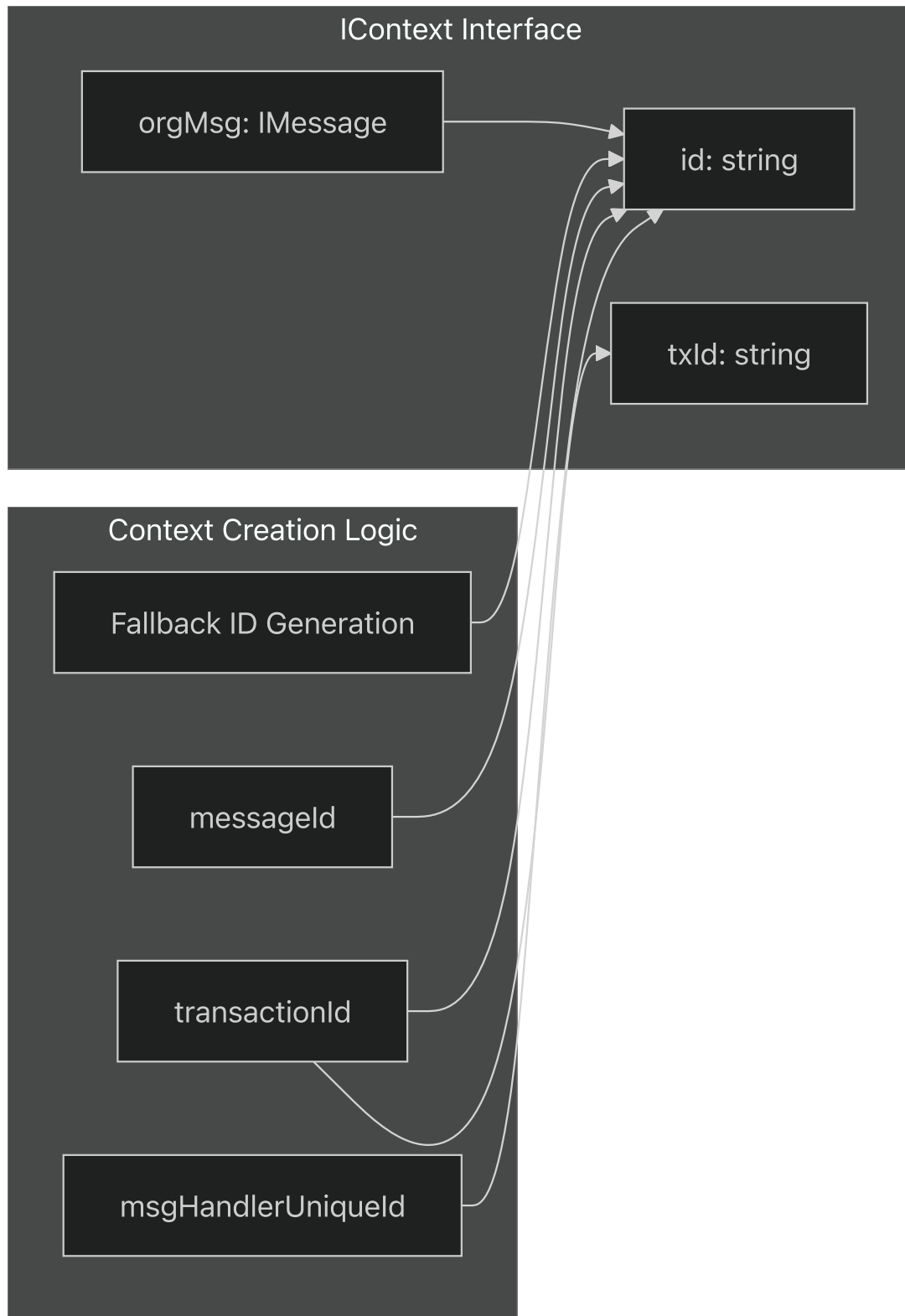
Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L45-L79>

[src/consumers/RequestHandler.ts45-79](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L45-L79) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L45-L79>)

Context Management

Each processed message creates an `IContext` object that provides tracking and metadata for the request processing pipeline.

Context Structure



The context object is constructed from the incoming Kafka message (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L40-L44>),

[src/consumers/RequestHandler.ts#L40-L44](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L40-L44) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L40-L44>) and provides:

- **id**: Unique identifier derived from `msgHandlerUniqueId` or combination of `transactionId` and `messageId`
- **txId**: Transaction identifier for request correlation

- **orgMsg**: Reference to the original Kafka message for additional metadata access

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L40-L44>

[src/consumers/RequestHandler.ts40-44](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L40-L44) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L40-L44>)

[src/models/IContext.ts3-7](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts#L3-L7)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts#L3-L7>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts#L3-L7>)

Error Handling

The message processing system implements structured error handling for different failure scenarios:

Input Validation

- Returns `Errors.SystemError()` for null messages or missing message data

[src/consumers/RequestHandler.ts37-39](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L37-L39)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L37-L39>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L37-L39>)

Route Resolution

- Returns `Errors.UriNotFound()` when no matching URI pattern is found

[src/consumers/RequestHandler.ts80](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L80-L80)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L80-L80>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L80-L80>)

Service Method Execution

- All service methods return Promise-based results, allowing for asynchronous error propagation
- Service-specific error handling is delegated to individual business service implementations

The error handling approach ensures that malformed messages, unknown routes, and service-level failures are properly categorized and reported through the Kafka message handling infrastructure.

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L37-L39>

[src/consumers/RequestHandler.ts37-39](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L37-L39) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L37-L39>)

[src/consumers/RequestHandler.ts80](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L80-L80)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L80-L80>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/consumers/RequestHandler.ts#L80-L80>)

External API Integration

Relevant source files

- [

[src/constants/VietStockAPI.ts](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/VietStockAPI.ts)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/VietStockAPI.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/VietStockAPI.ts>)

- [src/daos/VietStockCommonDao.ts](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts) (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts))

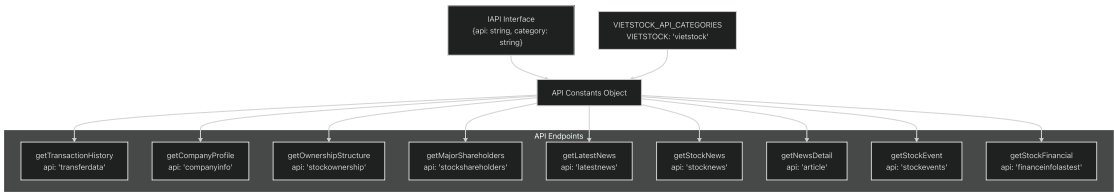
- [src/models/IAPITs](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IAPITs) (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IAPITs))

This document covers the external API integration patterns used by the viet-stock-bridge service to communicate with Vietnamese stock market data providers. The system primarily integrates with VietStock APIs to retrieve financial data, company information, and market news. For information about data storage and persistence patterns, see [Data Management](#). For details about internal message processing, see [Message Processing](#).

API Configuration and Endpoint Management

The system defines external API endpoints through a centralized configuration approach using the `IAPITs` interface and endpoint constants.

API Definition Structure



The system uses a structured approach to define API endpoints through the `API` constant object, where each endpoint is mapped to an `IAPITs` configuration containing the endpoint path and category.

Sources: (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/VietStockAPI.ts#L7-L44)

[src/constants/VietStockAPI.ts#7-44](#) (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/VietStockAPI.ts#L7-L44)[

src/models/IAPITs#1-4](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IAPITs#L1-L4) (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IAPITs#L1-L4))

Available API Endpoints

Endpoint Function	API Path	Purpose
getTransactionHistory	transferdata	Retrieve stock transaction history
getCompanyProfile	companyinfo	Get company profile information
getOwnershipStructure	stockownership	Fetch ownership structure data
getMajorShareholders	stockshareholders	Get major shareholder information
getLatestNews	latestnews	Retrieve latest market news
getStockNews	stocknews	Get stock-specific news
getNewsDetail	article	Fetch detailed news articles
getStockEvent	stockevents	Get stock event information
getStockFinancial	financeinfoatest	Retrieve latest financial data

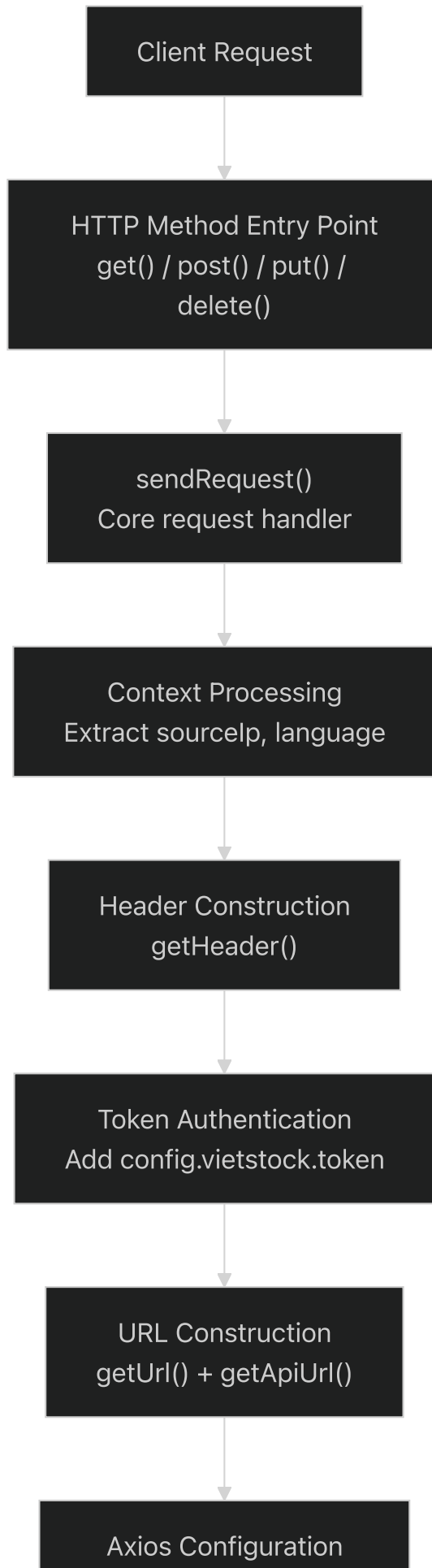
Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/VietStockAPI.ts#L8-L43>

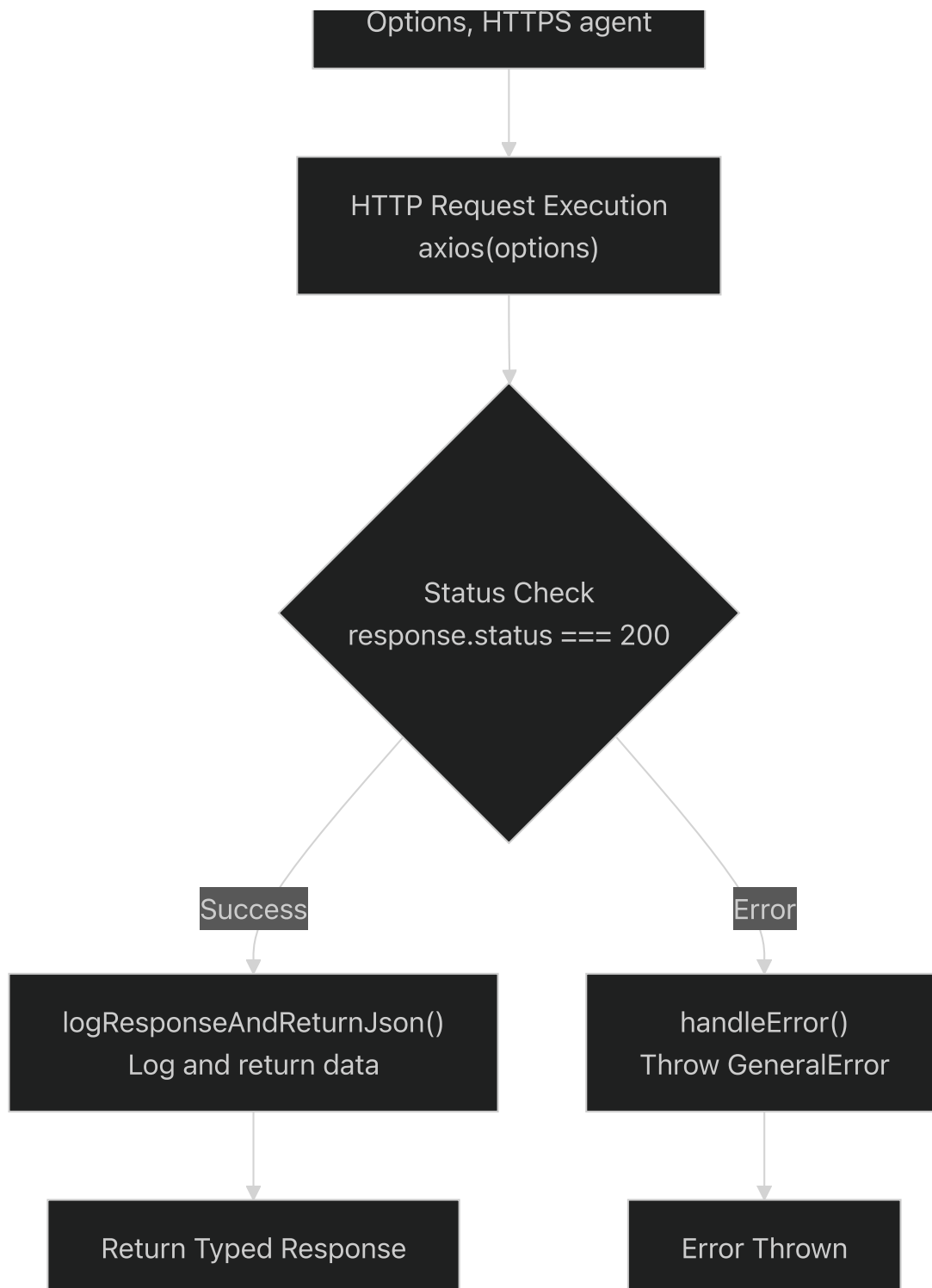
[src/constants/VietStockAPI.ts8-43](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/VietStockAPI.ts#L8-L43) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/VietStockAPI.ts#L8-L43>)

HTTP Client Implementation

The `VietStockCommonDao` class serves as the primary HTTP client for external API communication, providing a standardized interface for all API requests.

Request Processing Flow





Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L25-L153>),

[src/daos/VietStockCommonDao.ts25-153](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L25-L153) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L25-L153>).

Core HTTP Methods

The DAO provides standard HTTP method wrappers that delegate to the main `sendRequest` method:

- `get<T>()` - GET requests with optional query parameters and path parameters
- `post<T>()` - POST requests with request body support
- `put<T>()` - PUT requests for updates

- `delete<T>()` - DELETE requests with optional parameters

Each method supports generic typing for response data and accepts common parameters including headers, body data, path parameters, query parameters, and context information.

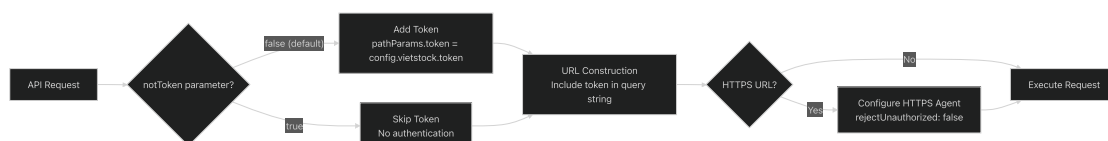
Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L25-L55>).

[src/daos/VietStockCommonDao.ts25-55](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L25-L55) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L25-L55>).

Authentication and Security

The system implements token-based authentication for VietStock API access along with security configurations for HTTPS communication.

Authentication Flow



The authentication token is retrieved from `config.vietstock.token` and automatically appended to request URLs as a query parameter unless explicitly disabled via the `notToken` parameter.

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L86-L91>).

[src/daos/VietStockCommonDao.ts86-91](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L86-L91) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L86-L91>)

[src/daos/VietStockCommonDao.ts107-113](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L107-L113) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L107-L113>)

Language Code Support

The system supports internationalization through language code mapping from HTTP headers:

```
// Language code extraction from context
if (ctx.orgMsg.data.headers !== null) {
  const language = ctx.orgMsg.data.headers['accept-language'];
  pathParams = {
    ...pathParams,
    languageID: VIET_STOCK_LANG_CODE[language],
  };
}
```

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L76-L82>).

[src/daos/VietStockCommonDao.ts76-82](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L76-L82) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L76-L82>).

URL Construction and Request Configuration

The system builds complete API URLs through a multi-stage process that combines base URLs, API paths, and query parameters.

URL Building Process

The `getUrl()` and `getApiUrl()` methods handle URL construction:

- 1. API Path Processing:** Converts `IAPI` object to URL string with parameters
- 2. Query Parameter Serialization:** Appends path parameters as query string
- 3. Base URL Resolution:** Combines with category-specific base URL from configuration

```
private getApiUrl(api: IAPI, urlData?: IParam): IAPI {
  if (urlData === null) {
    return api;
  } else {
    let data = api.api;
    const keys: string[] = Object.keys(urlData);
    for (let i = 0; i < keys.length; i++) {
      if (i === 0) {
        data += `?${keys[i]}=${urlData[keys[i]]}`;
        continue;
      }
      const key: string = keys[i];
      data += `&${key}=${urlData[key]}`;
    }
    // ... return constructed IAPI object
  }
}
```

The final URL is constructed by combining the base URL from configuration with the API path:

```
config.vietstock.baseUrl[apiUri.category]/${apiUri.api}.
```

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L186-L217>)

[src/daos/VietStockCommonDao.ts186-217](#) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L186-L217>)

Error Handling and Monitoring

The system implements comprehensive error handling with detailed logging and request monitoring capabilities.

Error Management Strategy

Error handling occurs at multiple levels:

1. **Axios Error Handling:** Catches network and HTTP errors
2. **Status Code Validation:** Checks for successful HTTP 200 responses
3. **Response Error Processing:** Handles API-specific error responses
4. **Generic Error Wrapping:** Converts all errors to `GeneralError` instances

```
private async handleError<T>(response: AxiosResponse): Promise<T> {
  if (response.status) {
    throw new Errors.GeneralError(response.statusText.replace(/ /g, '_').toUpperCase());
  }
  throw new Errors.GeneralError(response.data.error_desc);
}
```

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L138-L160>)

[src/daos/VietStockCommonDao.ts138-160](#) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L138-L160>)

Request Logging and Monitoring

The system provides detailed request and response logging:

- **Request Logging:** Converts requests to cURL format using `r2curl`
- **Response Logging:** Logs response data with size-based truncation
- **Performance Monitoring:** Tracks request duration using `process.hrtime()`
- **Configurable Verbosity:** Optional logging control via `log` parameter

Response logging includes intelligent truncation based on `config.queryLogLength` and `config.queryLogSub` settings to manage log size while preserving important information.

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L162-L184>)

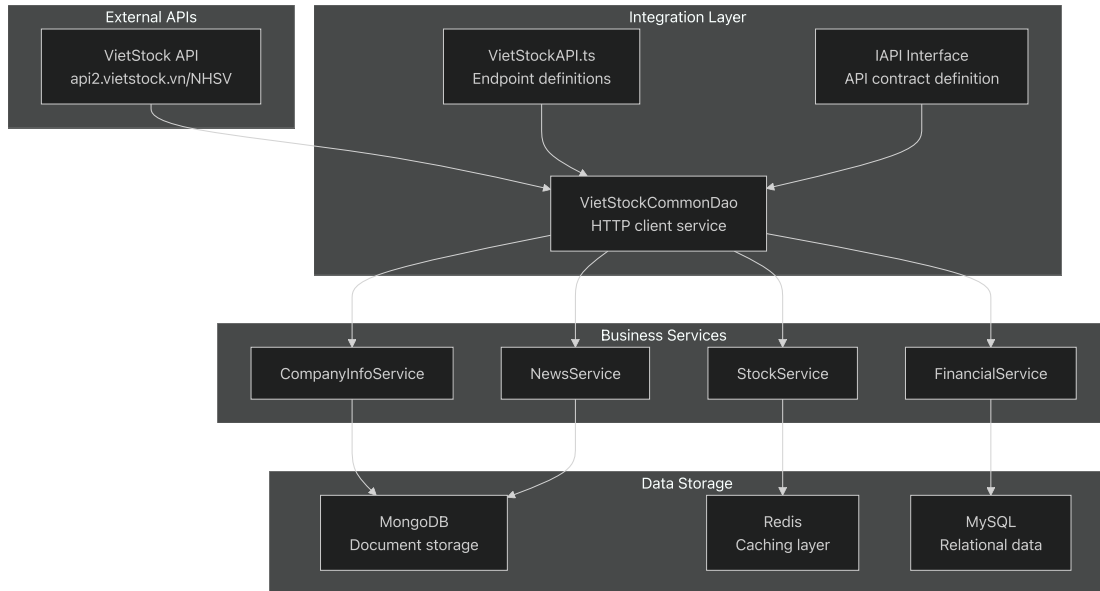
[src/daos/VietStockCommonDao.ts162-184](#) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L162-L184>)

[src/daos/VietStockCommonDao.ts115-128](#) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L115-L128>)

Integration Architecture

The external API integration fits into the broader system architecture as a data acquisition layer that feeds into the internal data processing pipeline.

System Integration Points



The integration architecture provides a clean separation between external API communication and internal business logic, allowing services to consume external data through a standardized interface without directly coupling to external API specifics.

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/VietStockAPI.ts#L1-L44>)

[src/constants/VietStockAPI.ts1-44](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/VietStockAPI.ts#L1-L44) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/VietStockAPI.ts#L1-L44>)

[src/daos/VietStockCommonDao.ts1-218](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L1-L218) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L1-L218>) ([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/daos/VietStockCommonDao.ts#L1-L218))

[src/models/IAPI.ts1-4](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IAPI.ts#L1-L4) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IAPI.ts#L1-L4>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IAPI.ts#L1-L4>)

Data Management

Overview

Relevant source files

- [
[src/init/RedisService.ts](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts>)
- [

src/models/db/dbConnection.ts](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/dbConnection.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/dbConnection.ts>))

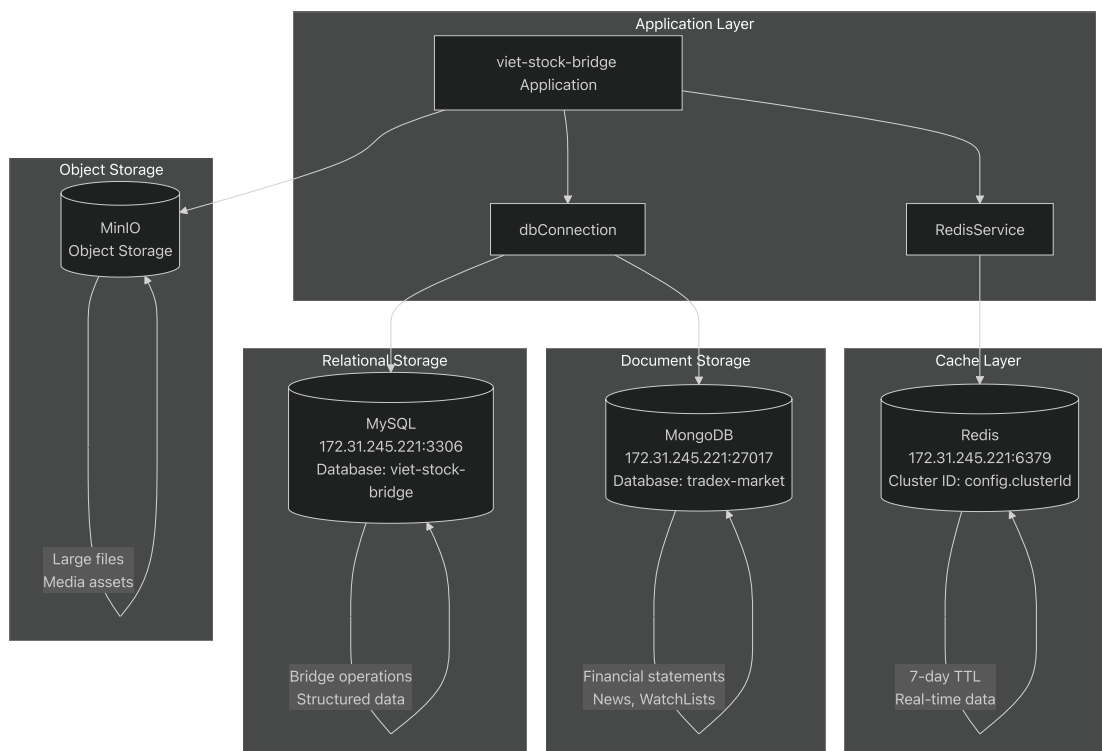
Purpose and Scope

This document covers the data management strategies employed by the viet-stock-bridge system, including database connections, caching mechanisms, and data persistence patterns. It details how the system integrates with multiple data stores (MongoDB, MySQL, Redis, and MinIO) to handle different types of financial data efficiently.

For detailed information about specific data structures and models, see [Data Models](#). For repository implementations and data access patterns, see [Data Access Layer](#). For caching-specific implementation details, see [Caching Strategy](#).

Data Storage Architecture

The viet-stock-bridge system employs a polyglot persistence approach, utilizing different databases optimized for specific data types and access patterns:



Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/dbConnection.ts#L1-L31>)

[src/models/db/dbConnection.ts1-31](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/dbConnection.ts#L1-L31) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/dbConnection.ts#L1-L31>)

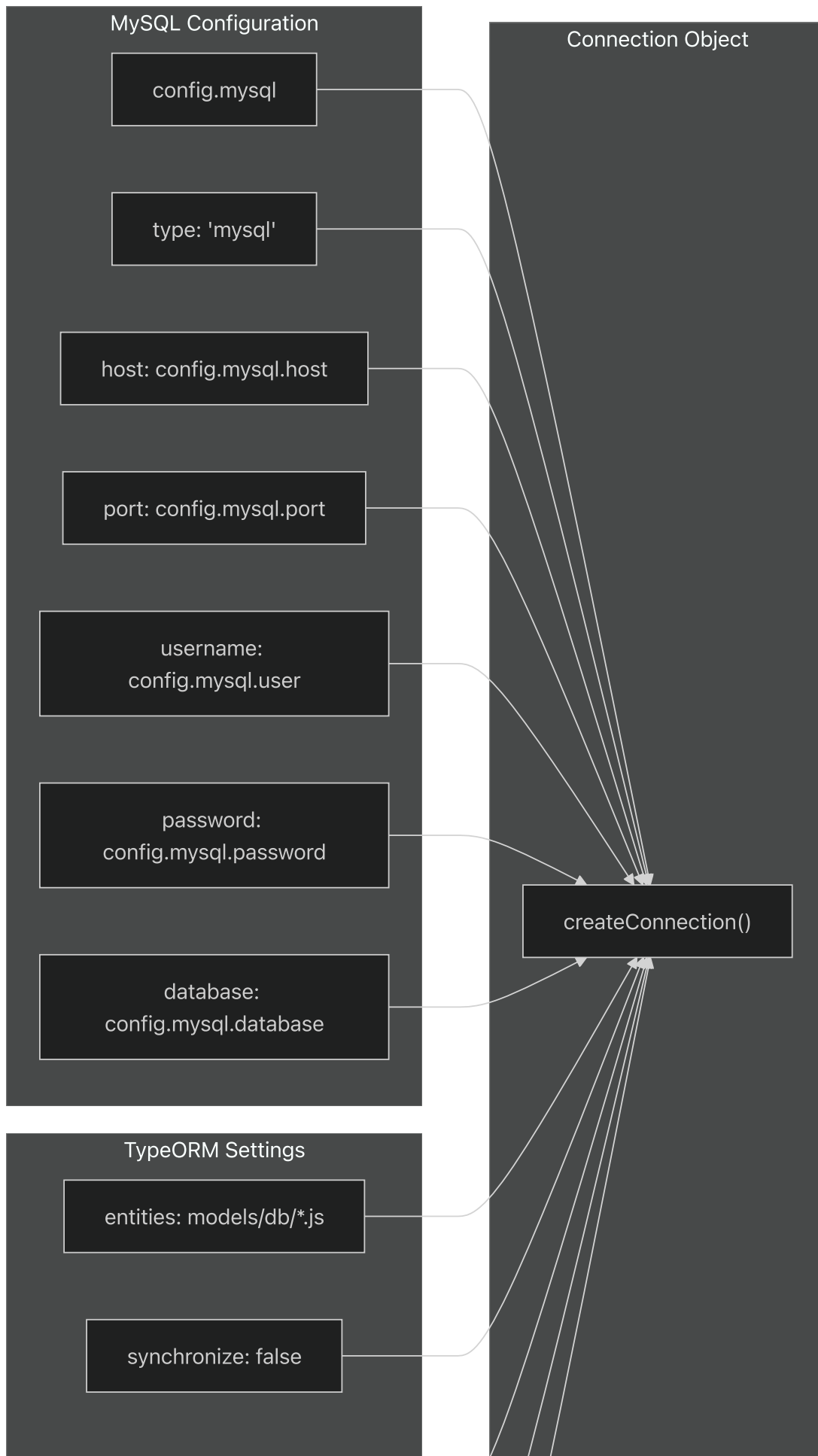
src/init/RedisService.ts66-78](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L66-L78>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L66-L78>))

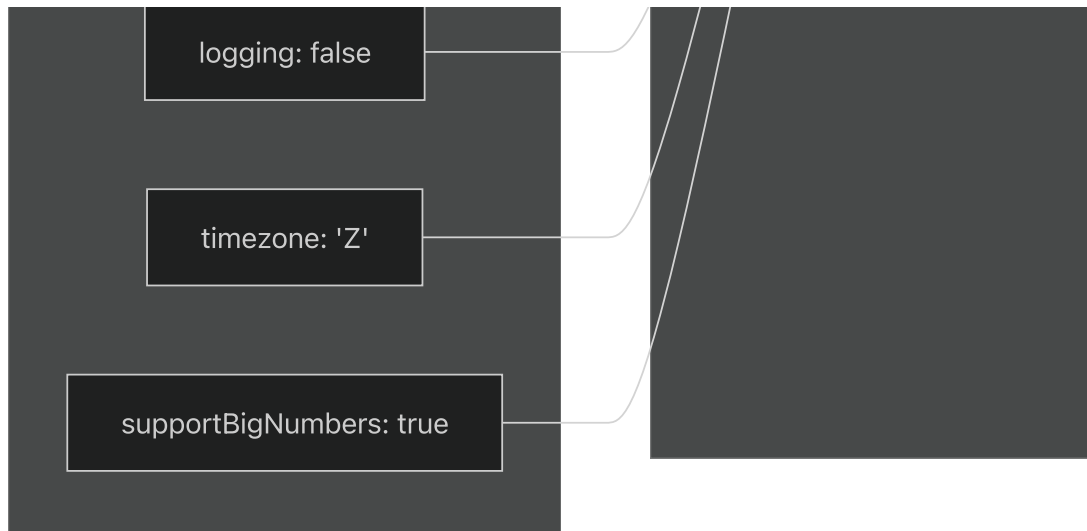
Database Connection Management

The system manages connections to multiple databases through a centralized connection module:

MySQL Connection

The MySQL connection is established using TypeORM with specific configuration for financial data handling:





Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/dbConnection.ts#L7-L20>

[src/models/db/dbConnection.ts7-20](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/dbConnection.ts#L7-L20) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/dbConnection.ts#L7-L20>)

MongoDB Connection

MongoDB connection is handled through native MongoClient with asynchronous initialization:

Configuration	Value	Purpose
Connection URL	<code>config.mongodb.connection.url</code>	MongoDB server endpoint
Database	<code>config.mongodb.connection.database</code>	Target database (tradex-market)
Parser	<code>useNewUrlParser: true</code>	Use new URL parser

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/dbConnection.ts#L24-L31>

[src/models/db/dbConnection.ts24-31](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/dbConnection.ts#L24-L31) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/dbConnection.ts#L24-L31>)

Redis Service Implementation

The Redis service provides a comprehensive caching layer with type-safe serialization and categorized data storage:

Data Categories and Keys

The system defines multiple categories for organizing cached data:



Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L15-L58>

[src/init/RedisService.ts15-58](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L15-L58) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L15-L58>)

Data Type Serialization

The Redis service implements a type-aware serialization system that preserves JavaScript data types:

Data Type	Prefix	Serialization Method
<code>null</code>	<code>'b'</code>	Direct string conversion
<code>undefined</code>	<code>'a'</code>	Direct string conversion
<code>boolean</code>	<code>'0'</code>	<code>'1'</code> for true, <code>'0'</code> for false
<code>string</code>	<code>'1'</code>	Direct string storage
<code>number</code>	<code>'2'</code>	String conversion
<code>Date</code>	<code>'3'</code>	Timestamp (<code>getTime()</code>)
<code>object</code>	<code>'4'</code>	<code>JSON.stringify()</code>

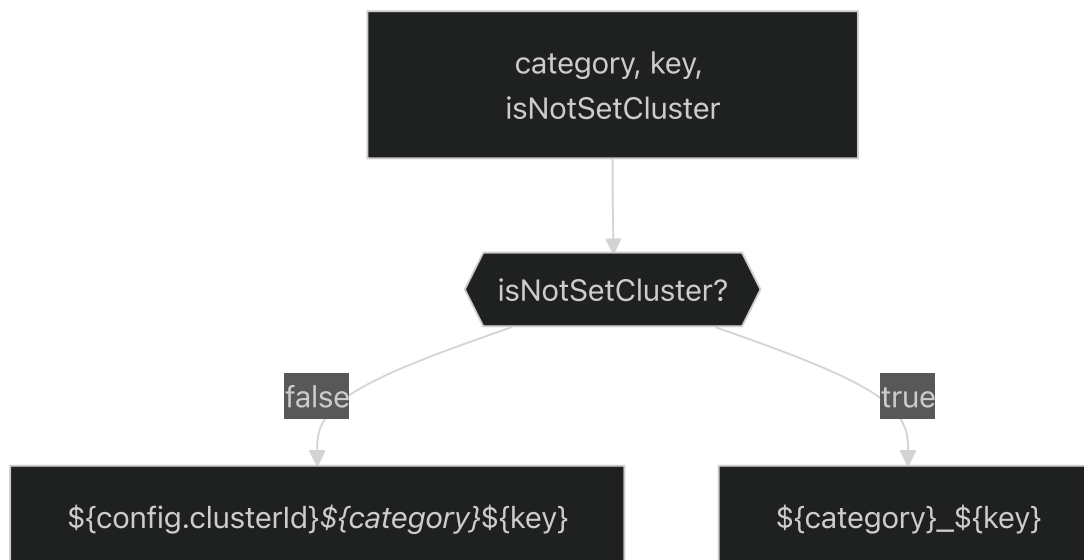
Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L5-L13>

[src/init/RedisService.ts5-13](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L5-L13) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L5-L13>)

[src/init/RedisService.ts87-102](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L87-L102) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L87-L102>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L87-L102>)

Key Generation Strategy

Redis keys are generated with cluster awareness to support distributed deployments:



Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L182-L192>

[src/init/RedisService.ts182-192](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L182-L192) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L182-L192>)

Cache Duration and TTL Management

The system implements configurable Time-To-Live (TTL) settings:

- **Default Duration:** `config.defaultLongRedisDuration` (7 days)
- **Custom Duration:** Configurable per operation via `durationInSeconds` parameter

- **Expiration Policy:** `EX` command with automatic key expiration

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L103-L113>)

[src/init/RedisService.ts#L103-L113](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L103-L113) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L103-L113>)

Error Handling and Key Management

The Redis service includes robust error handling mechanisms:

- **KeyNotExist Error:** Custom exception for missing keys[

[src/init/RedisService.ts#L60-L64](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L60-L64)](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L60-L64>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L60-L64>))

- **Connection Error Handling:** Redis client error event logging[

[src/init/RedisService.ts#L72](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L72-L72)](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L72-L72>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L72-L72>))

- **Promise-based Operations:** All operations return promises with proper error propagation

Hash Operations Support

The service supports Redis hash operations for complex data structures:

- **Hash Get:** `hget<T>(key: string, field: string)` with type deserialization
- **Hash Set:** `hset<T>(key: string, field: string, value: T)` with type serialization
- **Hash Get All:** `hgetall<T>(key: string)` returning arrays of typed values
- **Date Parsing:** Automatic date format recognition in JSON parsing

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L194-L318>)

[src/init/RedisService.ts#L194-318](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L194-L318) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L194-L318>)

Data Models

Relevant source files

- [
[src/models/db/IFinancialStatement.ts](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IFinancialStatement.ts)](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IFinancialStatement.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IFinancialStatement.ts>))
- [
[src/models/db/IStockNews.ts](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockNews.ts)](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockNews.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockNews.ts>))
- [
[src/models/db/IStockStockNews.ts](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockStockNews.ts)](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockStockNews.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockStockNews.ts>))
- [

src/models/db/ISymbolInfo.ts](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts>)

- [

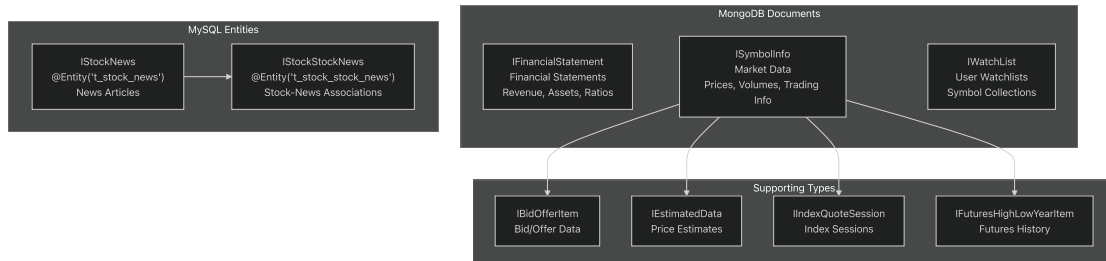
src/models/db/IWatchList.ts](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IWatchList.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IWatchList.ts>)

This document provides comprehensive documentation of the data structures and models used throughout the viet-stock-bridge system. These models define the schema for financial data, stock information, news articles, and user-generated content stored across MongoDB, MySQL, Redis, and MinIO storage systems.

For information about data access patterns and repository implementations, see [Data Access Layer](#). For caching strategies and data persistence patterns, see [Caching Strategy](#).

Data Model Architecture Overview

The system employs a multi-database strategy with different data models optimized for specific storage systems and use cases.



Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IFinancialStatement.ts#L1-L25>)

[src/models/db/IFinancialStatement.ts1-25](#) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IFinancialStatement.ts#L1-L25>)

src/models/db/ISymbolInfo.ts1-188](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts#L1-L188>) [([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts#L1-L188))

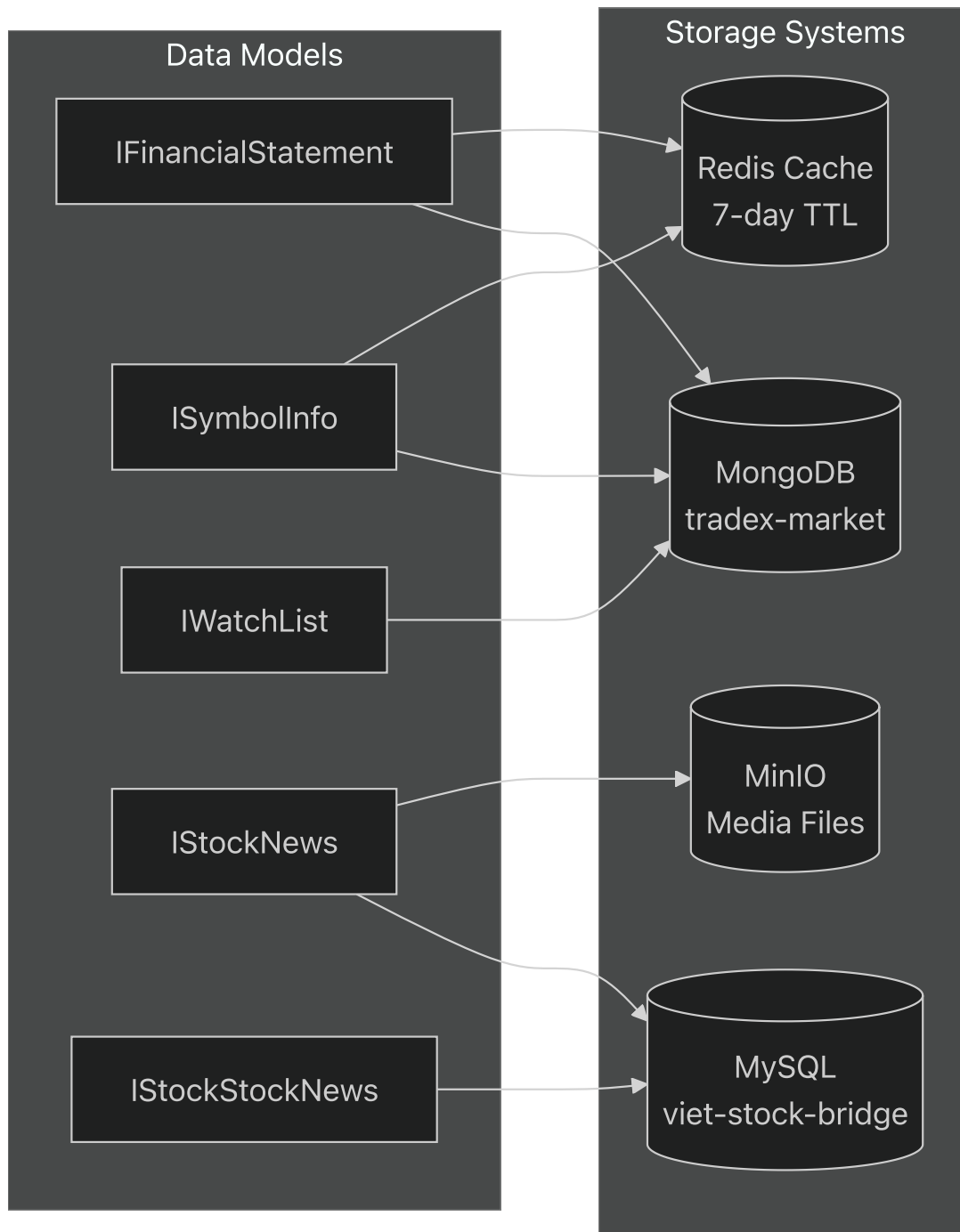
src/models/db/IWatchList.ts1-11](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IWatchList.ts#L1-L11>) [([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IWatchList.ts#L1-L11))

src/models/db/IStockNews.ts1-24](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockNews.ts#L1-L24>) [([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockNews.ts#L1-L24))

src/models/db/IStockStockNews.ts1-10](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockStockNews.ts#L1-L10>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockStockNews.ts#L1-L10>)

Storage System Mapping

Different data models are optimized for specific storage systems based on their data access patterns and structural requirements.



Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IFinancialStatement.ts#L1-L25>

[src/models/db/IFinancialStatement.ts1-25](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IFinancialStatement.ts#L1-L25) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IFinancialStatement.ts#L1-L25>

[src/models/db/ISymbolInfo.ts1-188](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts#L1-L188) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts#L1-L188> [%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts#L1-L188)

[src/models/db/IWatchList.ts1-11](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IWatchList.ts#L1-L11) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IWatchList.ts#L1-L11> [%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IWatchList.ts#L1-L11)

[src/models/db/IStockNews.ts1-24](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockNews.ts#L1-L24) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockNews.ts#L1-L24> [%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockNews.ts#L1-L24)

[bridge/blob/d4b51eda/src/models/db/IStockNews.ts#L1-L24\)%5B](#)

src/models/db/IStockStockNews.ts1-10](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockStockNews.ts#L1-L10> (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockStockNews.ts#L1-L10>))

MongoDB Document Models

IFinancialStatement

The `IFinancialStatement` interface represents comprehensive financial statement data for Vietnamese stocks, including income statement, balance sheet, and cash flow metrics.

Property	Type	Description
<code>_id</code>	string	MongoDB document identifier
<code>stock</code>	string	Stock symbol code
<code>year</code>	string	Financial year
<code>quarter</code>	string	Financial quarter
<code>revenue</code>	number	Total revenue
<code>gross_profit</code>	number	Gross profit
<code>operating_income</code>	number	Operating income
<code>net_income</code>	number	Net income
<code>eps</code>	number	Earnings per share
<code>total_assets</code>	number	Total assets
<code>total_liabilities</code>	number	Total liabilities
<code>total_equity</code>	number	Total shareholders' equity
<code>net_cash_from_operating_activities</code>	number	Operating cash flow
<code>net_cash_from_investing_activities</code>	number	Investing cash flow
<code>net_cash_from_financing_activities</code>	number	Financing cash flow
<code>pe</code>	number	Price-to-earnings ratio
<code>pb</code>	number	Price-to-book ratio
<code>roe</code>	number	Return on equity
<code>roa</code>	number	Return on assets
<code>bvps</code>	number	Book value per share
<code>created_at</code>	Date	Document creation timestamp
<code>updated_at</code>	Date	Document update timestamp

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IFinancialStatement.ts#L1-L25>.

[src/models/db/IFinancialStatement.ts1-25](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IFinancialStatement.ts#L1-L25) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IFinancialStatement.ts#L1-L25>.

ISymbolInfo

The `ISymbolInfo` interface provides comprehensive market data for stocks, futures, covered warrants, and indices. This is the most complex data model in the system, supporting multiple instrument types.

Core Symbol Properties

Property	Type	Description
<code>_id</code>	string	MongoDB document identifier
<code>code</code>	string	Symbol code
<code>type</code>	string	Instrument type
<code>name</code>	string	Vietnamese name
<code>nameEn</code>	string	English name
<code>marketType</code>	string	Market classification
<code>securitiesType</code>	string	Securities type classification

Market Data Properties

Property	Type	Description
<code>open</code>	number	Opening price
<code>high</code>	number	High price
<code>low</code>	number	Low price
<code>last</code>	number	Last traded price
<code>change</code>	number	Price change
<code>rate</code>	number	Change rate percentage
<code>tradingVolume</code>	number	Trading volume
<code>tradingValue</code>	number	Trading value
<code>time</code>	string	Last update time

Advanced Market Data

The model includes extensive properties for different instrument types:

- **Price Limits:** `ceilingPrice`, `floorPrice`, `referencePrice`
- **Bid/Offer Data:** `bidOfferList` array of `IBidOfferItem`
- **Foreign Trading:** `foreignerBuyVolume`, `foreignerSellVolume`, `foreignerTotalRoom`
- **Futures Specific:** `openInterest`, `maturityDate`, `theoryPrice`
- **Covered Warrants:** `exercisePrice`, `exerciseRatio`, `underlyingSymbol`

- **Index Data:** `upCount`, `downCount`, `indexSession` array

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts#L1-L188>

[src/models/db/ISymbolInfo.ts1-188](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts#L1-L188) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts#L1-L188>)

IWatchList

The `IWatchList` interface represents user-created symbol collections for portfolio tracking.

Property	Type	Description
<code>_id</code>	string	MongoDB document identifier
<code>username</code>	string	User identifier
<code>watchlistName</code>	string	Watchlist display name
<code>order</code>	number	Display order
<code>symbols</code>	string[]	Array of symbol codes
<code>createdAt</code>	Date	Creation timestamp
<code>updatedAt</code>	Date	Last update timestamp
<code>deletedAt</code>	Date	Soft delete timestamp

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IWatchList.ts#L1-L11>

[src/models/db/IWatchList.ts1-11](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IWatchList.ts#L1-L11) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IWatchList.ts#L1-L11>)

MySQL Entity Models

IStockNews

The `IStockNews` class is a TypeORM entity representing stock news articles stored in MySQL.

```
@Entity('t_stock_news')
```

IStockNews

```
+number id  
+string language  
+string title  
+string head  
+string url  
+string publishTime  
+string content  
+string author  
+string source
```

Property	Type	Annotations	Description
id	number	@PrimaryColumn()	News article identifier
language	string	@PrimaryColumn()	Language code (composite key)
title	string	@Column()	Article title
head	string	@Column()	Article headline
url	string	@Column()	Source URL
publishTime	string	@Column({ name: 'publish_time' })	Publication timestamp
content	string	@Column()	Article content
author	string	@Column()	Article author
source	string	@Column()	News source

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockNews.ts#L1-L24>.

[src/models/db/IStockNews.ts1-24](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockNews.ts#L1-L24) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockNews.ts#L1-L24>.

IStockStockNews

The `IStockStockNews` class represents the many-to-many relationship between stocks and news articles.

```
@Entity('t_stock_stock_news')
```

IStockStockNews

```
+number stockNewsId  
+string stockCode
```

Property	Type	Annotations	Description
stockNewsId	number	@PrimaryColumn()	Foreign key to IStockNews.id
stockCode	string	@PrimaryColumn()	Stock symbol code

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockStockNews.ts#L1-L10>

[src/models/db/IStockStockNews.ts1-10](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockStockNews.ts#L1-L10) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockStockNews.ts#L1-L10>).

Supporting Interface Types

IBidOfferItem

Represents bid and offer price levels in the order book.

Property	Type	Description
<code>bidPrice</code>	number	Bid price level
<code>bidVolume</code>	number	Volume at bid price
<code>bidVolumeChange</code>	number	Change in bid volume
<code>offerPrice</code>	number	Offer price level
<code>offerVolume</code>	number	Volume at offer price
<code>offerVolumeChange</code>	number	Change in offer volume

IEstimatedData

Price estimation data for ATO/ATC sessions.

Property	Type	Description
<code>ceilingPrice</code>	number	Maximum allowed price
<code>floorPrice</code>	number	Minimum allowed price

IIndexQuoteSession

Index performance data for specific trading sessions.

Property	Type	Description
<code>last</code>	number	Last index value
<code>change</code>	number	Index change
<code>rate</code>	number	Change rate percentage
<code>tradingVolume</code>	number	Session trading volume
<code>tradingValue</code>	number	Session trading value

IFuturesHighLowYearItem

Historical high/low data for futures contracts.

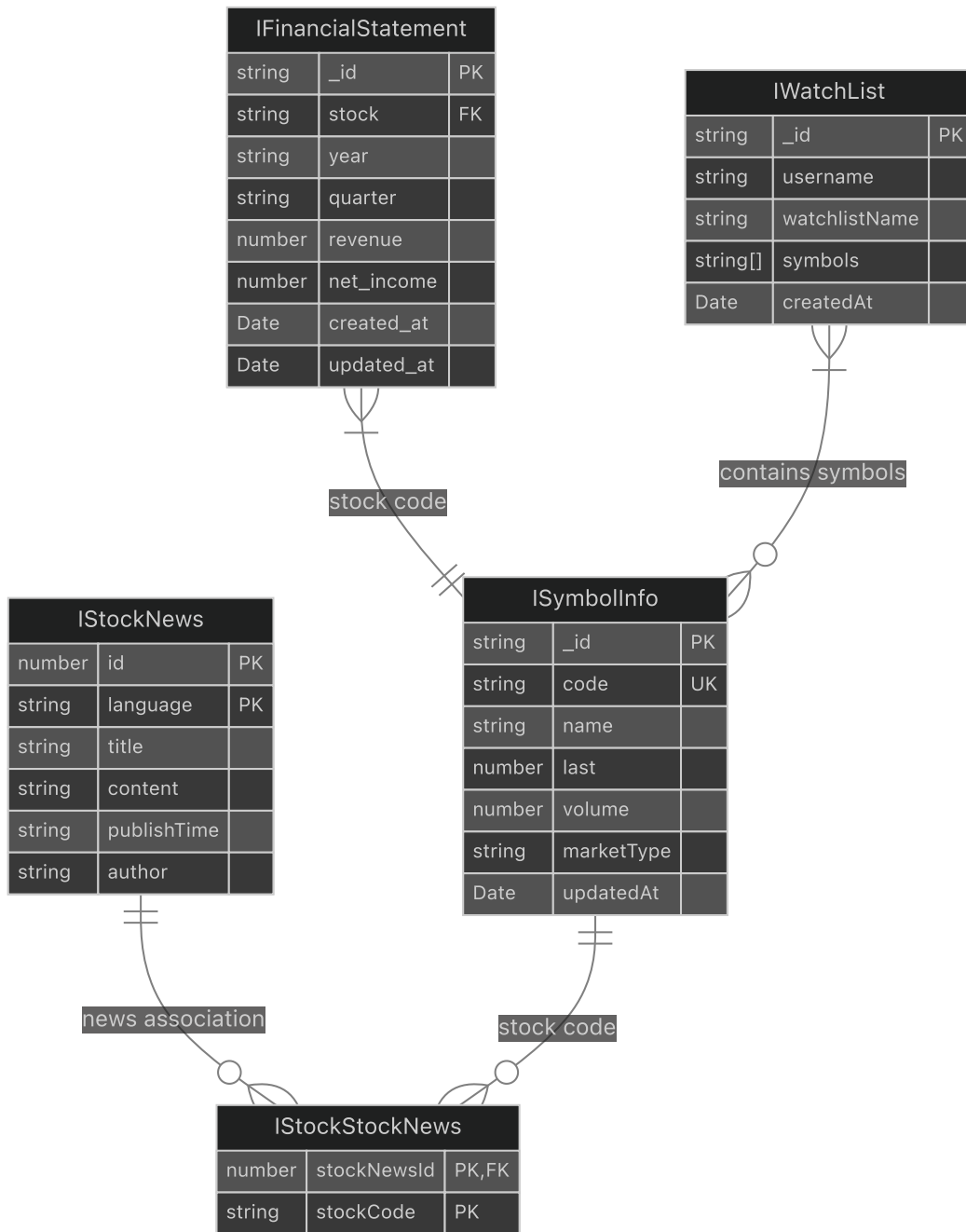
Property	Type	Description
highPrice	number	Highest price
dateOfHighPrice	string	Date of high price
lowPrice	number	Lowest price
dateOfLowPrice	string	Date of low price

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts#L160-L188>)

[src/models/db/ISymbolInfo.ts160-188](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts#L160-L188) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts#L160-L188>).

Data Model Relationships

The relationships between data models support complex financial data queries and associations.



Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IFinancialStatement.ts#L1-L25>)

[src/models/db/IFinancialStatement.ts1-25](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IFinancialStatement.ts#L1-L25) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IFinancialStatement.ts#L1-L25>)

[src/models/db/ISymbolInfo.ts1-188](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts#L1-L188) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/ISymbolInfo.ts#L1-L188>)

[src/models/db/IWatchList.ts1-11](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IWatchList.ts#L1-L11) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IWatchList.ts#L1-L11>)

[src/models/db/IStockNews.ts1-24](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockNews.ts#L1-L24) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockNews.ts#L1-L24>)

src/models/db/IStockStockNews.ts1-10](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockStockNews.ts#L1-L10>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/db/IStockStockNews.ts#L1-L10>))

Data Access Layer

Relevant source files

- [

 src/models/repo/QuarterFinancialStatementRepository.ts](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts>))
- [

 src/models/repo/WatchListRepository.ts](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/WatchListRepository.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/WatchListRepository.ts>))
- [

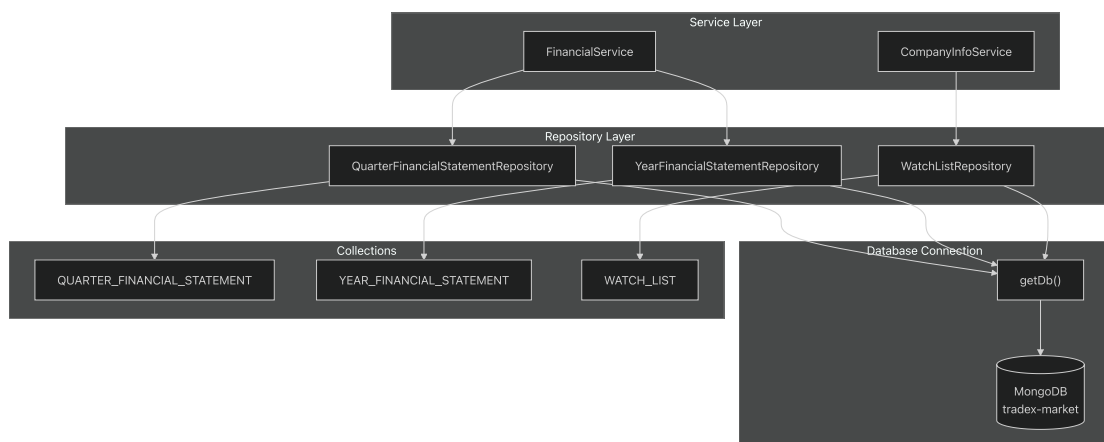
 src/models/repo/YearFinancialStatementRepository.ts](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts>))

Purpose and Scope

The Data Access Layer implements the repository pattern to provide a standardized interface for database operations in the viet-stock-bridge system. This layer abstracts MongoDB collection access and provides type-safe data persistence methods for financial statements and user watchlists. For information about the underlying data models, see [Data Models](#). For caching strategies that complement this data access layer, see [Caching Strategy](#).

Repository Pattern Implementation

The system implements a repository pattern using TypeScript services with dependency injection. Each repository is responsible for a specific data domain and provides consistent query and persistence interfaces.



Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L1-L10>).

[src/models/repo/QuarterFinancialStatementRepository.ts1-10](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L1-L10) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L1-L10>)

src/models/repo/YearFinancialStatementRepository.ts1-10](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L1-L10>)[[%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L1-L10)]

src/models/repo/WatchListRepository.ts1-10](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/WatchListRepository.ts#L1-L10>)([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/WatchListRepository.ts#L1-L10))

Database Connection Management

All repositories access MongoDB through a centralized connection function `getDb()` imported from the database connection module. This provides a single point of database access configuration.

Repository Class	Collection Accessed	Primary Interface
<code>QuarterFinancialStatementRepository</code>	<code>QUARTER_FINANCIAL_STATEMENT</code>	<code>IFinancialStatement</code>
<code>YearFinancialStatementRepository</code>	<code>YEAR_FINANCIAL_STATEMENT</code>	<code>IFinancialStatement</code>
<code>WatchListRepository</code>	<code>WATCH_LIST</code>	<code>IWatchList</code>

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L3-L11>).

[src/models/repo/QuarterFinancialStatementRepository.ts3-11](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L3-L11) ([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L3-L11))

src/models/repo/YearFinancialStatementRepository.ts3-11](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L3-L11>)[[%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L3-L11)]

src/models/repo/WatchListRepository.ts3-10](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/WatchListRepository.ts#L3-L10>)([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/WatchListRepository.ts#L3-L10))

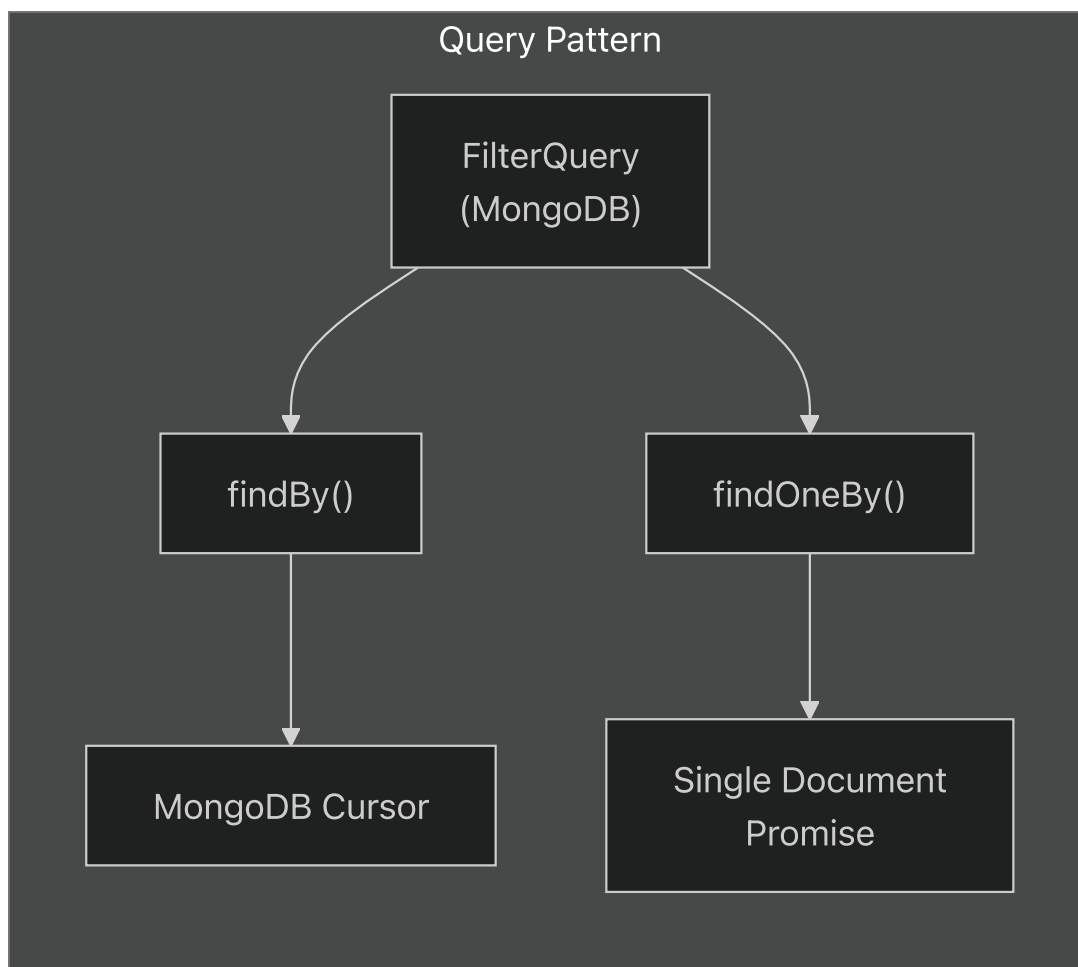
Collection Access Patterns

Each repository provides standardized collection access through consistent method signatures:

Query Operations

All repositories implement two primary query methods:

- `findBy(query: FilterQuery)` - Returns a MongoDB cursor for multiple documents
- `findOneBy(query: FilterQuery)` - Returns a single document promise



Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L10-L20>).

[src/models/repo/QuarterFinancialStatementRepository.ts10-20](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L10-L20) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L10-L20>)[

[src/models/repo/YearFinancialStatementRepository.ts10-20](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L10-L20)](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L10-L20>)[([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L10-L20)).

[src/models/repo/WatchListRepository.ts9-19](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/WatchListRepository.ts#L9-L19)](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/WatchListRepository.ts#L9-L19> (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/WatchListRepository.ts#L9-L19>)).

Data Persistence Operations

Financial statement repositories implement sophisticated persistence logic with upsert operations:

Quarter Financial Statements

The `QuarterFinancialStatementRepository.save()` method handles bulk financial statement persistence:

- Generates composite keys: `${stock}_${year}_${quarter}`
- Performs upsert operations with metadata tracking
- Logs modification and insertion counts

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L22-L44>)

[src/models/repo/QuarterFinancialStatementRepository.ts22-44](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L22-L44) (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L22-L44)

Year Financial Statements

The `YearFinancialStatementRepository.save()` method follows similar patterns:

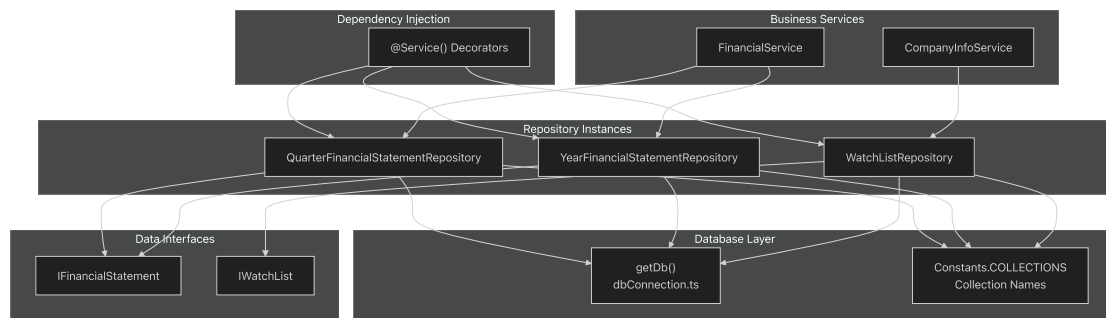
- Generates composite keys: `${stock}_${year}`
- Uses MongoDB `$set`, `$setOnInsert`, and `$currentDate` operators
- Tracks operation counts for logging

Sources: (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L22-L48)

[src/models/repo/YearFinancialStatementRepository.ts22-48](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L22-L48) (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L22-L48)

Repository Integration Architecture

The repositories integrate with the broader system through dependency injection and standardized interfaces:



Sources: (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L1-L8)

[src/models/repo/QuarterFinancialStatementRepository.ts1-8](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L1-L8) (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L1-L8)

[src/models/repo/YearFinancialStatementRepository.ts1-8](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L1-L8) (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L1-L8)

[src/models/repo/WatchListRepository.ts1-7](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/WatchListRepository.ts#L1-L7) (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/WatchListRepository.ts#L1-L7)

Logging and Monitoring

The financial statement repositories include comprehensive logging for database operations:

- Operation success tracking through `modifiedCount` and `upsertedCount`
- Structured logging using `tradex-common` Logger
- Collection-specific log messages with stock identifiers

Sources: (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L6-L42)

[src/models/repo/QuarterFinancialStatementRepository.ts6-42](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L6-L42) (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L6-L42)

src/models/repo/YearFinancialStatementRepository.ts6-46](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L6-L46> (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L6-L46>))

Type Safety and Error Handling

The repositories leverage TypeScript's type system for compile-time safety:

- Generic type parameters for MongoDB collection operations
- Strongly typed interfaces for data models
- FilterQuery types from MongoDB driver for query safety

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L2-L16>)

[src/models/repo/QuarterFinancialStatementRepository.ts2-16](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L2-L16) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/QuarterFinancialStatementRepository.ts#L2-L16>)

src/models/repo/YearFinancialStatementRepository.ts2-16](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L2-L16>) ([%5B">https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L2-L16](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/YearFinancialStatementRepository.ts#L2-L16)%)

src/models/repo/WatchListRepository.ts2-15](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/WatchListRepository.ts#L2-L15> (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/repo/WatchListRepository.ts#L2-L15>))

Caching Strategy

Relevant source files

- [

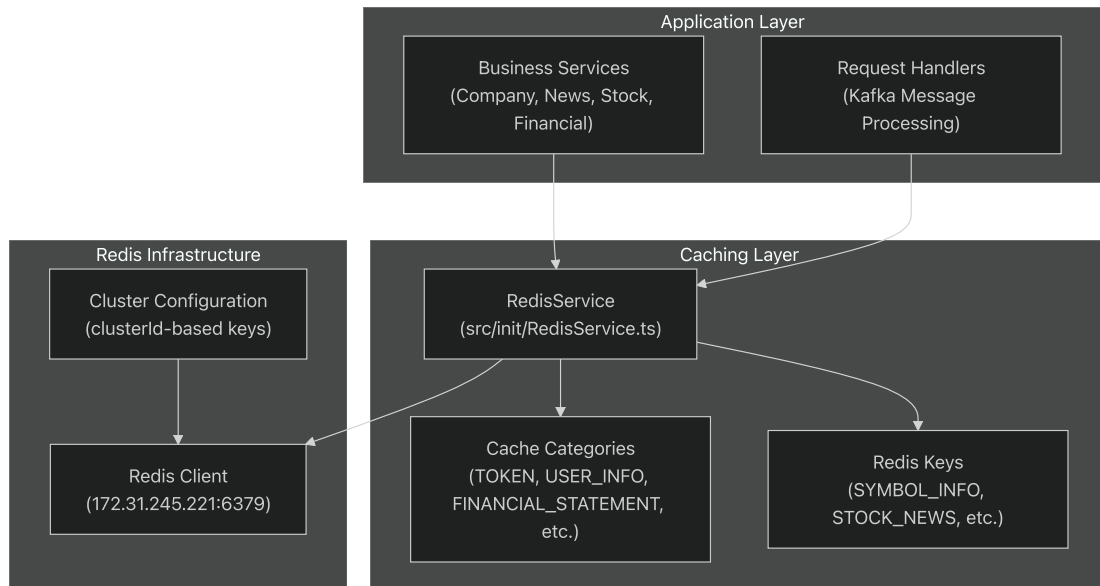
src/init/RedisService.ts](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts> (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts>))

This document covers the Redis-based caching implementation used throughout the viet-stock-bridge system for performance optimization and data persistence. The caching strategy includes data categorization, type-safe serialization, cluster-aware key management, and TTL policies for different types of financial data.

For database storage patterns, see [Data Access Layer](#). For data structure definitions, see [Data Models](#).

Architecture Overview

The system implements a centralized caching strategy using Redis as the primary cache store. The `RedisService` class serves as the main abstraction layer for all caching operations, providing type-safe data storage and retrieval with automatic serialization.



Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L1-L320>

<src/init/RedisService.ts#L320> <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L1-L320>

Data Organization and Categories

The caching system organizes data into predefined categories that reflect different business domains and usage patterns. Each category serves a specific functional area of the application.

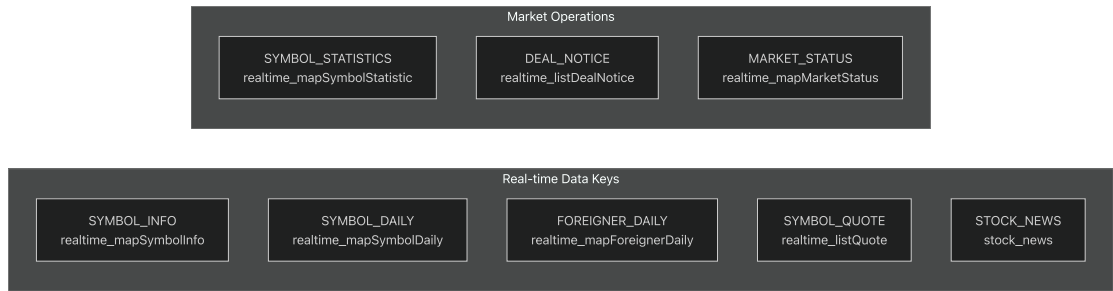
Cache Categories

The system defines cache categories through the `Category` constant, grouping related data types for efficient management:

Category	Purpose	Usage Pattern
<code>TOKEN</code>	Authentication tokens	Short-term, high-frequency access
<code>USER_INFO</code>	User profile data	Medium-term, moderate access
<code>FINANCIAL_STATEMENT_*</code>	Financial data by period/type	Long-term, analytical queries
<code>BANK_INFO</code>	Banking information	Long-term reference data
<code>TIME_QUERY</code>	Temporal queries	Short-term optimization

Real-time Market Data Keys

The `REDIS_KEY` constant defines specific keys for real-time market data caching:



Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L15-L58>, [src/init/RedisService.ts15-58](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L15-L58) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L15-L58>

Data Type Handling and Serialization

The caching service implements a sophisticated type system that preserves JavaScript data types across Redis storage operations. This ensures type safety and data integrity for all cached values.

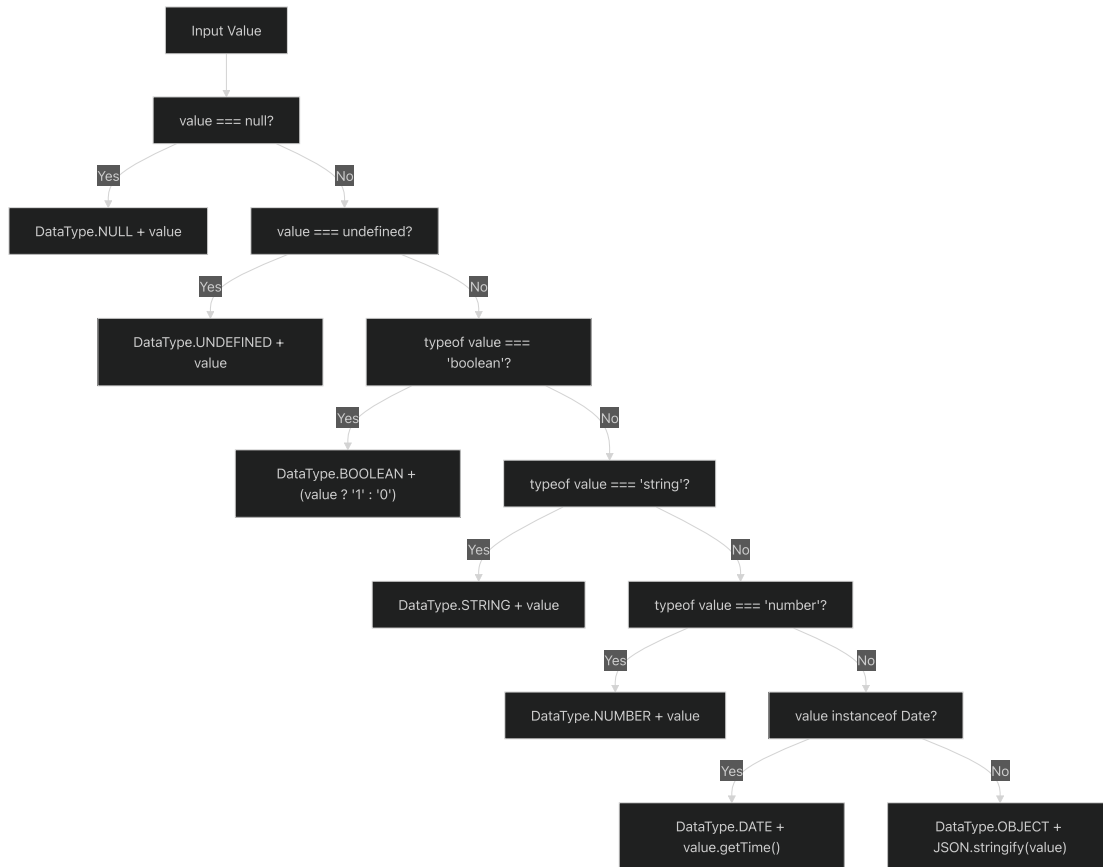
Type Encoding System

The service uses single-character prefixes to encode data types during serialization:

Data Type	Prefix	Encoding Logic
undefined	'a'	Preserved as string representation
null	'b'	Explicit null handling
boolean	'0'	'1' for true, '0' for false
string	'1'	Direct string storage
number	'2'	String representation of numeric value
Date	'3'	Millisecond timestamp storage
object	'4'	JSON stringified representation

Serialization Process

The serialization logic in the `set` method handles type detection and encoding:



Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L5-L13>.

[src/init/RedisService.ts5-13](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L5-L13) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L5-L13>

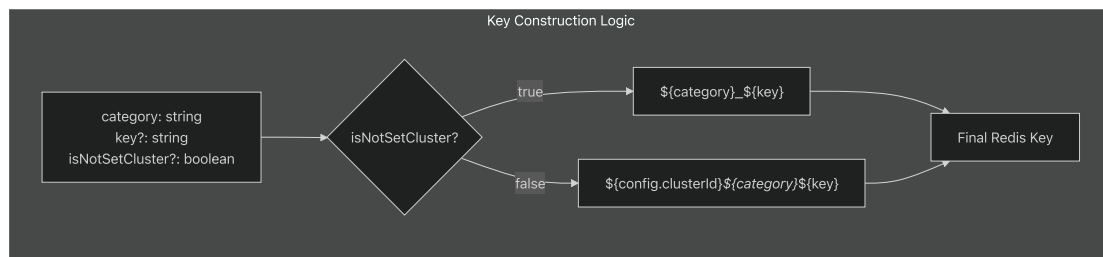
[src/init/RedisService.ts80-113](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L80-L113) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L80-L113> <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L80-L113>

Key Management and Clustering

The system implements cluster-aware key management to support distributed deployments and avoid key collisions across different application instances.

Key Generation Strategy

The `getRedisKey` method constructs Redis keys using a configurable clustering scheme:



Cluster Configuration

The clustering mechanism uses the `config.clusterId` to namespace keys, preventing conflicts in multi-instance deployments. This ensures data isolation between different application environments or instances.

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L182-L192>

[src/init/RedisService.ts#L182-192](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L182-L192) (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L182-L192)

Cache Operations

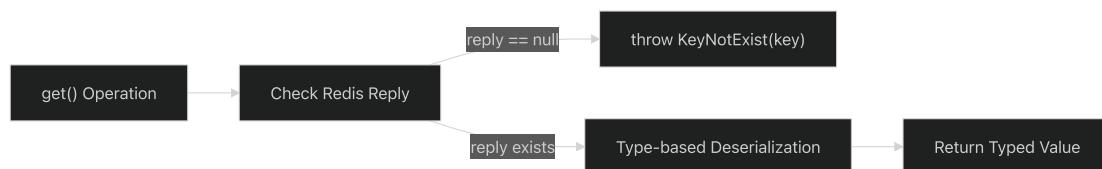
The `RedisService` class provides comprehensive Redis operations with type safety and error handling. Each operation maintains the type encoding system and handles Redis client communication.

Core Operations

Operation	Method	Purpose	Type Safety
Store	<code>set<T>()</code>	Cache data with TTL	Generic type preservation
Retrieve	<code>get<T>()</code>	Fetch typed data	Type-safe deserialization
Remove	<code>del()</code>	Delete by category/key	Boolean return
Hash Store	<code>hset<T>()</code>	Store hash field	Type encoding applied
Hash Retrieve	<code>hget<T>()</code>	Fetch hash field	Type-safe retrieval
Hash Get All	<code>hgetall<T>()</code>	Retrieve all hash fields	Array of typed values
Exists Check	<code>exists()</code>	Key existence test	Boolean result

Error Handling

The service implements custom error handling with the `KeyNotExist` exception for missing keys:



Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L60-L64>

[src/init/RedisService.ts#60-64](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L60-L64) (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L60-L64)[

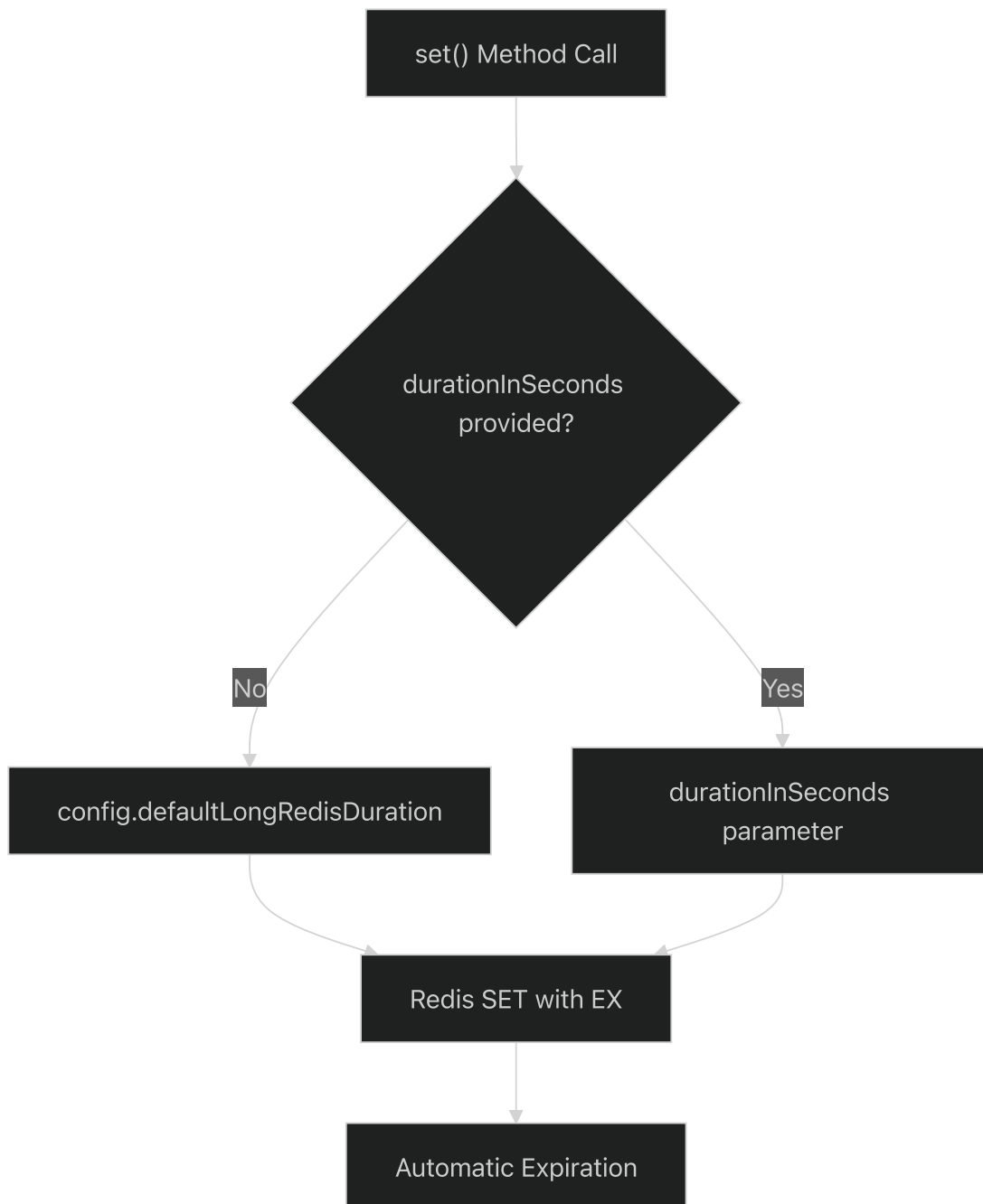
[src/init/RedisService.ts#123-158](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L123-L158)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L123-L158 (https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L123-L158))

TTL and Expiration Policies

The caching system implements configurable Time-To-Live (TTL) policies to balance data freshness with performance optimization.

Default Duration Configuration

The system uses `config.defaultLongRedisDuration` as the default TTL for cached items, with override capability per operation:



Expiration Strategy

The Redis `EX` parameter ensures automatic cleanup of expired entries, preventing memory bloat and ensuring data freshness for time-sensitive financial information.

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L103-L112>

[src/init/RedisService.ts#L103-L112](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L103-L112) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L103-L112>

Service Initialization

The `RedisService` is configured as a TypeDI service and requires initialization before use. The initialization process establishes the Redis client connection and sets up error handling.

Initialization Process

The `init()` method establishes the Redis connection using configuration parameters:

1. Creates Redis client with host, port, and options from configuration
2. Registers error handler for connection issues
3. Returns Promise that resolves when client is ready
4. Provides connection status feedback through events

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L70-L78>

[src/init/RedisService.ts70-78](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L70-L78) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/init/RedisService.ts#L70-L78>

System Constants and Enumerations

Relevant source files

- [[src/constants/Constants.ts](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts>)
- [[src/constants/SellBuyType.ts](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/SellBuyType.ts)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/SellBuyType.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/SellBuyType.ts>)
- [[src/constants/enum.ts](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts>)

This document covers all system-wide constants, enumerations, and type definitions used throughout the viet-stock-bridge application. These constants provide standardized values for financial data processing, trading operations, authentication workflows, and database operations.

For information about request data models, see page [6](#). For data models and structures, see page [4.1](#).

Overview

The system maintains constants in three primary files that serve different functional domains:

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts#L1-L61>

[src/constants/Constants.ts1-61](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts#L1-L61) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts#L1-L61>

[

[src/constants/enum.ts1-231](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L1-L231)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L1-L231>) [[%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L1-L231)] ([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L1-L231))

[src/constants/SellBuyType.ts1-5](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/SellBuyType.ts#L1-L5)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/SellBuyType.ts#L1-L5>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/SellBuyType.ts#L1-L5>)

Authentication and Security Constants

The `Constants` class provides string constants for authentication workflows, login attempts, and security operations.

Category	Constants	Purpose
Login Errors	<code>LOGIN_WRONG_PASSWORD_5_TIMES</code> , <code>LOGIN_WRONG_PASSWORD</code>	Track authentication failures
OTP Management	<code>WRONG_OTP</code> , <code>OTP_NOT_EXIST</code> , <code>OTP_GENERATE_TO_FAST</code>	One-time password validation
Account Operations	<code>ACCOUNT_CHANGE_PASSWORD</code> , <code>ACCOUNT_CHANGE HTS_PASSWORD</code>	Password management workflows
Authorization	<code>UNAUTHORIZED</code> , <code>WRONG_ACCOUNT_INFORMATION</code>	Access control

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts#L2-L47>)

[src/constants/Constants.ts2-47](#) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts#L2-L47>)

Trading and Order Management Constants

Order Operations

The system defines comprehensive constants for all trading operations:

Order Status					
0: Receipt	1: Send	2: Order Confirm	4: Full Filled	5: Partial Filled	X: Reject

Order Types				
LO: Limit Order	MP: Market Price	ATO: At The Open	ATC: At The Close	AON: All Or None

Order Lifecycle				
ORDER_PLACE_	ORDER_MODIFY_	ORDER_CANCEL_	ORDER_CONFIRM_	ORDER_HISTORY_

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts#L6-L15>)

[src/constants/Constants.ts6-15](#) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts#L6-L15>)

[

[src/constants/enum.ts108-125\]\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L108-L125\)\[](#) ([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L108-L125))

[src/constants/enum.ts145-154\]\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L145-L154](#) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L145-L154>))

Market and Trading Types

Constant Group	Values	Description
MARKET_TYPE	HOSE: '1', HNX: '2', UPCOM: '3'	Vietnamese stock exchanges
SELL_BUY_TYPE	SELL: 'SELL', BUY: 'BUY'	Transaction directions
MATCH_TYPE	MATCHED: '1', UNMATCHED: '2'	Order execution status
CHANNEL_TYPE	BOS: '00', HTS: '04', API: '30'	Trading channels

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L127-L176>).

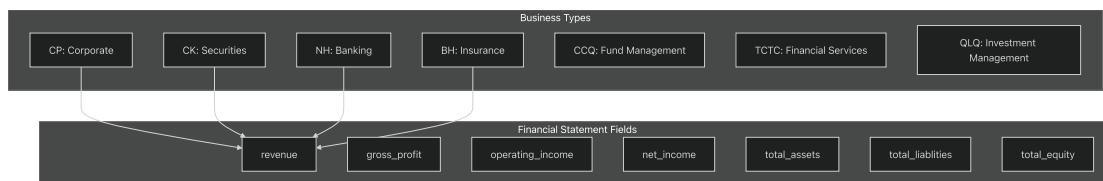
[src/constants/enum.ts#L127-L176](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L127-L176) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L127-L176>)

src/constants/SellBuyType.ts#L1-L5](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/SellBuyType.ts#L1-L5>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/SellBuyType.ts#L1-L5>))

Financial Data Constants

Business Type Classification

The system categorizes companies into business types with corresponding financial statement field mappings:



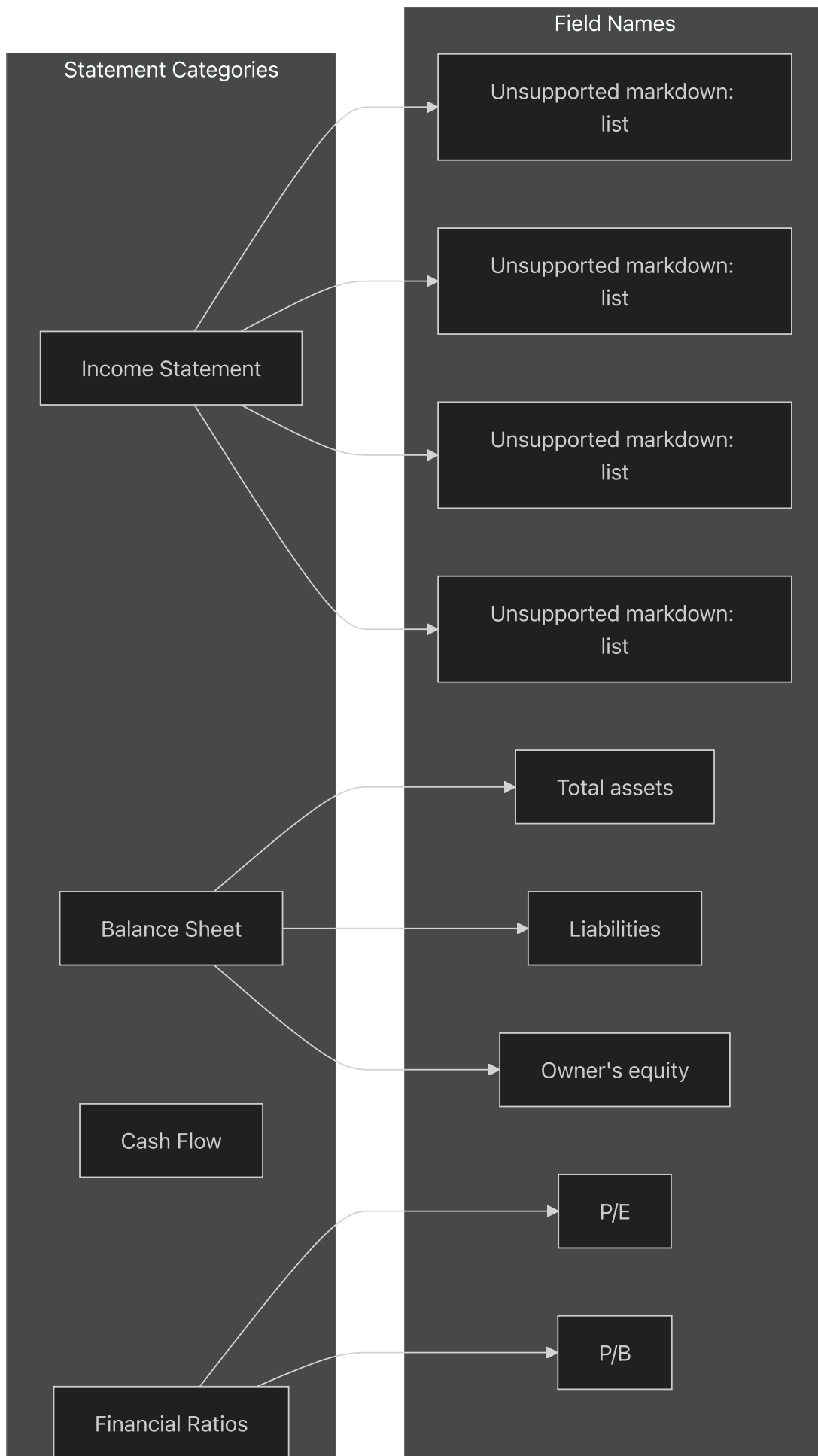
Each business type maps to specific field indices in the VietStock API responses. For example, corporate (`CP`) companies use index `3` for revenue, while securities (`CK`) companies use index `21` .

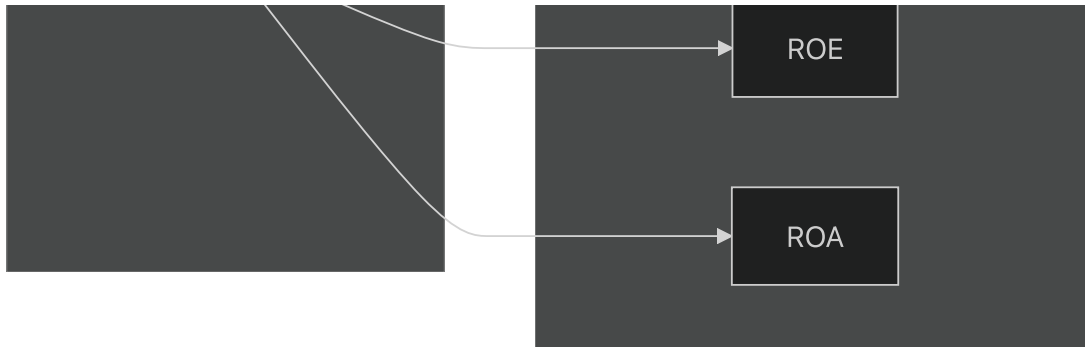
Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L18-L77>).

[src/constants/enum.ts#L18-77](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L18-L77) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L18-L77>).

Financial Statement Naming

The `FINANCIAL_STATEMENT_NAME` constant provides human-readable names for financial metrics:





Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L79-L96>)

[src/constants/enum.ts#L79-L96](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L79-L96) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L79-L96>).

Database and Collection Constants

MongoDB Collections

The system defines standardized collection names in the `COLLECTIONS` constant:

Collection	Purpose
<code>t_stock_news</code>	Stock news and articles
<code>t_watch_list</code>	User watchlists
<code>t_quarter_financial_statement</code>	Quarterly financial data
<code>t_year_financial_statement</code>	Annual financial data

Pagination Defaults

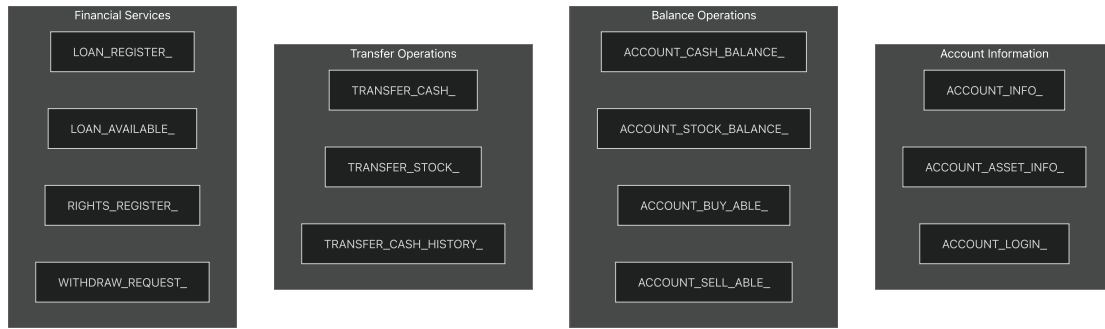
Constant	Value	Usage
<code>DEFAULT_PAGE_SIZE</code>	<code>'20'</code>	Standard API response size
<code>DEFAULT_OFFSET</code>	<code>'0'</code>	Starting offset for queries
<code>DEFAULT_PAGE_NUMBER_VIET_STOCK</code>	<code>1</code>	VietStock API page numbering
<code>DEFAULT_WATCH_LIST_NEWS_PAGE_SIZE</code>	<code>8</code>	Watchlist news items per page

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts#L49-L60>)

[src/constants/Constants.ts#L49-60](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts#L49-L60) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts#L49-L60>).

Account and Portfolio Constants

The system provides constants for account operations and portfolio management:



Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts#L16-L38>

[src/constants/Constants.ts16-38](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts#L16-L38) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/Constants.ts#L16-L38>

Type Definitions and Utilities

The system exports TypeScript type definitions for all enumeration values:

```
export type SellBuyType = typeof SELL_BUY_TYPE[keyof typeof SELL_BUY_TYPE];
export type OrderType = typeof ORDER_TYPE[keyof typeof ORDER_TYPE];
export type MarketType = typeof MARKET_TYPE[keyof typeof MARKET_TYPE];
```

Utility Functions

The `getKeyByValue` function provides reverse lookup functionality for constant objects:

```
export function getKeyByValue(value: string | number, obj: object) {
  const indexofs = Object.values(obj).indexOf(value);
  const key = Object.keys(obj)[indexofs];
  return key;
}
```

This enables finding enumeration keys when only the value is known, useful for API response processing.

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L204-L230>

[src/constants/enum.ts204-230](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L204-L230) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L204-L230>

Language and Localization

The system supports multiple languages through the `VIET_STOCK_LANG_CODE` enum:

Language	Code	VietStock API Value
Vietnamese	vi	'1'
English	en	'2'

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L98-L101>

[src/constants/enum.ts98-101](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L98-L101) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/constants/enum.ts#L98-L101>

Request Models

Relevant source files

- [`src/models/IContext.ts`](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts>)
- [`src/models/request/IGetBalanceSheetRequest.ts`](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/request/IGetBalanceSheetRequest.ts>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/request/IGetBalanceSheetRequest.ts>)

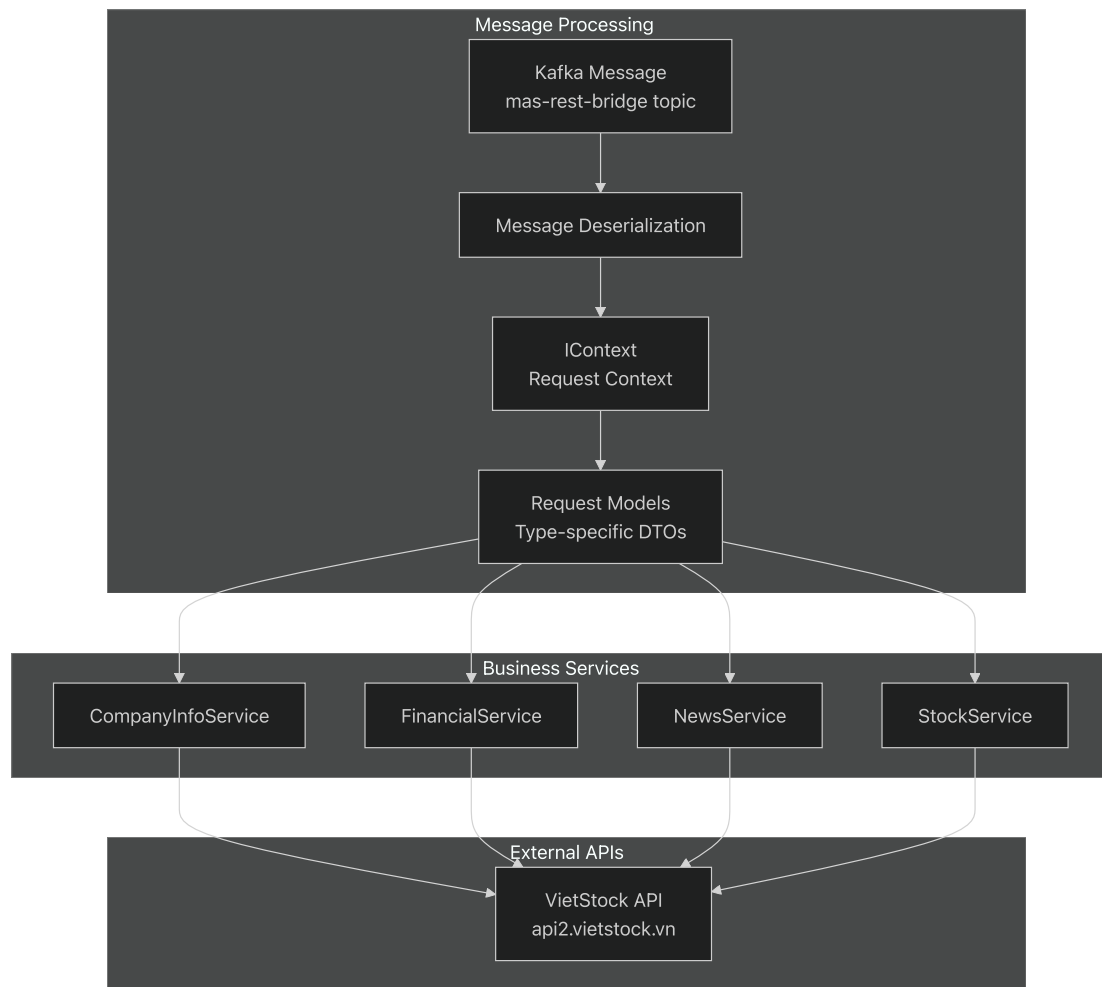
Purpose and Scope

This document covers the request model interfaces and data transfer objects (DTOs) used throughout the viet-stock-bridge system. These models define the structure of incoming requests processed via Kafka messages and specify the parameters required for various financial data operations.

Request models serve as the contract between external clients and the viet-stock-bridge service, ensuring type safety and consistent data structure for all API interactions. For information about the actual message processing implementation, see [Message Processing](#). For details about the data structures returned by the system, see [Data Models](#).

System Integration Context

Request models in the viet-stock-bridge system work within the Kafka-based message processing architecture. When messages arrive from the `mas-rest-bridge` topic, they are deserialized into these request interfaces before being routed to appropriate business services.



Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts#L1-L8>

[src/models/IContext.ts1-8](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts#L1-L8) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts#L1-L8>

[src/models/request/IGetBalanceSheetRequest.ts1-6](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/request/IGetBalanceSheetRequest.ts1-6) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/request/IGetBalanceSheetRequest.ts#L1-L6> <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/request/IGetBalanceSheetRequest.ts#L1-L6>

Core Request Context

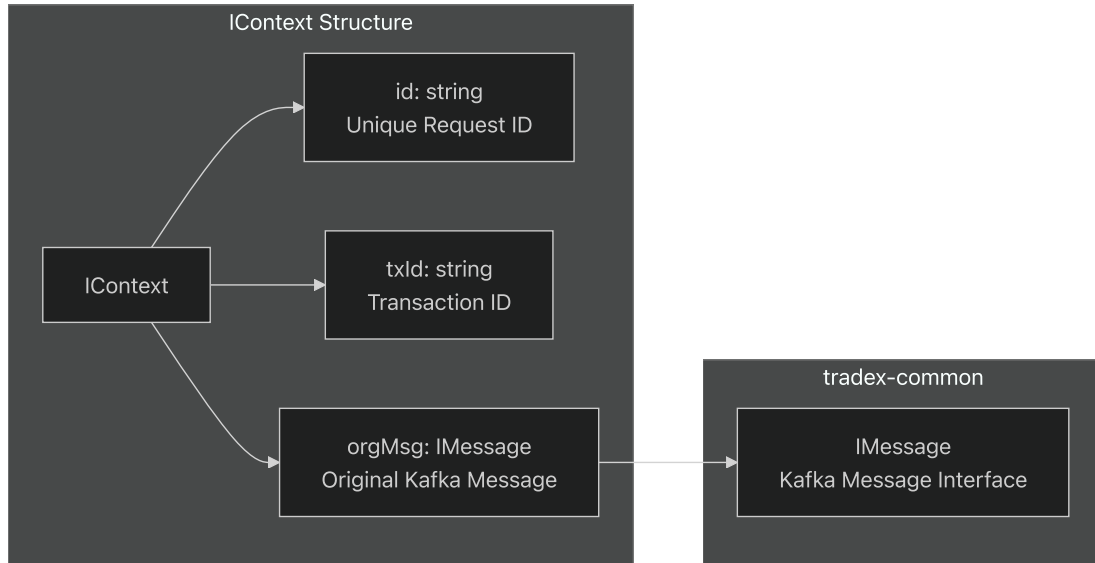
IContext Interface

The `IContext` interface provides the foundational structure for all request processing within the system. It encapsulates the essential metadata required for message handling and transaction tracking.

Property	Type	Purpose
<code>id</code>	<code>string</code>	Unique identifier for the request
<code>txId</code>	<code>string</code>	Transaction ID for tracking purposes
<code>orgMsg</code>	<code>IMessage</code>	Original Kafka message from tradex-common

The `IContext` interface serves as a wrapper around incoming Kafka messages, providing standardized access to request metadata across all service operations. The `orgMsg` property contains the original message structure

from the tradex-common library's Kafka module.



Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts#L1-L8>)

[src/models/IContext.ts1-8](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts#L1-L8) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts#L1-L8>)

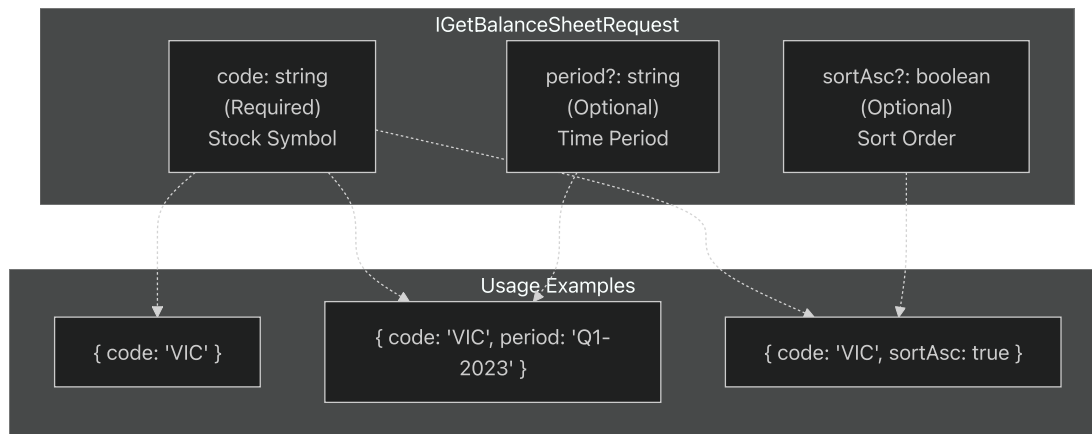
Financial Request Models

Balance Sheet Requests

The `IGetBalanceSheetRequest` interface defines the structure for requesting balance sheet data from the VietStock API. This model supports flexible querying with optional parameters for period filtering and result ordering.

Property	Type	Required	Description
<code>code</code>	<code>string</code>	Yes	Stock symbol or company code
<code>period</code>	<code>string</code>	No	Time period for financial data (e.g., "Q1-2023")
<code>sortAsc</code>	<code>boolean</code>	No	Sort results in ascending order when true

The request model supports both current and historical balance sheet data retrieval. When `period` is omitted, the system typically returns the most recent available data. The `sortAsc` parameter allows clients to control the ordering of time-series financial data.

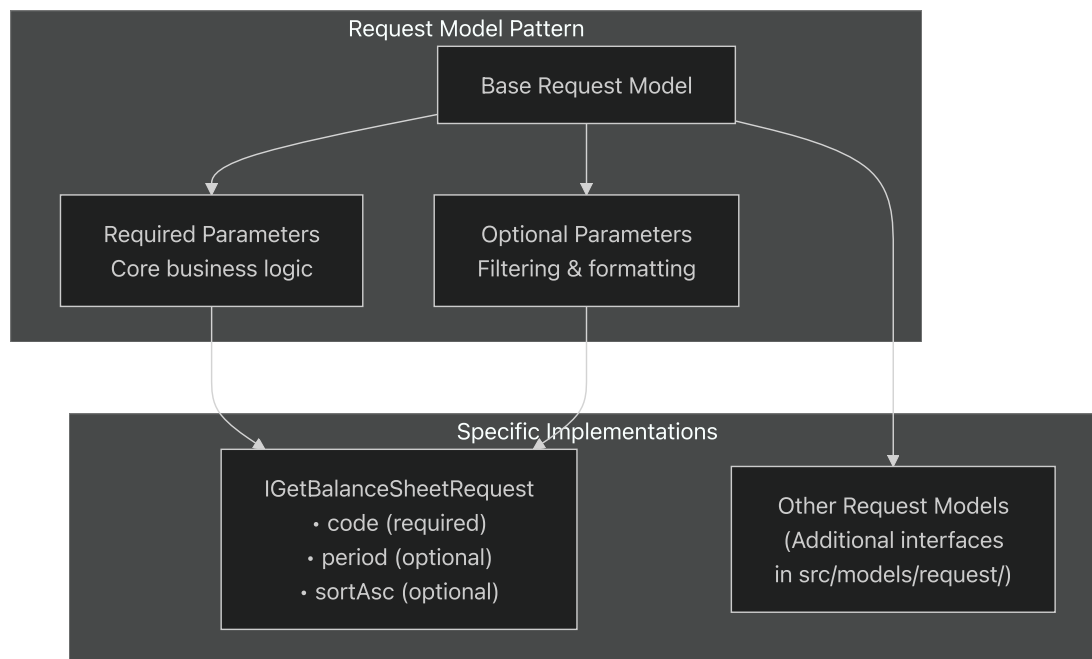


Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/request/IGetBalanceSheetRequest.ts#L1-L6>

[src/models/request/IGetBalanceSheetRequest.ts1-6](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/request/IGetBalanceSheetRequest.ts#L1-L6) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/request/IGetBalanceSheetRequest.ts#L1-L6>)

Request Model Architecture

The request models follow a consistent pattern where each interface defines the minimal required parameters for a specific operation, with optional parameters providing additional filtering and formatting capabilities.



Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/request/IGetBalanceSheetRequest.ts#L1-L6>

[src/models/request/IGetBalanceSheetRequest.ts1-6](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/request/IGetBalanceSheetRequest.ts#L1-L6) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/request/IGetBalanceSheetRequest.ts#L1-L6>)

Integration with Message Processing

Request models integrate seamlessly with the Kafka-based message processing system. The `RequestHandler` component deserializes incoming messages into appropriate request model instances before routing them to business services.

The type system ensures that each service receives properly structured request data, eliminating runtime errors related to missing or incorrectly typed parameters. This design pattern supports the system's high-throughput

financial data processing requirements while maintaining code reliability and maintainability.

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts#L1-L8>)

[src/models/IContext.ts1-8](#) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/IContext.ts#L1-L8>)

[src/models/request/IGetBalanceSheetRequest.ts1-6](#)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/request/IGetBalanceSheetRequest.ts#L1-L6>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/src/models/request/IGetBalanceSheetRequest.ts#L1-L6>)

Development Setup

Overview

Relevant source files

- [[.nvmrc](#)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc>)
- [[package-lock.json](#)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package-lock.json>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package-lock.json>)
- [[package.json](#)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json>)
- [[prettier.config.js](#)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/prettier.config.js>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/prettier.config.js>)

This document provides comprehensive guidance for setting up a development environment for the viet-stock-bridge application. It covers prerequisite software installation, dependency management, build toolchain configuration, and development workflow setup.

For information about deployment configuration, see [Deployment](#). For details about the build and packaging processes, see [Build and Packaging](#).

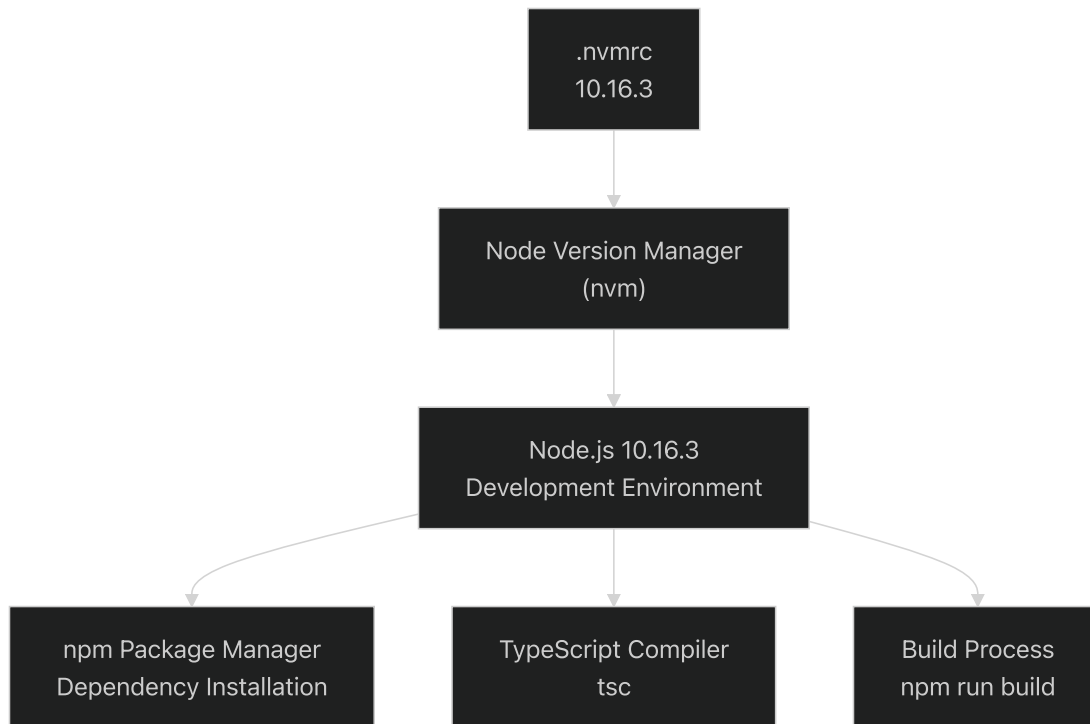
Prerequisites and Environment Configuration

The development environment requires specific Node.js version management and tooling setup to ensure consistency across different development machines.

Node.js Version Management

The project uses Node.js version `10.16.3` as specified in [.nvmrc1](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc#L1-L1) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc#L1-L1>)

which ensures compatibility with the target deployment environment. This version constraint aligns with the Docker container base image `node:10-alpine` used in production.



Development Environment Setup Flow

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc#L1-L1>

* [.nvmrc1](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc#L1-L1) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc#L1-L1>]

package.json62-68](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L62-L68>
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L62-L68>))*

Package Management and Dependencies

The project uses npm for dependency management with locked versions to ensure reproducible builds across environments.

Core Dependencies Structure



Package Dependency Architecture

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L22-L55>

* [package.json22-55](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L22-L55) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L22-L55>]

package-lock.json1-6](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package-lock.json#L1-L6>
<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package-lock.json#L1-L6>))*

Installation Process

The dependency installation follows standard npm workflows with additional TypeScript compilation requirements:

Command	Purpose	Dependencies
<code>npm install</code>	Install all dependencies	package.json, package-lock.json
<code>npm run check</code>	TypeScript and lint validation	gts, tslint
<code>npm run compile</code>	TypeScript compilation	typescript compiler
<code>npm run fix</code>	Automatic code formatting	gts, prettier

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L5-L18>

[package.json5-18](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L5-L18) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L5-L18>)

Development Toolchain Configuration

The project implements a comprehensive development toolchain with automated code quality enforcement and formatting.

Code Formatting and Style

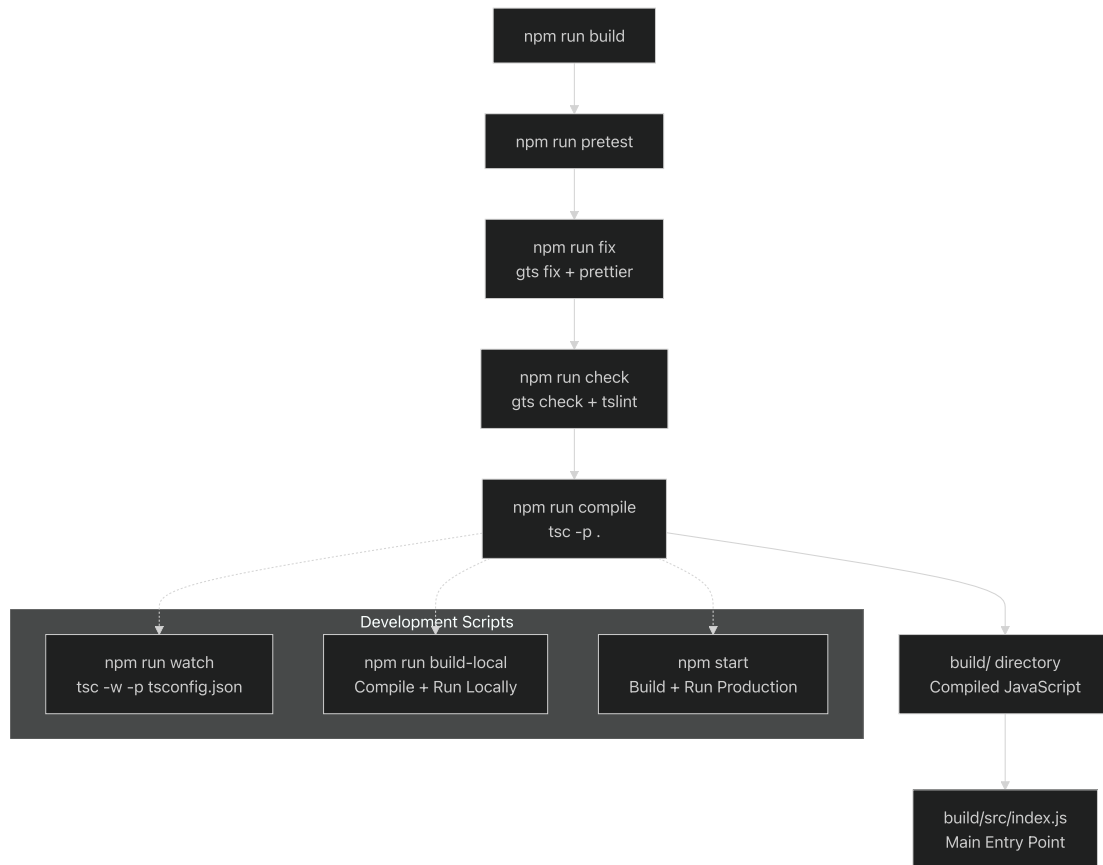
Prettier configuration is centralized in `prettier.config.js` with specific formatting rules:

```
// prettier.config.js configuration
{
  singleQuote: true,
  trailingComma: 'es5',
  arrowParens: 'always',
  endOfLine: 'lf',
  tabWidth: 2,
  printWidth: 120
}
```

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/prettier.config.js#L1-L8>

[prettier.config.js1-8](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/prettier.config.js#L1-L8) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/prettier.config.js#L1-L8>)

Build Process Integration



Build Process and Development Scripts Flow

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L6-L18>)

[package.json6-18](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L6-L18) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L6-L18>)

TypeScript Configuration and Compilation

The project uses TypeScript for type safety and modern JavaScript features, with compilation targeting the Node.js 10 runtime environment.

Build Output Structure

The TypeScript compilation process transforms source files from `src/` to `build/src/` maintaining the directory structure:

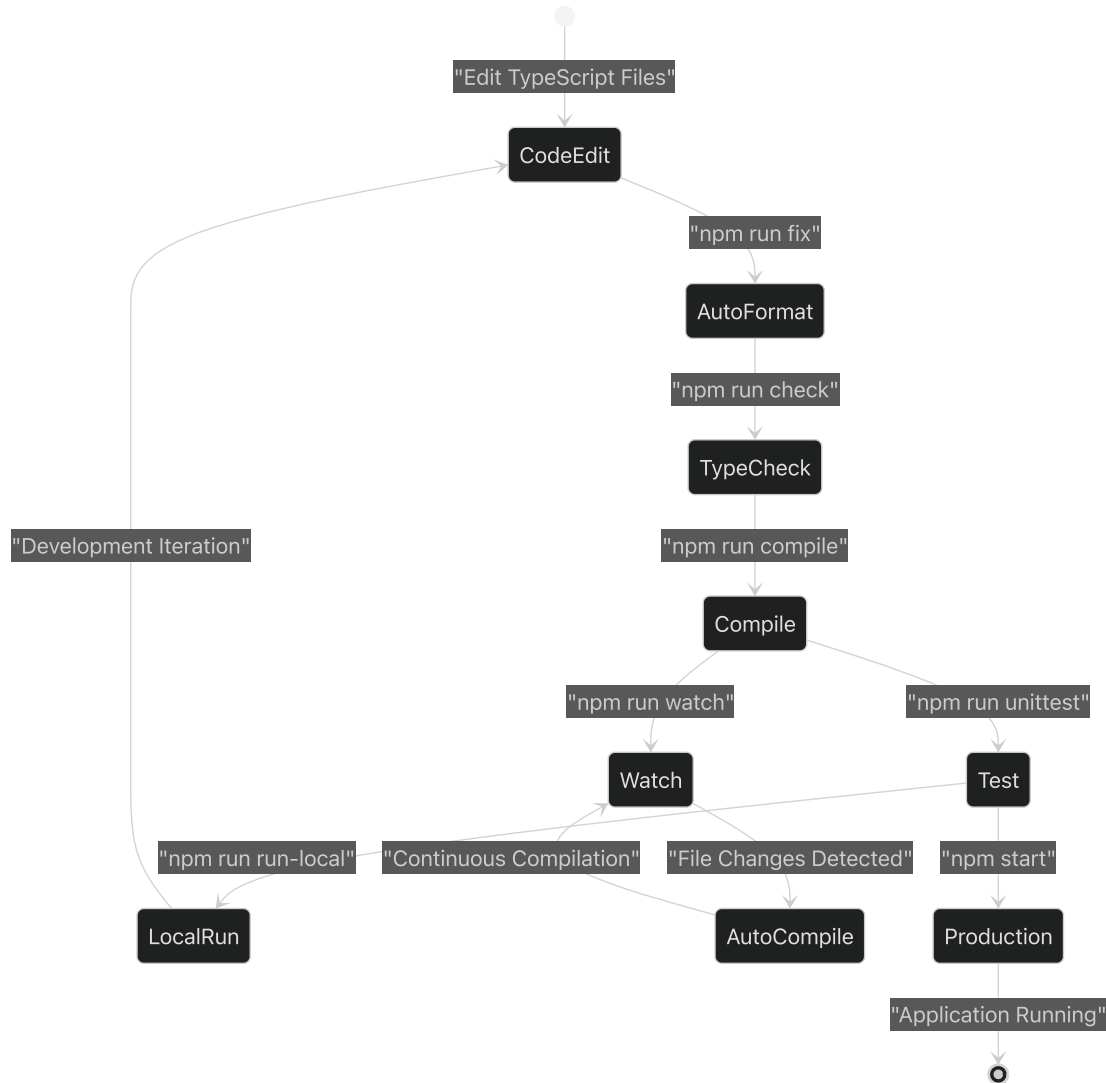
Source Path	Build Output	Purpose
<code>src/index.ts</code>	<code>build/src/index.js</code>	Main application entry point
<code>src/**/*.ts</code>	<code>build/src/**/*.js</code>	Application modules
<code>tsconfig.json</code>	Compilation configuration	TypeScript compiler settings

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L61-L61>)

*[package.json61](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L61-L61) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L61-L61>)

[package.json15](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L15-L15) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L15-L15>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L15-L15>))*

Development Workflow Integration



Development Workflow State Transitions

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L8-L18>

[package.json8-18](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L8-L18) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L8-L18>

Testing and Quality Assurance Setup

The development environment includes automated testing capabilities and code quality enforcement through multiple validation layers.

Test Execution Pipeline

The testing setup uses Mocha for unit testing with TypeScript compilation integration:

```
npm run unittest
```

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L18-L18>

[package.json18](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L18-L18) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L18-L18>

Code Quality Tools Integration

Tool	Configuration	Purpose
<code>gts</code>	Google TypeScript Style	Code formatting and linting
<code>tslint</code>	TypeScript linting rules	Static code analysis
<code>prettier</code>	Code formatting	Consistent code style
<code>husky</code>	Git hooks	Pre-commit quality checks

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L22-L34>)

[package.json#L22-L34](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L22-L34) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L22-L34>)

Development Environment Variables

The application supports environment-specific configuration through the `TRADEX_ENV_DOMAIN` environment variable and dynamic configuration loading, enabling different database connections and API endpoints for development versus production environments.

Sources: Based on system architecture context and tradex-common dependency

Development Environment

Relevant source files

- [`.nvmrc`](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc>)
- [`.vscode/settings.json`](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.vscode/settings.json>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.vscode/settings.json>)
- [`prettier.config.js`](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/prettier.config.js>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/prettier.config.js>)

This document covers the development environment setup for the viet-stock-bridge application, including Node.js version requirements, IDE configuration, and code formatting standards. For information about build processes and TypeScript compilation, see [Build and Packaging](#). For deployment configuration, see [Docker Configuration](#).

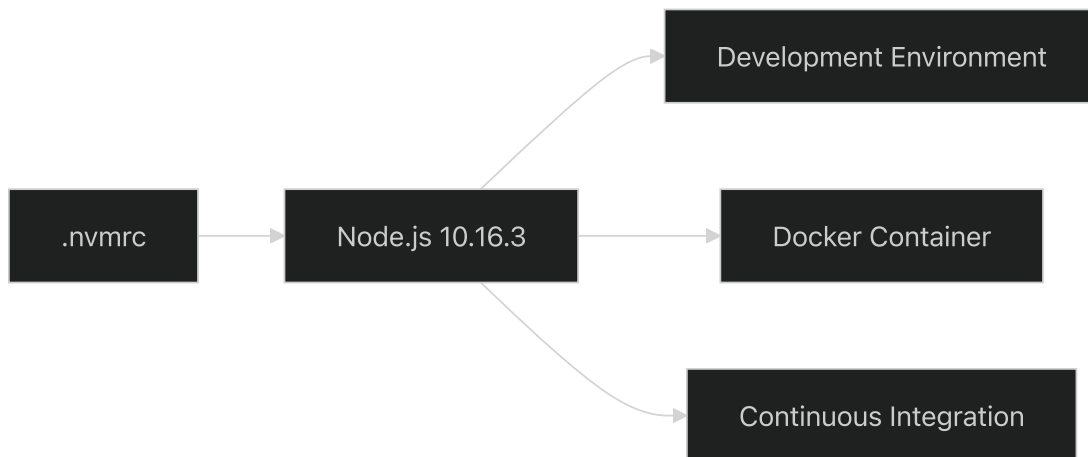
Node.js Version Management

The application requires a specific Node.js version to ensure consistent behavior across development and production environments.

Version Requirement

The project specifies Node.js version `10.16.3` in the `.nvmrc` file (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc#L1-L1>)

[.nvmrc1](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc#L1-L1) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc#L1-L1>), which serves as the canonical version reference for all development environments.



Development Toolchain Version Control

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc#L1-L1>).

[.nvmrc1](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc#L1-L1) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc#L1-L1>).

Using Node Version Manager

Developers can use `nvm` (Node Version Manager) to automatically switch to the correct Node.js version:

```
nvm use
```

This command reads the version specification from `.nvmrc` and switches the local environment to Node.js 10.16.3.

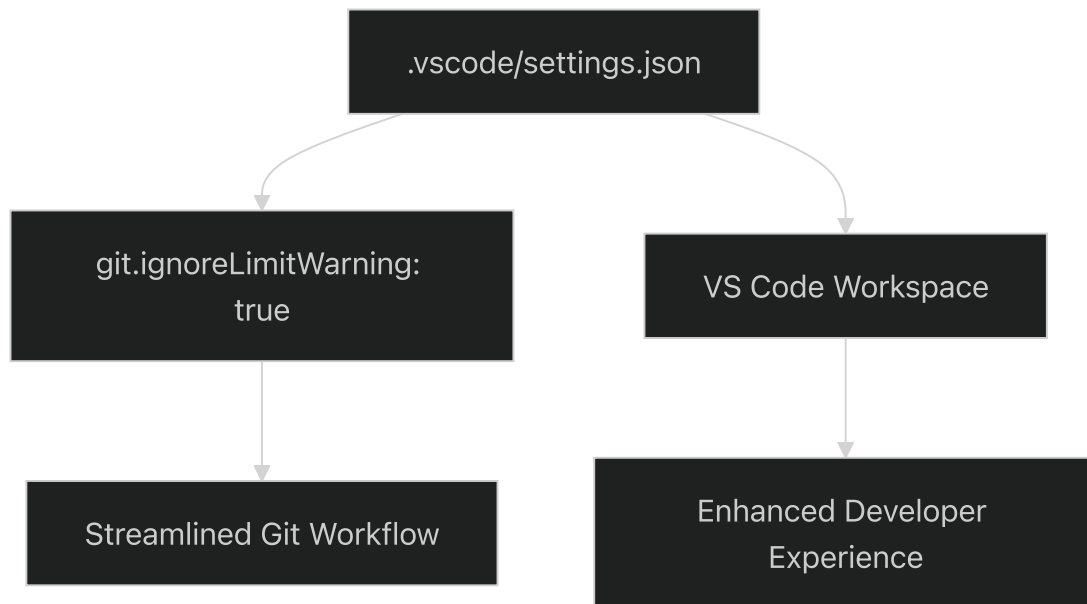
IDE Configuration

The development environment includes specific configuration for Visual Studio Code to optimize the development workflow.

VS Code Settings

The workspace configuration is defined in `.vscode/settings.json` with the following settings:

Setting	Value	Purpose
<code>git.ignoreLimitWarning</code>	<code>true</code>	Prevents Git warning messages for large repositories



VS Code Workspace Configuration

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.vscode/settings.json#L1-L3>.

[.vscode/settings.json1-3](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.vscode/settings.json#L1-L3) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.vscode/settings.json#L1-L3>).

The minimal configuration suggests the project relies primarily on standard VS Code functionality with selective optimization for Git operations in large codebases.

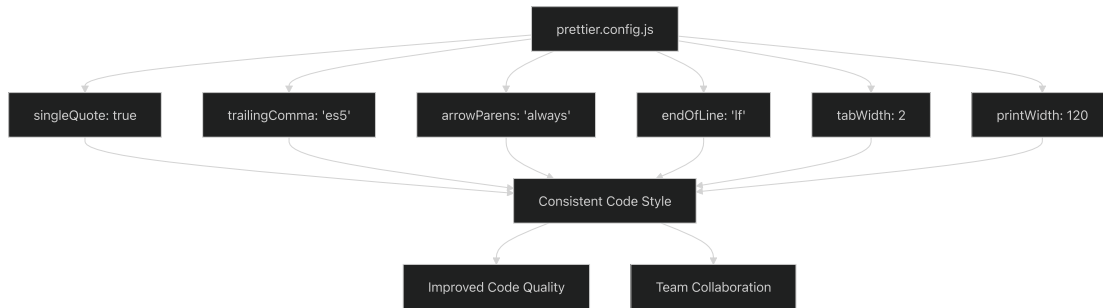
Code Formatting Standards

The project uses Prettier for consistent code formatting across all JavaScript and TypeScript files.

Prettier Configuration

The formatting rules are defined in `prettier.config.js` with the following specifications:

Configuration	Value	Purpose
<code>singleQuote</code>	<code>true</code>	Use single quotes for strings
<code>trailingComma</code>	<code>'es5'</code>	Add trailing commas where valid in ES5
<code>arrowParens</code>	<code>'always'</code>	Always include parentheses around arrow function parameters
<code>endOfLine</code>	<code>'lf'</code>	Use Unix-style line endings
<code>tabWidth</code>	<code>2</code>	Use 2 spaces for indentation
<code>printWidth</code>	<code>120</code>	Wrap lines at 120 characters



Prettier Formatting Pipeline

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/prettier.config.js#L1-L8>

[prettier.config.js#L1-L8](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/prettier.config.js#L1-L8) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/prettier.config.js#L1-L8>

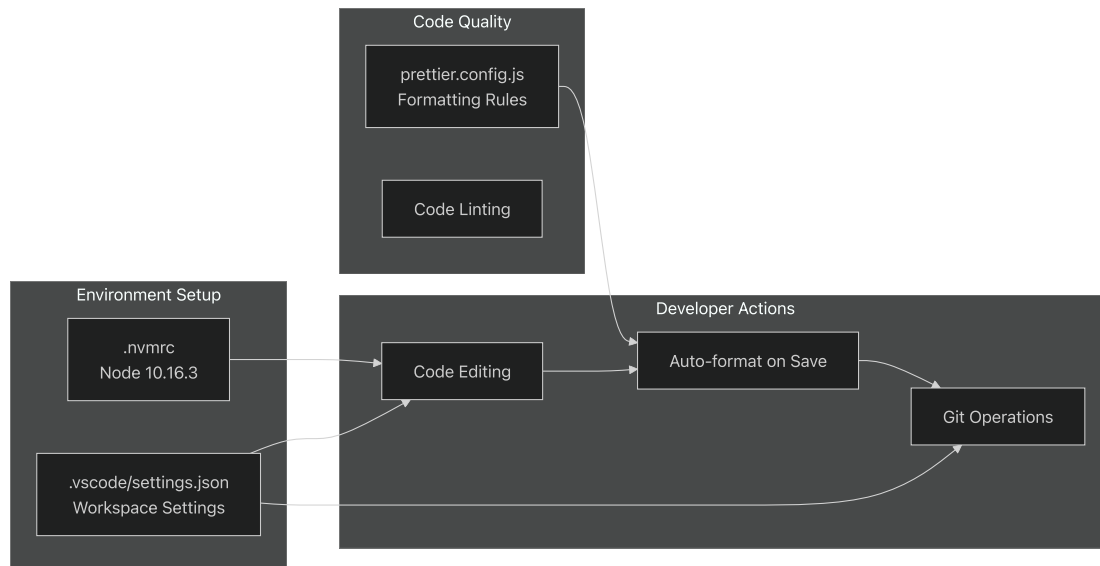
Format Enforcement

The Prettier configuration ensures consistent code style across the development team by:

- Enforcing single quotes for string literals
- Adding trailing commas in ES5-compatible locations (objects, arrays)
- Requiring parentheses around all arrow function parameters
- Using Linux/Unix line endings (LF) for cross-platform compatibility
- Setting a 2-space indentation standard
- Limiting line length to 120 characters for readability

Development Workflow Integration

The development environment tools work together to create a streamlined workflow for contributors.



Complete Development Environment Flow

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc#L1-L1>

[.nvmrc#L1-L1](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc#L1-L1) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.nvmrc#L1-L1>

[.vscode/settings.json#L1-L3](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.vscode/settings.json#L1-L3) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.vscode/settings.json#L1-L3> [%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.vscode/settings.json#L1-L3)

prettier.config.js#L1-L8](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/prettier.config.js#L1-L8>)
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/prettier.config.js#L1-L8>))

The integrated development environment provides:

1. **Version Consistency:** `.nvmrc` ensures all developers use the same Node.js version
2. **IDE Optimization:** VS Code settings reduce friction during Git operations
3. **Code Standardization:** Prettier automatically enforces consistent formatting
4. **Workflow Automation:** Format-on-save integration maintains code quality without manual intervention

This configuration supports the TypeScript-based architecture documented in [Build and Packaging](#) and prepares code for the containerized deployment process described in [Docker Configuration](#).

Build and Packaging

Relevant source files

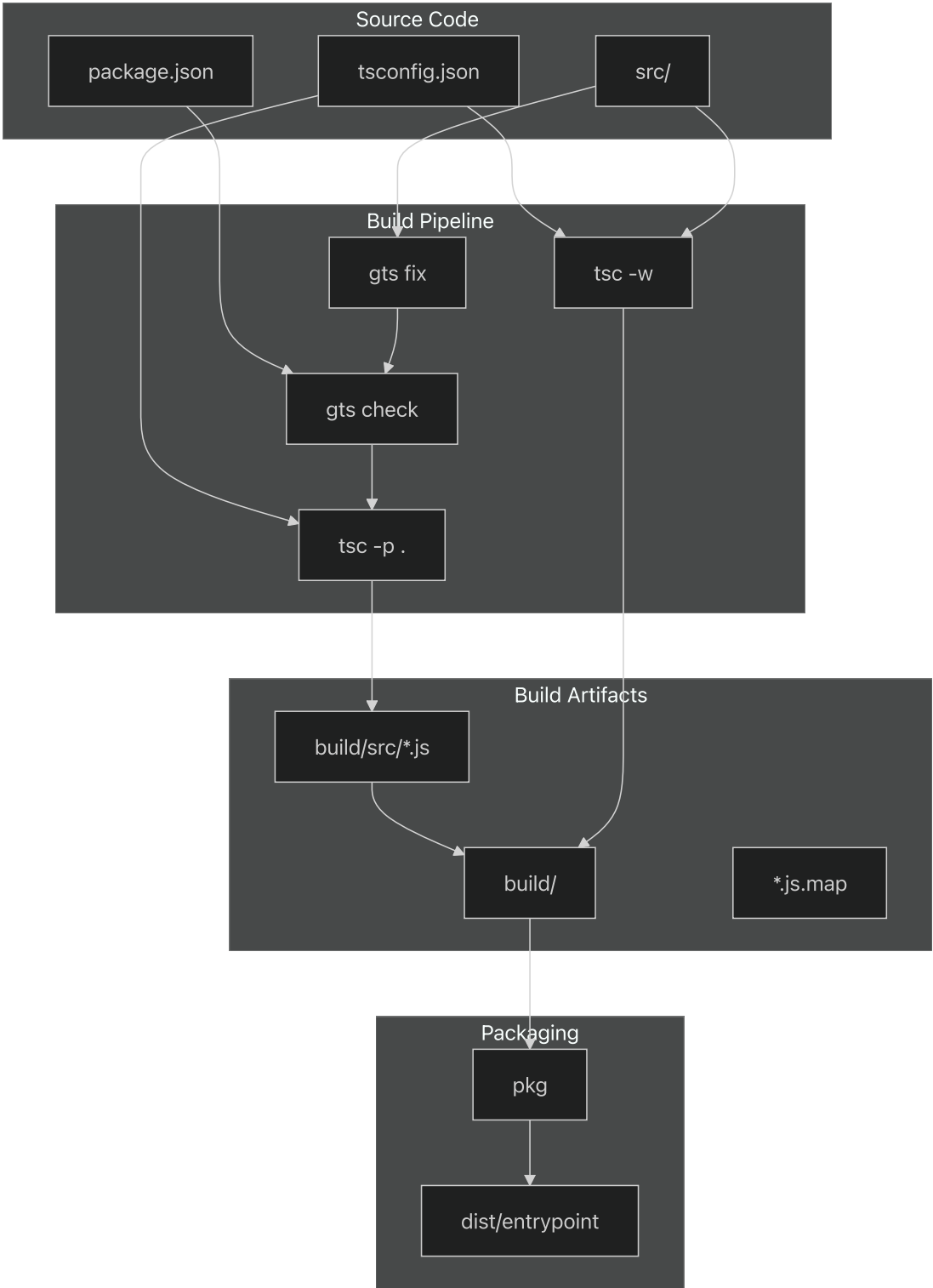
- [`.gitignore`](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.gitignore>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.gitignore>))
- [`package-lock.json`](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package-lock.json>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package-lock.json>))
- [`package.json`](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json>))

This document describes the build system, dependency management, and application packaging processes for the viet-stock-bridge service. It covers TypeScript compilation, dependency resolution, development workflows, and executable generation.

For deployment configuration details, see [Docker Configuration](#). For development environment setup, see [Development Environment](#).

Build Process Overview

The build process transforms TypeScript source code into executable JavaScript and manages project dependencies through npm. The system uses a multi-stage approach with linting, type checking, and compilation steps.



Build Process Flow

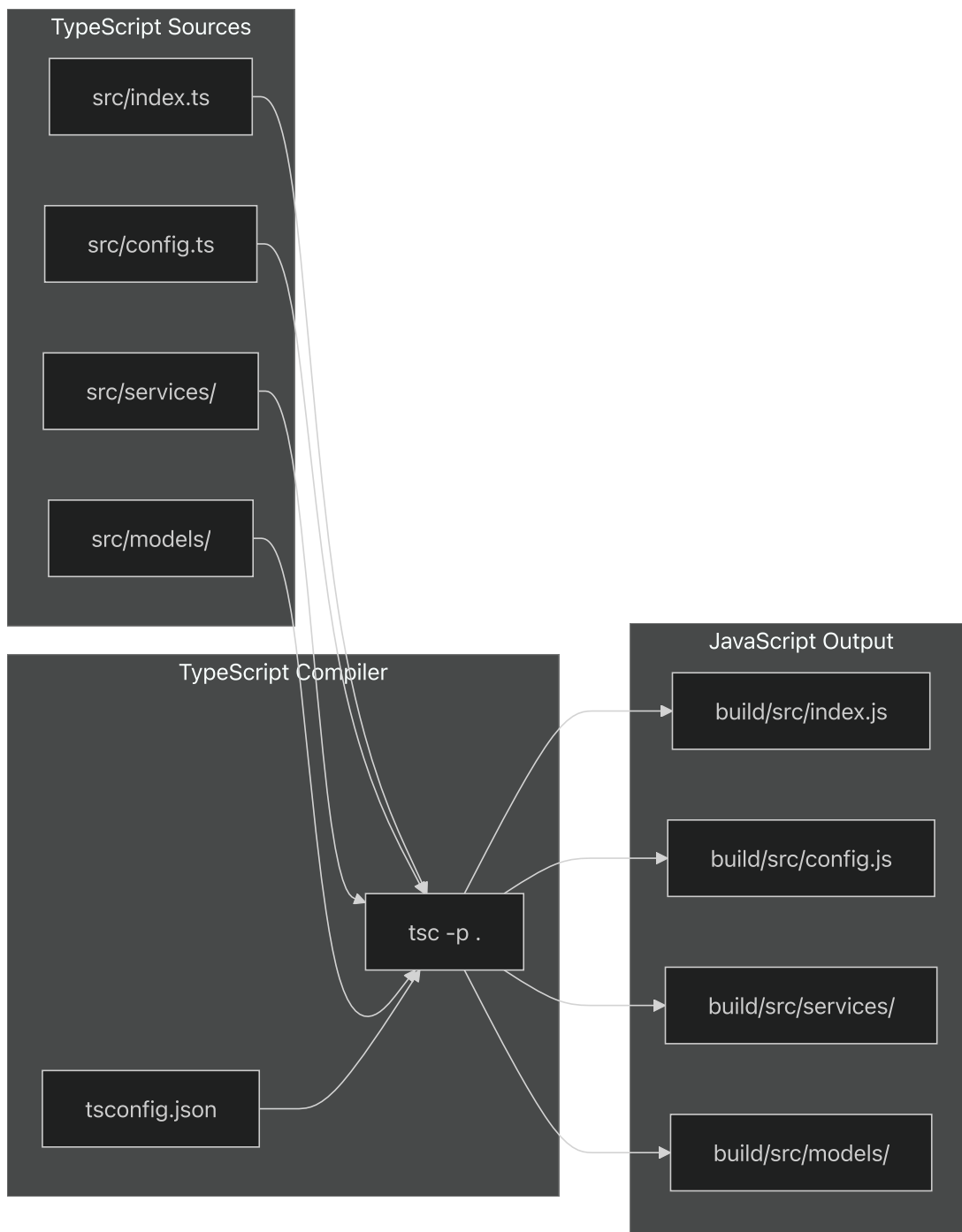
Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L5-L18>.
[package.json5-18](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L5-L18) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L5-L18>.

TypeScript Compilation

The application uses TypeScript 4.0.2 with strict compilation settings. The build process compiles all source files from `src/` to `build/src/` while preserving the directory structure.

Script	Command	Purpose
compile	<code>tsc -p .</code>	Compile TypeScript to JavaScript
watch	<code>tsc -w -p tsconfig.json</code>	Watch mode for development
build	<code>npm run pretest</code>	Full build with linting
pretest	<code>npm run fix && npm run check && npm run compile</code>	Complete build pipeline

The TypeScript compiler configuration references `tsconfig.json` and outputs JavaScript files with source maps to the `build/` directory.



TypeScript Compilation Pipeline

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L8-L8>)

[package.json#L8-L8](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L8-L8) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L8-L8>)

package.json#L15-L15) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L15-L15>)

([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L15-L15))

package.json#L10-L10) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L10-L10>)

(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L10-L10>)

Dependency Management

The project uses npm with a locked dependency tree to ensure reproducible builds across environments. Dependencies are categorized into development and runtime dependencies.

Runtime Dependencies

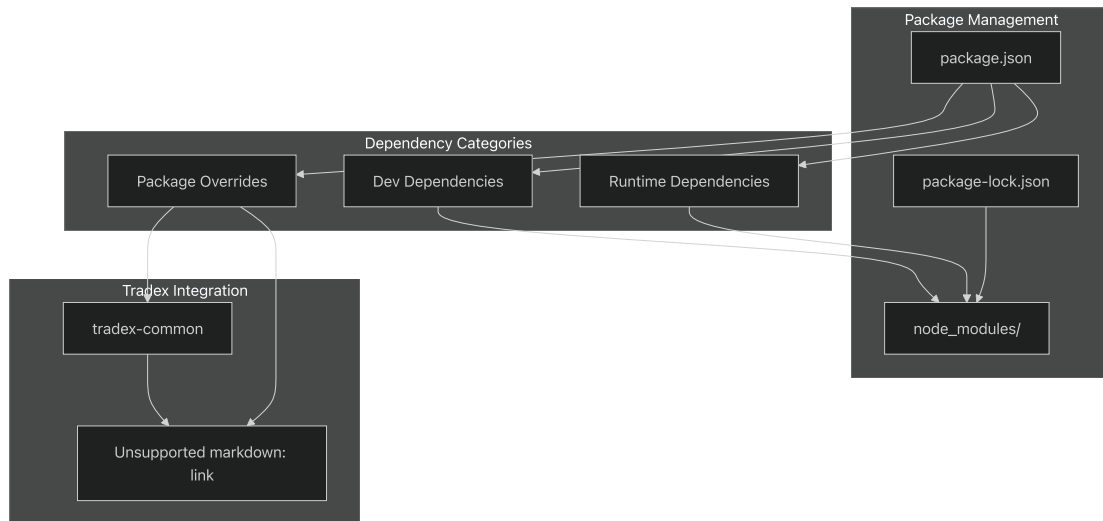
Core application dependencies required for production execution:

Category	Dependencies
HTTP Client	<code>axios@^0.21.4</code> , <code>node-fetch@^2.6.0</code>
Database	<code>mongodb@^3.3.0</code> , <code>mysql@^2.18.1</code> , <code>redis@^3.0.2</code>
ORM/ODM	<code>typeorm@^0.2.29</code> , <code>typeorm-typedi-extensions@^0.2.3</code>
File Storage	<code>minio@^7.0.18</code>
DI Container	<code>typedi@^0.8.0</code> , <code>reflect-metadata@^0.1.13</code>
Task Scheduling	<code>node-schedule@^1.3.2</code>
Utilities	<code>moment@^2.29.4</code> , <code>uuid@^9.0.1</code> , <code>btoa@^1.2.1</code>
Tradex Platform	<code>tradex-common@git+https://github.com/...</code>

Development Dependencies

Tools and libraries used during development and build processes:

Category	Dependencies
TypeScript	<code>typescript@^4.0.2</code> , <code>@types/node@^14.18.63</code>
Code Quality	<code>gts@^1.1.2</code> , <code>tslint@^5.20.1</code> , <code>prettier@^2.1.1</code>
Git Hooks	<code>husky@^1.3.1</code>
Type Definitions	<code>@types/btoa</code> , <code>@types/mocha</code> , <code>@types/node-fetch</code>



Dependency Resolution and Overrides

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L22-L35>).

[package.json22-35](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L22-L35) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L22-L35>)

[package.json36-55](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L36-L55) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L36-L55>)
([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L36-L55)).

[package.json56-60](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L56-L60) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L56-L60>)
([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L56-L60)).

[package-lock.json1-6](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package-lock.json#L1-L6) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package-lock.json#L1-L6>)
([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package-lock.json#L1-L6)).

Code Quality Pipeline

The build system integrates code quality tools to maintain consistent code style and catch potential issues before compilation.

Google TypeScript Style (GTS)

The project uses GTS for consistent code formatting and linting:

- `gts check` : Validates code style without making changes
- `gts fix` : Automatically fixes style violations

Linting and Formatting

Tool	Configuration	Purpose
TSLint	<code>tslint@^5.20.1</code>	TypeScript linting
Prettier	<code>prettier@^2.1.1</code>	Code formatting
GTS	<code>gts@^1.1.2</code>	Google TypeScript style guide

The `pretest` script orchestrates the complete quality pipeline:

1. **Fix:** Auto-format code with GTS
2. **Check:** Validate style compliance

3. Compile: TypeScript compilation with type checking

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L9-L9>

[package.json9](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L9-L9) [\[https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L9-L9\]](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L9-L9)

[package.json14](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L14-L14) [\[https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L14-L14\]](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L14-L14)
[%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L14-L14)

[package.json16](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L16-L16) [\[https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L16-L16\]](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L16-L16)
[%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L16-L16)

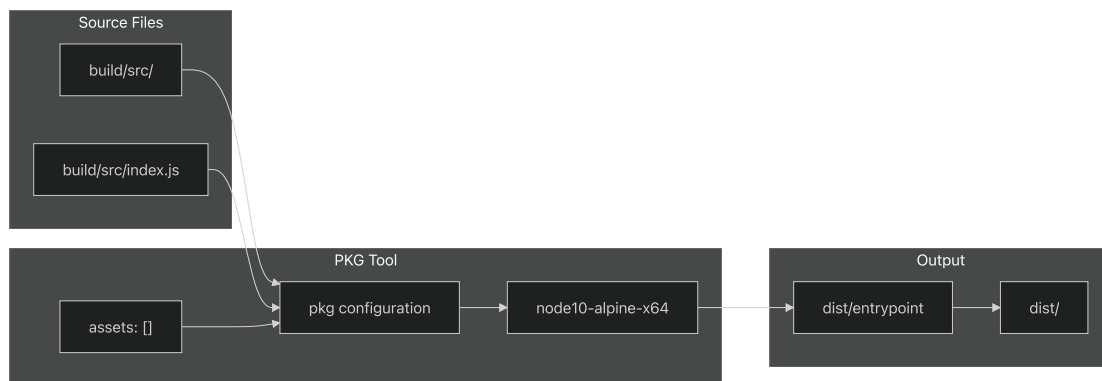
[package.json29](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L29-L29) [\[https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L29-L29\]](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L29-L29)
[%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L29-L29)

[package.json31-33](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L31-L33) [\https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L31-L33
[\]\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L31-L33\)\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L31-L33)

Application Packaging

The project supports executable packaging using the `pkg` tool, which bundles the Node.js runtime with the application code into a single binary.

PKG Configuration



PKG Packaging Process

The packaging configuration targets Alpine Linux with Node.js 10:

Setting	Value	Purpose
<code>targets</code>	<code>node10-alpine-x64</code>	Alpine Linux 64-bit with Node.js 10
<code>output</code>	<code>dist/entrypoint</code>	Executable output path
<code>assets</code>	<code>[]</code>	No additional assets bundled
<code>bin</code>	<code>build/src/index.js</code>	Application entry point

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L61-L68>

[package.json61-68](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L61-L68) [\[https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L61-L68\]](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L61-L68)

Build Artifacts and Exclusions

The build process generates multiple artifact types that are managed through `.gitignore` configuration:

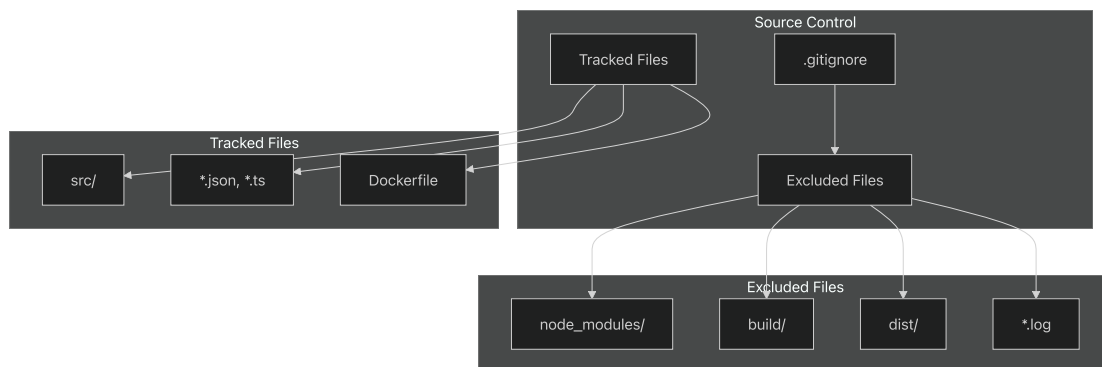
Generated Artifacts

Directory/File	Type	Purpose
build/	Compiled JavaScript	TypeScript compilation output
dist/	Packaged executables	PKG tool output
node_modules/	Dependencies	npm package installation
*.log	Log files	Runtime and build logs

Git Exclusions

The `.gitignore` file excludes build artifacts while preserving source configuration:

- **Node.js artifacts:** `node_modules/`, `dist/`, `build/`
- **Log files:** `*.log`
- **Platform files:** `.DS_Store`, `Thumbs.db`
- **IDE files:** `.idea/`



Git Exclusion Strategy

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.gitignore#L6-L10>).

[.gitignore6-10](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.gitignore#L6-L10) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.gitignore#L6-L10>)

[.gitignore17-18](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.gitignore#L17-L18) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.gitignore#L17-L18>)

[.gitignore27-28](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.gitignore#L27-L28) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.gitignore#L27-L28>)

[.gitignore33-37](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.gitignore#L33-L37) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/.gitignore#L33-L37>)

Development Workflows

The build system supports multiple development workflows through npm scripts:

Development Scripts

Script	Workflow	Use Case
start	build && node build/src/index.js	Production-like execution
run-local	cd build/src && node index.js	Direct execution
build-local	compile && run-local	Local development build
watch	tsc -w -p tsconfig.json	Live compilation

Memory Optimization

The build process includes memory optimization for large TypeScript compilations:

```
export NODE_OPTIONS='--max-old-space-size=4096' && npm run pretest
```

This allocation supports compilation of the complex dependency tree including `tradex-common` and extensive type definitions.

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L7-L7>)

[package.json#L7-L7](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L7-L7) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L7-L7>)

[package.json#L11-L13](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L11-L13) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L11-L13>)

([%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L11-L13))

[package.json#L8-L8](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L8-L8) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L8-L8>)

(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/package.json#L8-L8>)

Deployment

Overview

Relevant source files

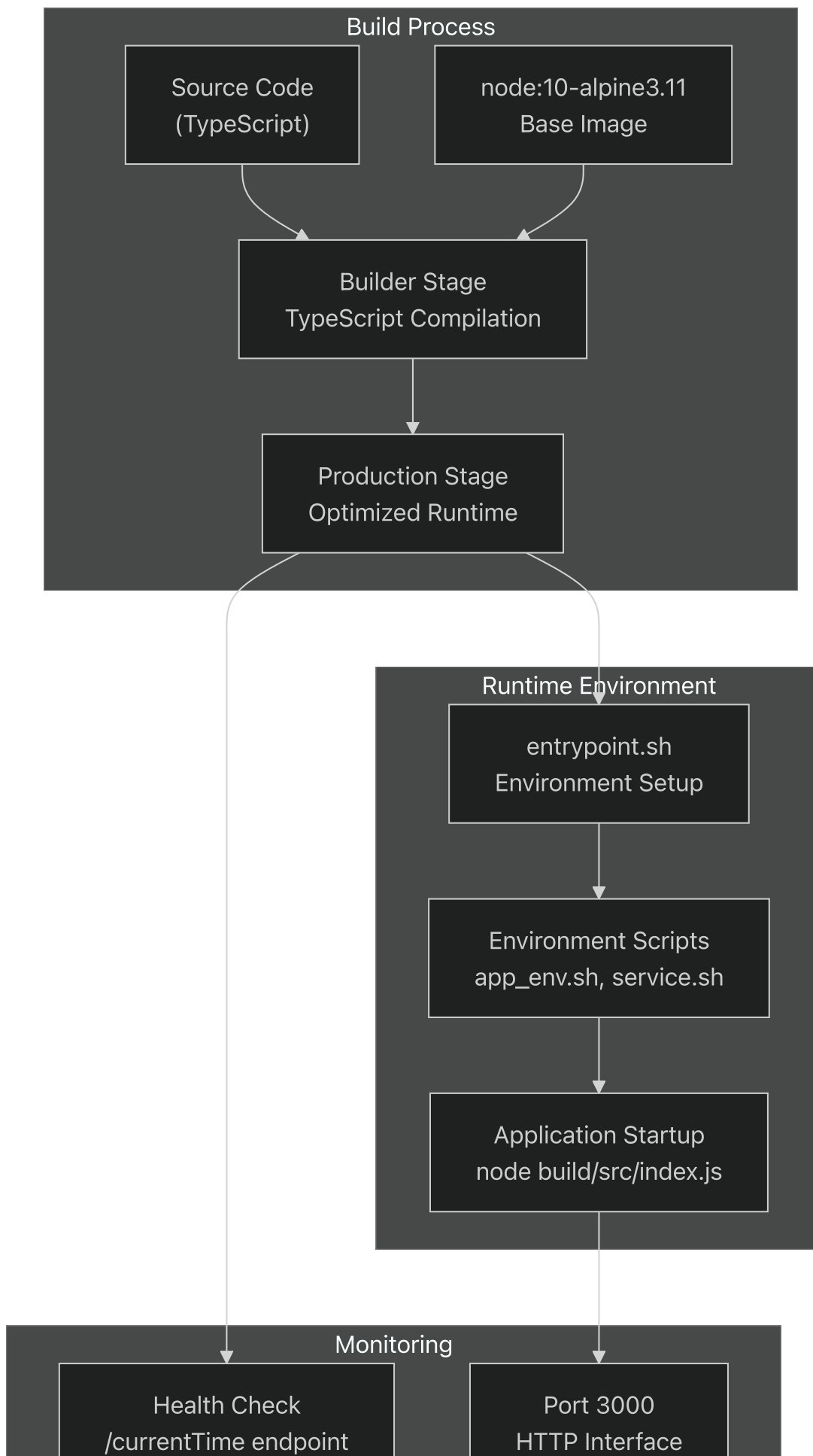
- [[dockerfile](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile>)
- [[entrypoint.sh](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh>) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh>)

This document covers the containerization strategy, deployment configuration, and production environment setup for the viet-stock-bridge service. It details how the application is packaged into Docker containers and the environment-specific orchestration mechanisms used for service startup and configuration management.

For Docker-specific build configurations, see [Docker Configuration](#). For environment setup and service orchestration details, see [Service Orchestration](#).

Deployment Overview

The viet-stock-bridge service uses a containerized deployment strategy built around Docker multi-stage builds and flexible environment configuration. The deployment process transforms TypeScript source code into a production-ready Node.js application with comprehensive health monitoring and environment-specific configuration loading.





Sources: [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L1-L57\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L1-L57).

[dockerfile1-57](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L1-L57) [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L1-L57\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L1-L57)[

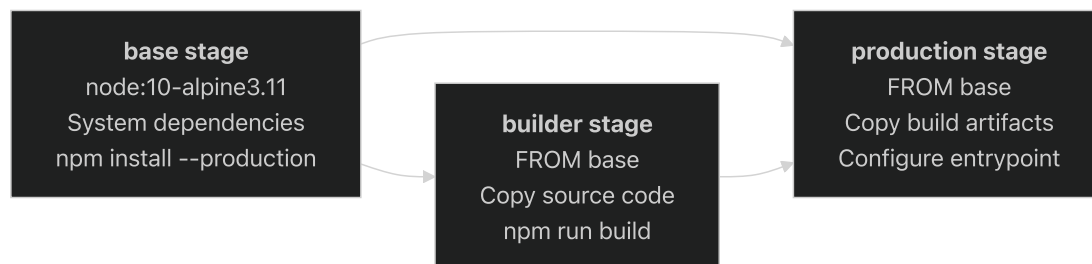
entrypoint.sh1-90](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L1-L90>
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L1-L90>))

Multi-Stage Container Build

The Docker build process implements a three-stage approach optimizing for both build efficiency and production runtime performance.

Build Stages

Stage	Purpose	Key Operations
base	Foundation	Install system dependencies, copy package files, install npm dependencies
builder	Compilation	Copy source code, install dev dependencies, compile TypeScript
production	Runtime	Copy built artifacts, configure health checks, set entrypoint



Sources: [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L2-L36\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L2-L36).

[dockerfile2-36](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L2-L36) [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L2-L36\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L2-L36).

System Dependencies

The base image includes native compilation tools required for Node.js native modules:

- `bash` - Shell scripting support
- `git` - Version control operations
- `python3` - Native module compilation
- `make` - Build system
- `g++` - C++ compiler

Sources: [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L5-L11\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L5-L11).

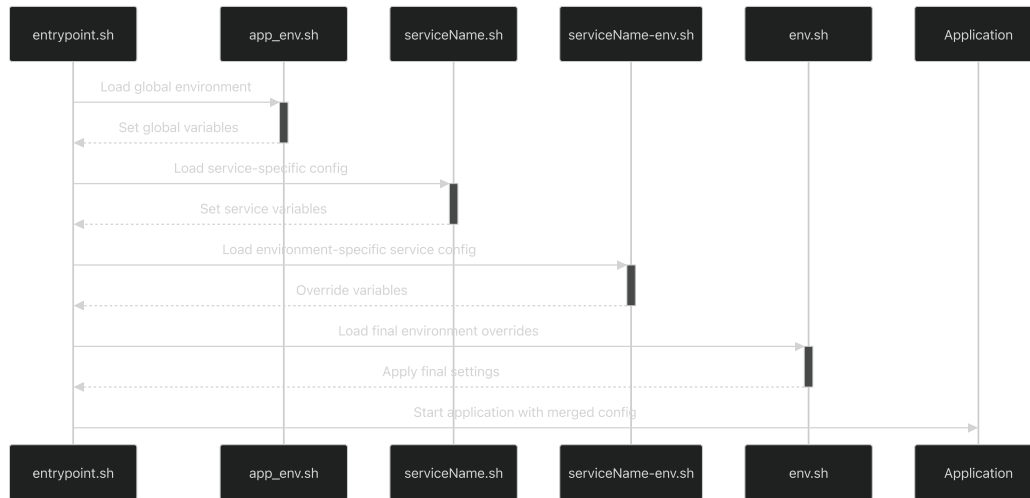
[dockerfile5-11](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L5-L11) [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L5-L11\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L5-L11).

Runtime Configuration System

The service uses a hierarchical environment configuration system managed through `entrypoint.sh`. This script loads configuration in a specific order, allowing for environment-specific overrides and service-specific

customizations.

Configuration Loading Sequence



Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L37-L62>).

[entrypoint.sh37-62](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L37-L62) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L37-L62>)

Environment Variable Management

The entrypoint script manages several key environment variables:

- `TRADEX_WORKING_DIR` - Application working directory
- `SERVICE_NAME` - Service identifier for configuration lookup
- `TRADEX_ENV_ENVIRONMENT` - Runtime environment designation

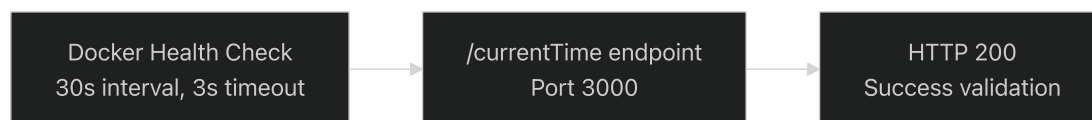
Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L35-L87>).

[entrypoint.sh35-87](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L35-L87) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L35-L87>)

Health Monitoring and Service Management

The production container includes comprehensive health monitoring through Docker health checks and HTTP endpoint validation.

Health Check Configuration



The health check validates service availability by making HTTP requests to the `/currentTime` endpoint and verifying a 200 status code response.

Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L53-L54>).

[dockerfile53-54](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L53-L54) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L53-L54>)

Port Configuration

The service exposes port 3000 for HTTP traffic, configured in the Docker container and used for both application traffic and health monitoring.

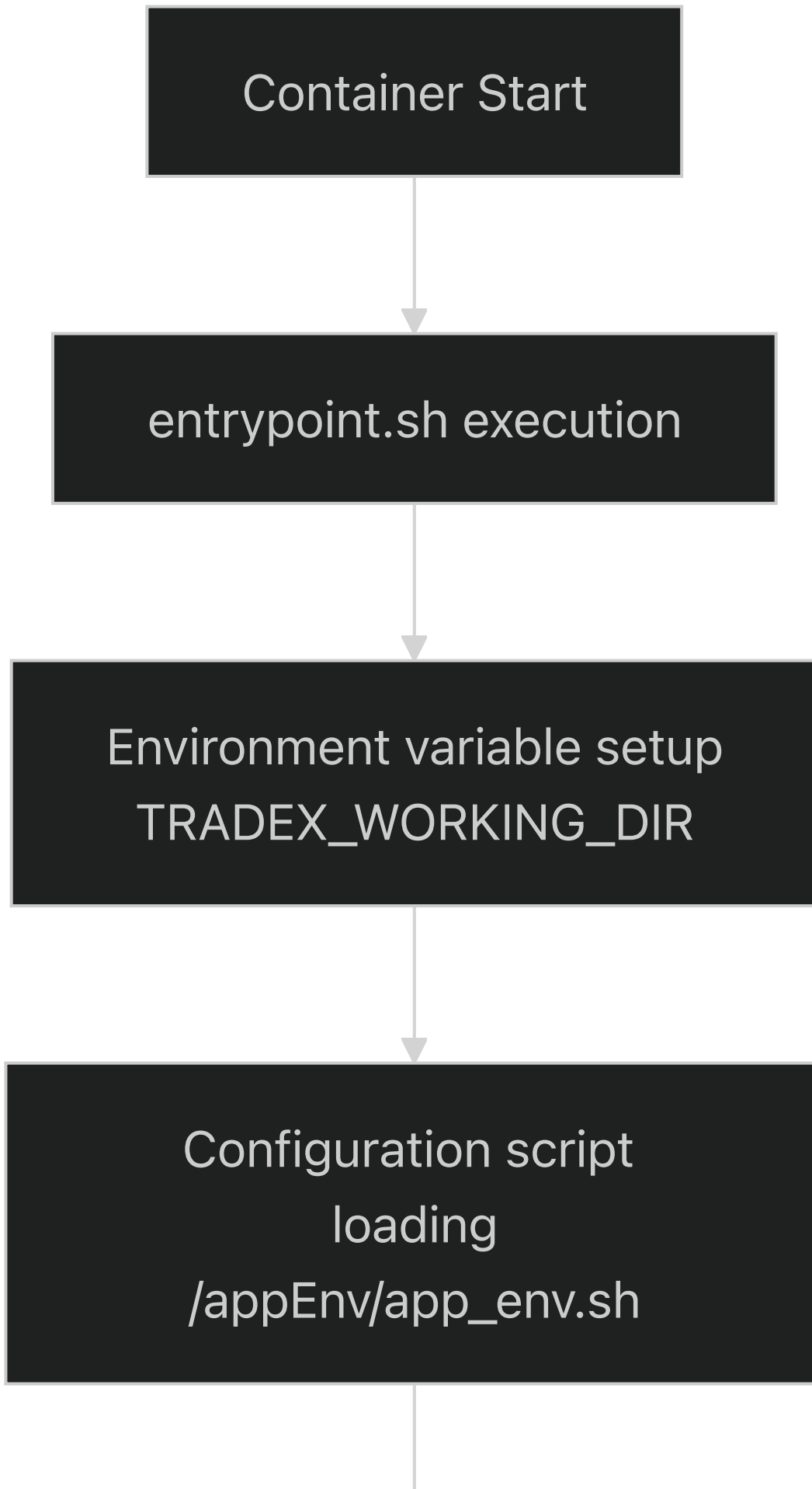
Sources: [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L50-L50\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L50-L50).

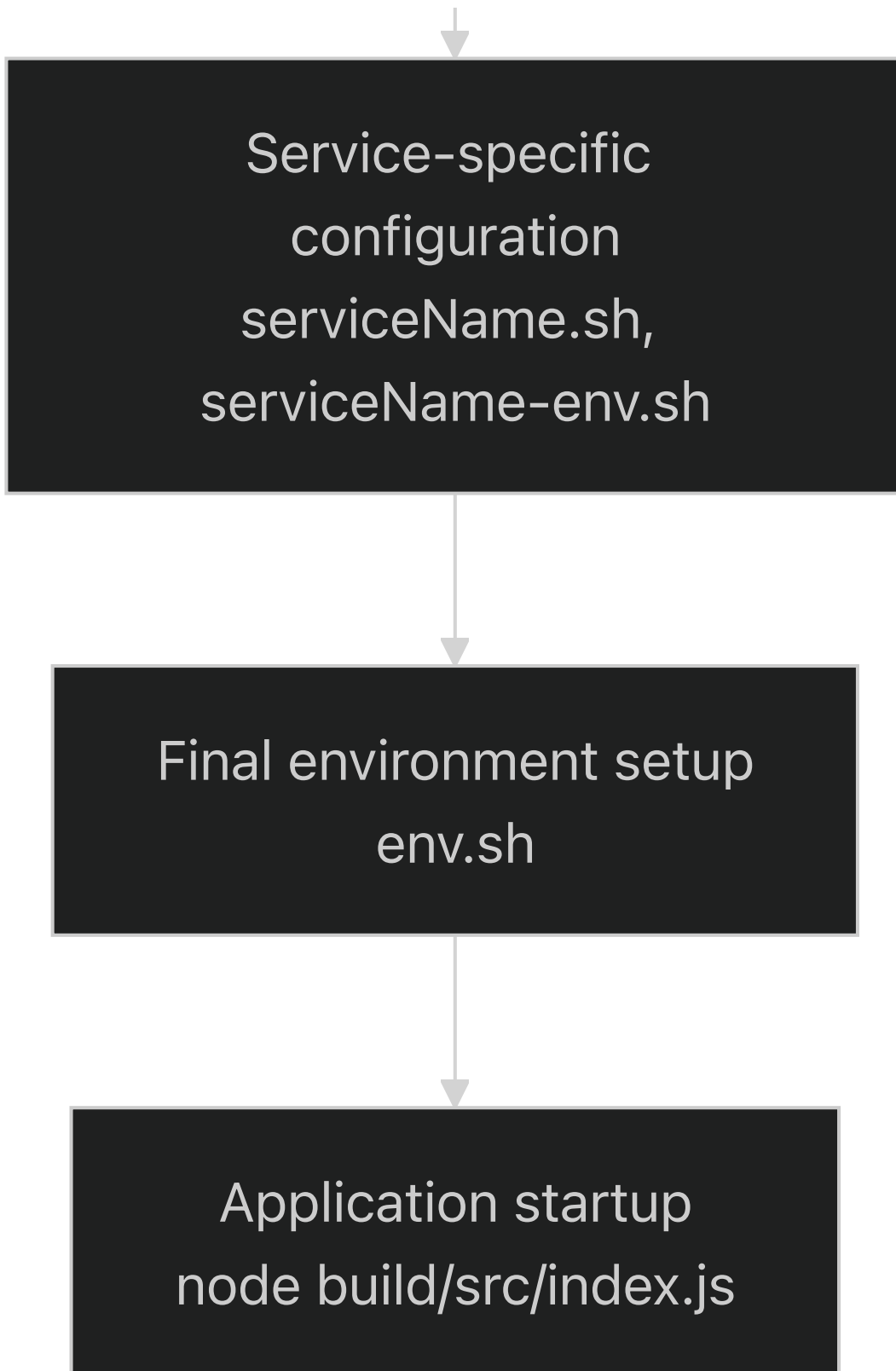
[dockerfile50](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L50-L50) [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L50-L50\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L50-L50)

Service Startup Process

The complete service startup process involves environment preparation, configuration loading, and application initialization.

Startup Command Flow





Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L32-L90>).

[entrypoint.sh32-90](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L32-L90) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L32-L90>) [

dockerfile56](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L56-L56>

(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L56-L56>)).

The startup process allows for flexible deployment across different environments by supporting environment-specific configuration files and service-specific overrides. The final application command is executed with all

environment variables properly configured for the target deployment environment.

Docker Configuration

Relevant source files

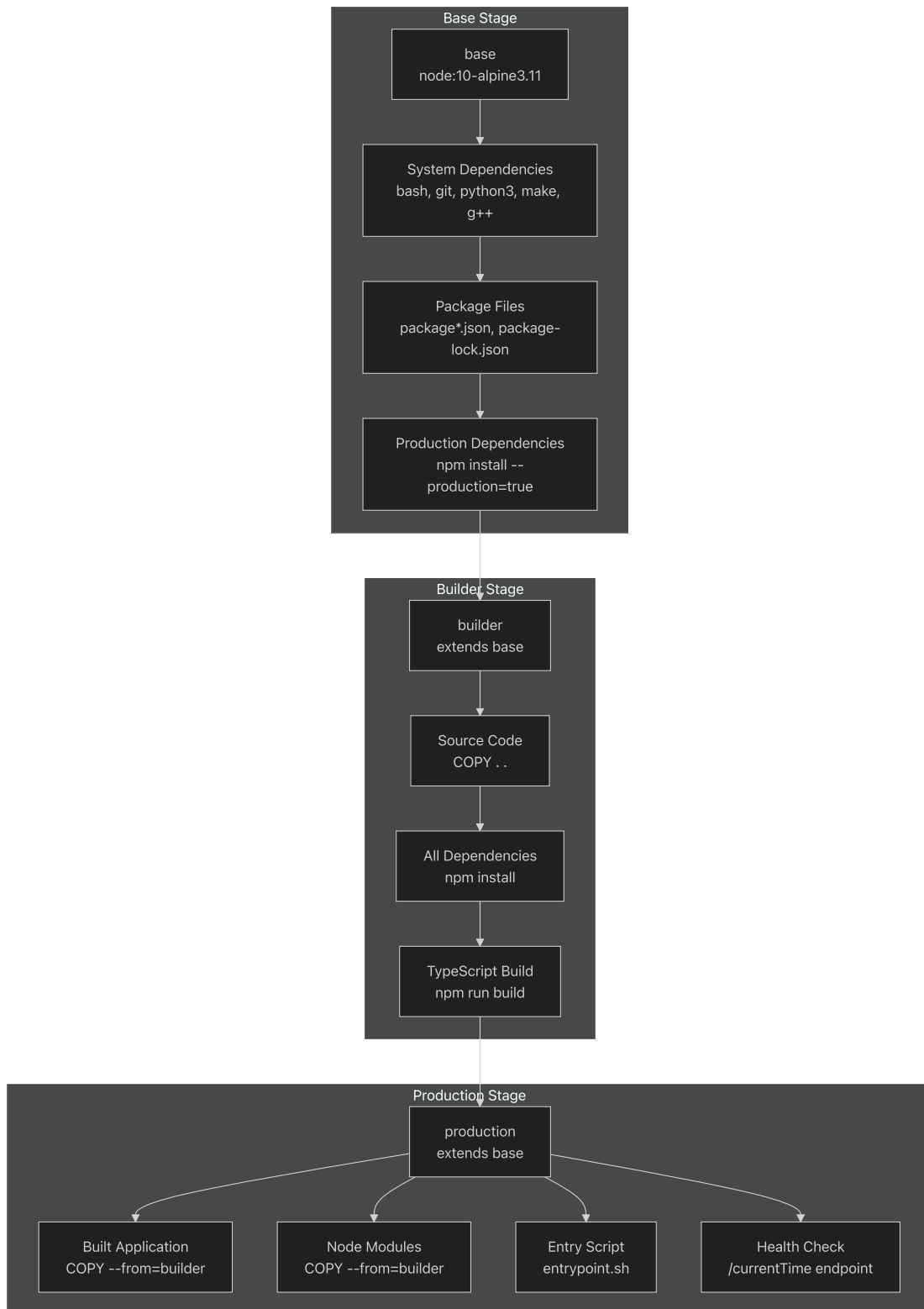
- [dockerfile](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile> (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile>))

This document covers the Docker containerization strategy for the viet-stock-bridge application, including the multi-stage build process, container optimization, and health check implementation. For information about service orchestration and runtime environment management, see [Service Orchestration](#).

Multi-Stage Build Process

The viet-stock-bridge application uses a multi-stage Docker build strategy to optimize the final production container while maintaining a clean build process. The build consists of three distinct stages: `base`, `builder`, and `production`.

Build Stages Overview



Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L1-L57>).

[dockerfile1-57](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L1-L57) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L1-L57>).

Base Stage Configuration

The base stage establishes the foundation for both build and production environments using Node.js 10 on Alpine Linux 3.11. This stage installs essential build dependencies required for native module compilation.

Component	Purpose	Configuration
Base Image	<code>node:10-alpine3.11</code>	Lightweight Node.js runtime
Build Tools	<code>bash, git, python3, make, g++</code>	Native module compilation
Package Management	<code>npm install --frozen-lockfile --production=true</code>	Production dependencies only
Working Directory	<code>/app</code>	Consistent application path

The base stage copies package definition files and installs only production dependencies to minimize the dependency footprint in the final image.

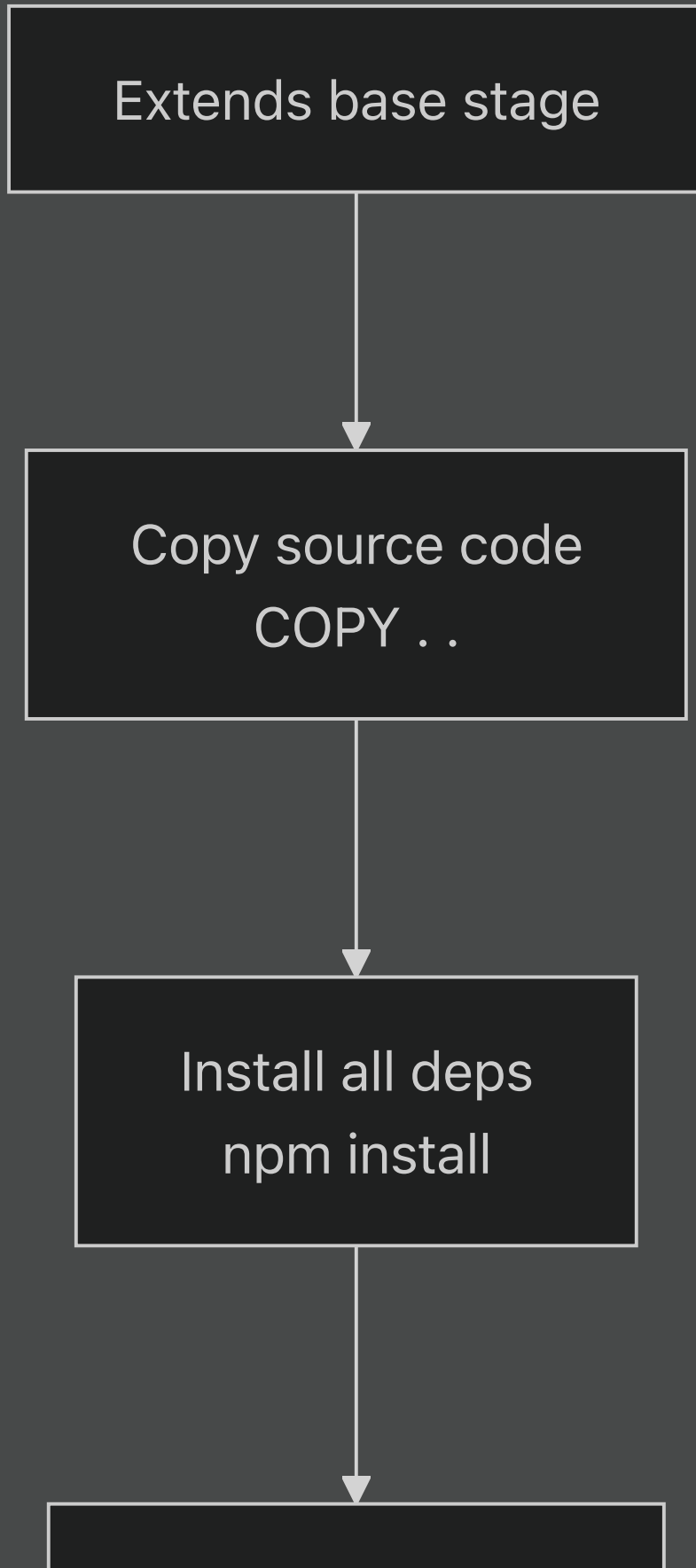
Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L2-L22>.

[dockerfile2-22](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L2-L22) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L2-L22>).

Builder Stage Process

The builder stage extends the base configuration to include development dependencies and perform the TypeScript compilation process.

Builder Stage Flow



Build TypeScript

npm run build

The builder stage performs the complete application build including TypeScript compilation, generating the `build/` directory containing the compiled JavaScript output.

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L24-L32>

[dockerfile24-32](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L24-L32) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L24-L32>)

Production Stage Optimization

The production stage creates the minimal runtime container by copying only the necessary artifacts from the builder stage without including development dependencies or source code.

Artifact	Source	Purpose
<code>./build</code>	Builder stage	Compiled JavaScript application
<code>./node_modules</code>	Builder stage	Runtime dependencies
<code>entrypoint.sh</code>	Host filesystem	Service startup script

The production stage exposes port 3000 and configures comprehensive health checking to ensure service availability.

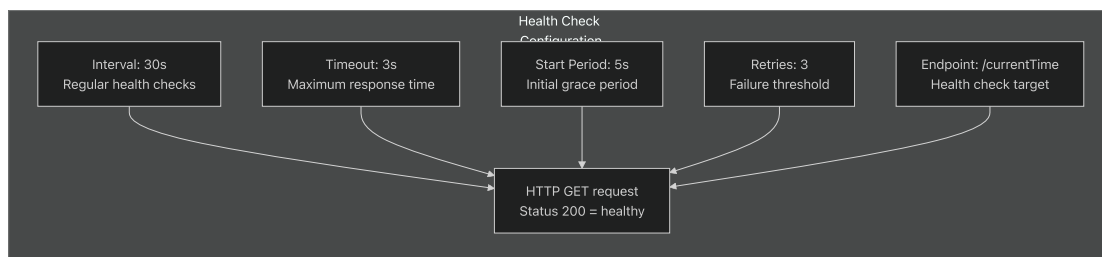
Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L35-L50>

[dockerfile35-50](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L35-L50) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L35-L50>)

Container Runtime Configuration

Health Check Implementation

The container implements a sophisticated health check mechanism that validates service availability through the `/currentTime` endpoint:



The health check executes a Node.js script that performs an HTTP GET request to `http://localhost:3000/currentTime` and validates a 200 status response.

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L52-L54>

[dockerfile52-54](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L52-L54) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L52-L54>)

Service Entrypoint

The container uses `/bin/bash` as the entrypoint executor, running the `entrypoint.sh` script with `node build/src/index.js` as the primary command argument. This configuration enables environment-specific initialization before starting the Node.js application.

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L56-L56>

[dockerfile56](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L56-L56) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L56-L56>)

Container Optimization Strategies

Image Size Optimization

The multi-stage build strategy significantly reduces the final image size by:

- Using Alpine Linux as the base image for minimal footprint
- Installing build dependencies only in intermediate stages
- Copying only production artifacts to the final stage
- Excluding source code and development dependencies from production

Security Considerations

The container configuration includes several security-focused practices:

- Running on a minimal Alpine Linux base image
- Installing only necessary system packages
- Using specific Node.js version (10) for consistency
- Implementing proper signal handling preparation (commented `dumb-init`)

The dockerfile includes preparation for `dumb-init` installation, which would provide proper signal handling for containerized processes, though it's currently commented out.

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L37-L38>

[dockerfile37-38](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L37-L38) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L37-L38>)

Build Dependencies

System Package Requirements

The build process requires specific system packages for native module compilation:

Package	Purpose
<code>bash</code>	Shell scripting support
<code>git</code>	Version control operations
<code>python3</code>	Native module build scripts
<code>make</code>	Build automation
<code>g++</code>	C++ compiler for native modules

These dependencies are installed in the base stage and available throughout the build process, but do not impact the final production image size due to the multi-stage architecture.

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L5-L11>.

[dockerfile5-11](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L5-L11) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/dockerfile#L5-L11>).

Service Orchestration

Relevant source files

- [[entrypoint.sh](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh)](<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh>
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh>))

This document covers the service orchestration mechanisms used to initialize and configure the viet-stock-bridge application at runtime. The orchestration system handles environment-specific startup procedures, configuration script execution, and runtime environment management through the `entrypoint.sh` script.

For Docker container configuration and build processes, see [Docker Configuration](#). For overall system architecture, see [System Architecture](#).

Purpose and Scope

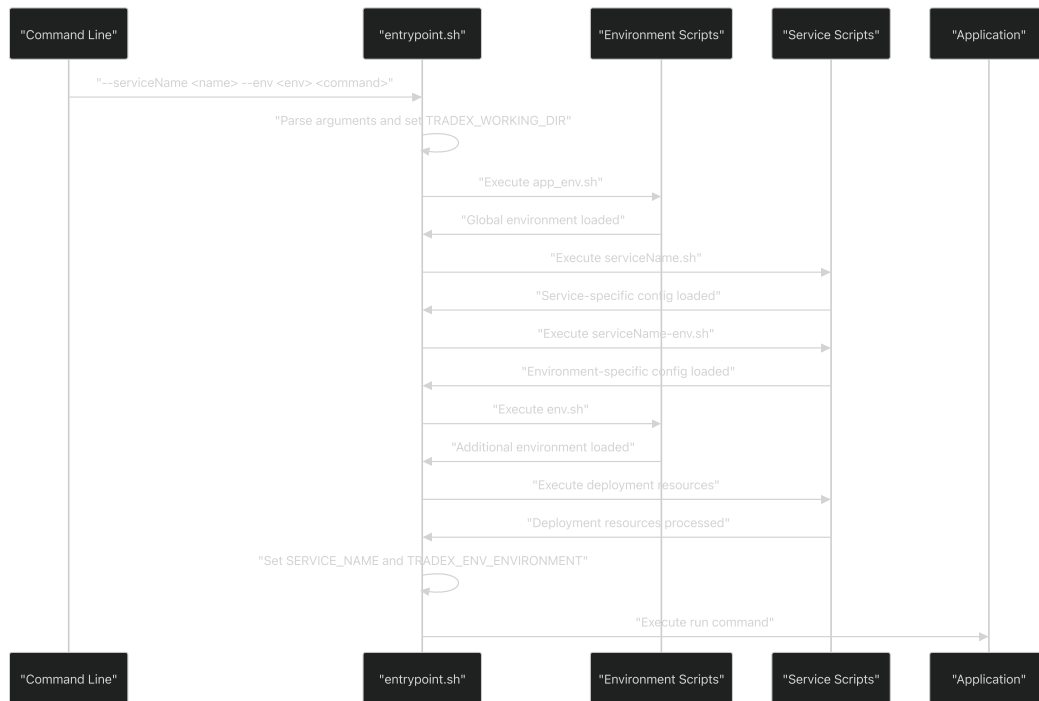
The service orchestration system provides a standardized approach to:

- Environment-specific service initialization
- Hierarchical configuration script loading
- Runtime environment variable management
- Service startup command execution
- Working directory and resource management

Orchestration Flow Overview

The orchestration process follows a structured sequence of configuration loading and environment setup, managed entirely through the `entrypoint.sh` script.

Service Startup Sequence



Sources: (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L1-L90>)

[entrypoint.sh1-90](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L1-L90) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L1-L90>)

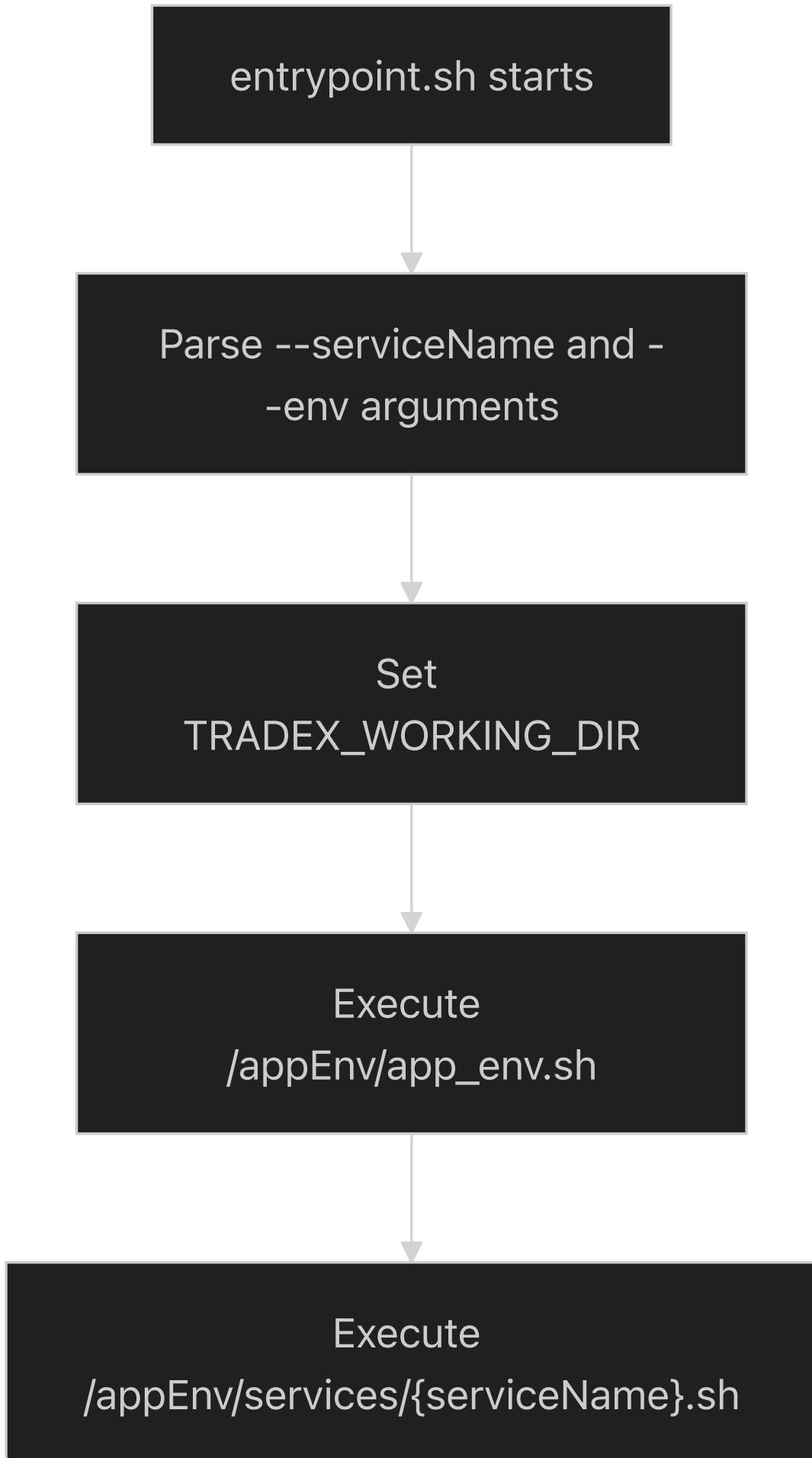
Configuration Script Hierarchy

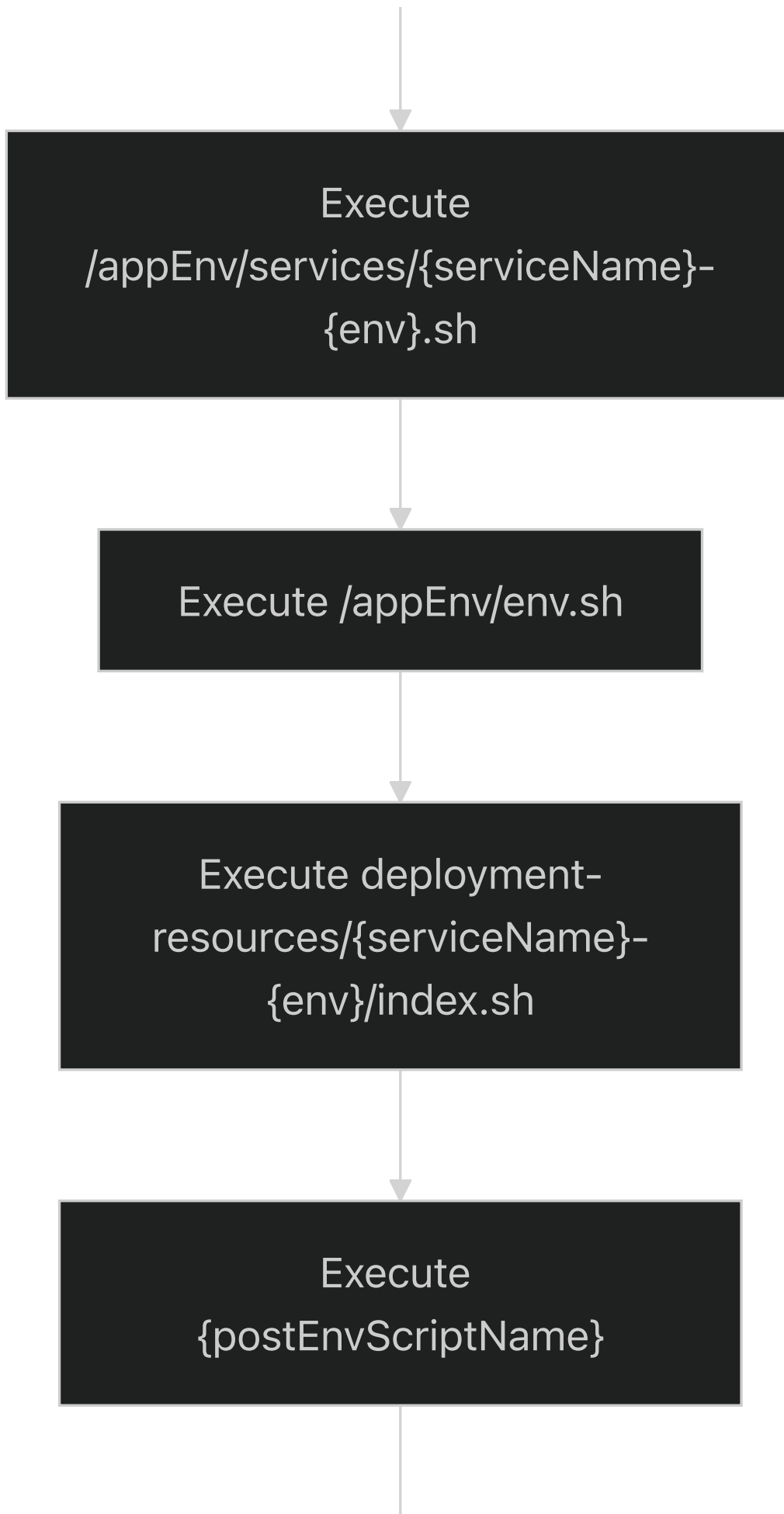
The orchestration system implements a layered configuration approach where scripts are executed in a specific order to allow for proper override behavior.

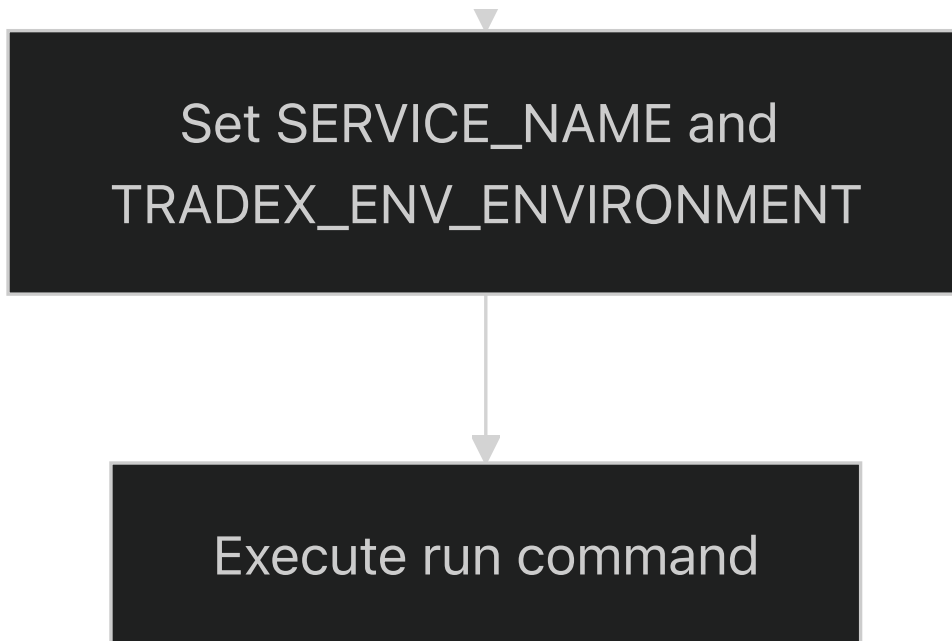
Configuration Loading Order

Order	Script Pattern	Purpose	Location
1	<code>app_env.sh</code>	Global environment settings	<code>/appEnv/app_env.sh</code>
2	<code><serviceName>.sh</code>	Service-specific configuration	<code>/appEnv/services/<serviceName>.sh</code>
3	<code><serviceName>--<env>.sh</code>	Environment-specific service config	<code>/appEnv/services/<serviceName>--<env>.sh</code>
4	<code>env.sh</code>	Additional environment overrides	<code>/appEnv/env.sh</code>
5	<code>index.sh</code>	Deployment resource scripts	<code>/appEnv/services/deployment-resources/<serviceName>--<env>/index.sh</code>
6	<code>\$postEnvScriptName</code>	Post-environment script	Working directory

Script Execution Flow







Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L37-L82>.

[entrypoint.sh37-82](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L37-L82) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L37-L82>

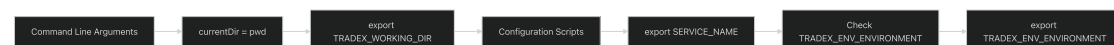
Environment Variable Management

The orchestration system manages several key environment variables that configure the application runtime.

Core Environment Variables

Variable	Source	Purpose
TRADEX_WORKING_DIR	entrypoint.sh	Application working directory
SERVICE_NAME	Command line argument	Service identifier
TRADEX_ENV_ENVIRONMENT	Command line or existing	Environment designation

Environment Variable Setting Process



Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L34-L87>.

[entrypoint.sh34-87](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L34-L87) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L34-L87>

Service Initialization Process

The orchestration handles both standard and specialized initialization patterns through argument parsing and conditional script execution.

Argument Processing

The system accepts command line arguments using a standardized pattern:

- `-s|--serviceName` : Specifies the service name for configuration selection
- `-e|--env` : Specifies the environment (development, staging, production)

- Remaining arguments: Preserved as the run command

Directory Navigation Pattern

The orchestration follows a specific directory navigation pattern to ensure proper script execution:

1. Navigate to `/appEnv` for global configuration
2. Navigate to `/appEnv/services` for service-specific configuration
3. Navigate to deployment resources directory if available
4. Return to working directory for final execution

Sources: [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L7-L30\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L7-L30)

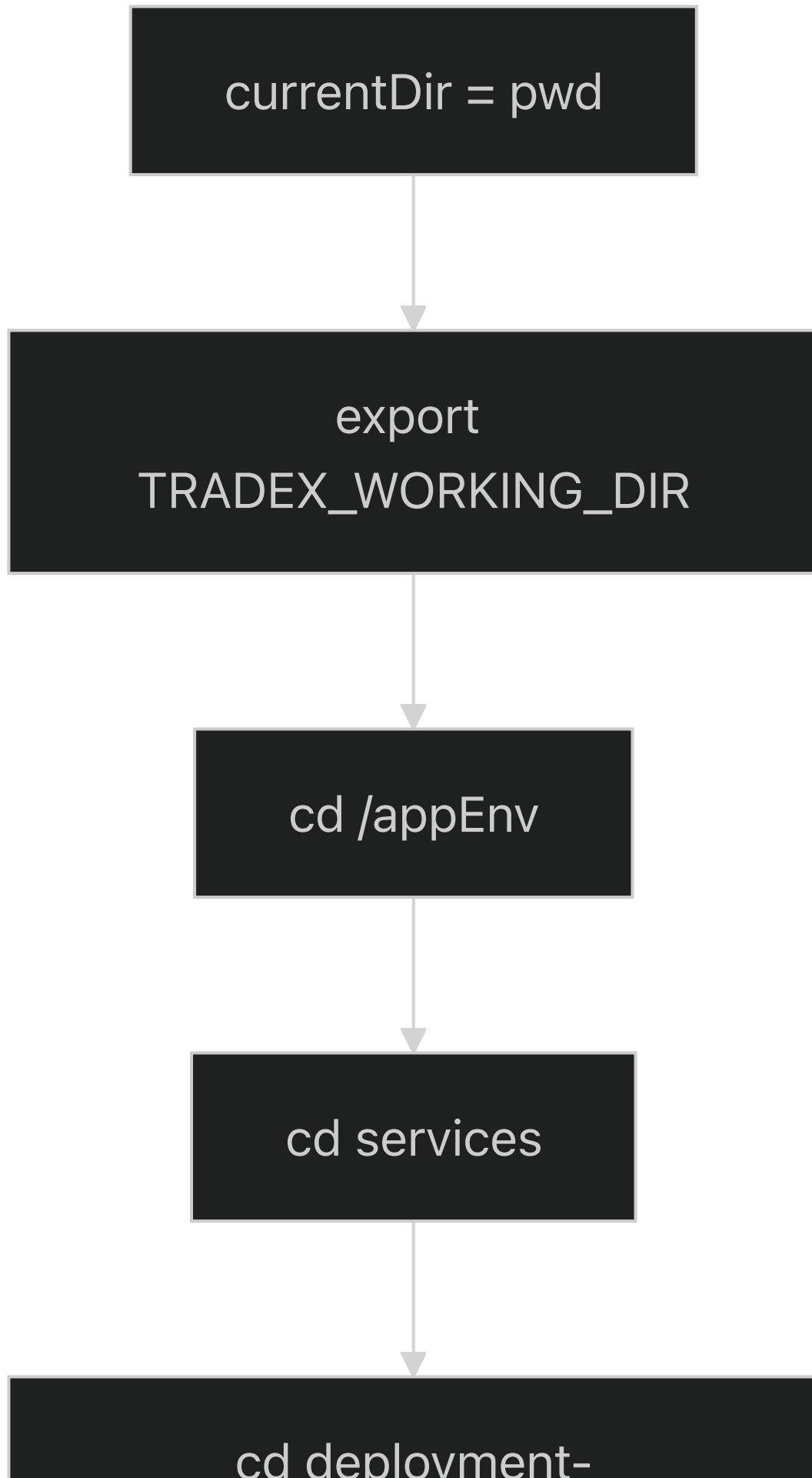
[entrypoint.sh7-30](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L7-L30) [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L7-L30\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L7-L30)[

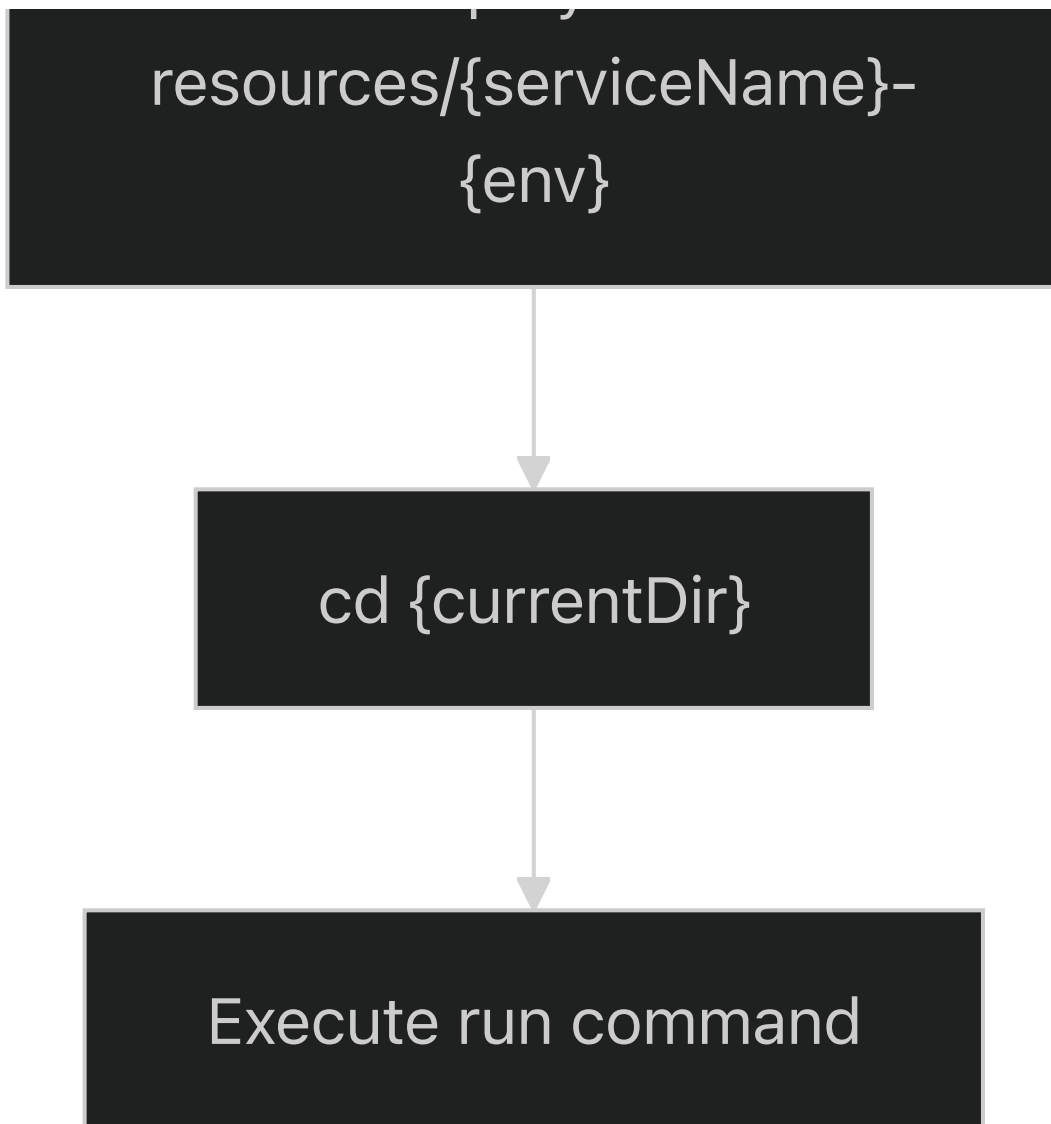
[entrypoint.sh37-76\]](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L37-L76) [_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L37-L76\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L37-L76)

[_\(https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L37-L76\)](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L37-L76))

Runtime Environment Setup

Working Directory Management





Script Permission Management

The orchestration ensures all scripts have execution permissions before running:

```
chmod -f +x ${globalEnvFileName}
chmod -f +x "$serviceName.sh"
chmod -f +x "$serviceName-$env.sh"
```

This pattern is applied consistently across all configuration scripts to prevent permission-related execution failures.

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L38-L38>.

[entrypoint.sh38](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L38-L38) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L38-L38>[(

[entrypoint.sh45\]](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L45-L45) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L45-L45>[(

[%5B](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L45-L45)).

[entrypoint.sh51\]](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L51-L51) <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L51-L51>

<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L51-L51>)).

Error Handling and Execution

The orchestration implements fail-fast behavior with `set -e` to ensure that any script failure terminates the entire initialization process. This prevents the application from starting with incomplete configuration.

The final execution uses `eval $runCommand` to execute the preserved command line arguments, allowing for complex command structures and argument passing.

Sources: <https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L32-L33>.

[entrypoint.sh#32-33](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L32-L33) (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L32-L33>)

[entrypoint.sh#89-90](https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L89-L90)] (<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L89-L90>
(<https://github.com/nhsv-tradex/viet-stock-bridge/blob/d4b51eda/entrypoint.sh#L89-L90>))
